# COMP11120 Mathematical Techniques for Computer Science
# Chapters 6–8

Andrea Schalk
A.Schalk@manchester.ac.uk

20th January 2025

**Abstract**

These notes cover much of the content for the unit; two chapters (on Logic and on Linear Algebra) are made available separately. The remaining notes are split into two documents, one for Semester 1 and one for Semester 2.

Please see the separate document on organization of the course unit for a description of the course mechanics as well as a list of text books that cover some of the material taught. This document is available in the *Course information* space on Blackboard.

# Contents


Contents 2

0 Basics 4
- 0.1 Numbers . . . 4
- 0.2 Sets . . . 18
- 0.3 Functions . . . 34
- 0.4 Relations . . . 47

1 Complex Numbers 50
- 1.1 Basic definitions . . . 50
- 1.2 Operations . . . 52
- 1.3 Properties . . . 59
- 1.4 Applications . . . 61

2 Statements and Proofs 62
- 2.1 Motivation . . . 62
- 2.2 Precision . . . 63
- 2.3 Properties of numbers . . . 74
- 2.4 Properties of sets and their operations . . . 79
- 2.5 Properties of binary operations . . . 80
- 2.6 Properties of functions . . . 91

3 Formal Logic Systems 114

4 Probability Theory 115
- 4.1 Analysing probability questions . . . 115
- 4.2 Axioms for probability . . . 133
- 4.3 Conditional probabilities and independence . . . 154
- 4.4 Random variables . . . 180
- 4.5 Averages for algorithms . . . 223
- 4.6 Some selected well-studied distributions . . . 231

5 Comparing sets and functions 237
- 5.1 Comparing functions . . . 237
- 5.2 Comparing sets . . . 243

6 Recursion and Induction 290
- 6.1 Lists . . . 291
- 6.2 Trees . . . 310
- 6.3 Syntax . . . 321

6.4 The natural numbers . . . 330
6.5 Further properties with inductive proofs . . . 349
6.6 More on induction . . . 350

**7 Relations** **352**
7.1 General relations . . . 353
7.2 Partial functions . . . 358
7.3 Binary relations . . . 363
7.4 Equivalence relations . . . 366
7.5 Partial orders . . . 415

**8 Applications to Computer Science** **441**

**Glossary** **446**

**Exercise Sheets** **451**

**Solutions to the Exercises** **521**

# Chapter 6

# Recursion and Induction

**Recursion** is a powerful principle. It can be used to

- define sets,
- define data types in some programming language such as Java, C, and Haskell,
- define operations on recursively defined sets,
- define operations on recursively defined datatypes and
- design recursive algorithms.

Recursion as a tool for defining data types and operations on these is covered in COMP11212 and as a tool in designing algorithms. Recursion goes hand in hand with a proof principle which is known as **induction**.

There is a general principle at work in all these.

- In order to recursively define an entity we need
  - a number of base cases—these create particularly simple instances of our set or data type and
  - a number of step cases—these create more complicated instances from simpler ones already available.
- In order to define an operation on such a recursively defined entity all one has to do is
  - define what the operation should do for each of the base cases and
  - define what the operation should do for each step case, assuming that it has already been defined for the simpler entities.
- In order to prove that a recursively defined entity or operation has a particular property all one has to do is
  - check each base case and
  - assuming that the property holds for simpler constructs (known as the *induction hypothesis*) show that it holds after applying each step case.

Some of you will have come across *natural induction* as a proof principle. This is a special case of induction which relies on the idea that the natural numbers can be constructed via recursion:

- the *base case* is given by the number 0 and
- the *step case* is given by the fact that every number $n$ has a unique successor, the number $n+1$.

We illustrate the general idea with a number of examples. We begin with the idea of recursive data types such as lists and trees. Many programming languages allow us to define recursive data types, typically using pointers or references. You will meet these ideas other course units, such as COMP164 and COMP260 and COMP261.

When computer scientists define syntax they often use recursive definitions. For example, regular expressions (Example 6.26) are defined recursively, and in any programming language the definition of what constitutes a valid program provides another example of a recursive definition (see Example 6.32), typically using something called a *grammar*, see Examples 6.29 and 6.30. These topics are taught in COMP11212, but the recursive nature of the definitions is not the main focus, so we briefly discuss that here.

We finally look at more mathematical examples, based on the natural numbers. A recurring theme on this course unit has been the fact that mathematics is all about rigour. There are a number of formal definitions in the notes in preceding chapters, while we have taken for granted certain facts about sets of numbers, and familiar operations on these. In this and the following chapter we give an idea of how these sets of numbers, and their operations, can be formally defined. This allows us to formally prove properties for these operations, such as commutativity or associativity, that we have taken for granted. This illustrates that rigour in mathematics is not something built on top of systems of numbers, but instead these can be formally defined, and we don't have to rely on 'common sense' or 'experience' to justify properties of their operations.

Note that for recursively defined structures as introduced below we have an explicit notion of *equality*: Two such structures are equal if and only if they have been created using exactly the same instances of base and step cases.

## 6.1 Lists

We begin our study of recursion by looking at lists. Lists are a standard recursive data type. The idea is quite simple: For a set $S$ we can have a list with entries from $S$, say

$$[s_n, s_{n-1}, \ldots, s_2, s_1].$$

For example, this might be a list of pages relevant to a particular topic, or a list of heights of people that somebody has measured, or a list of students on a particular course unit.

### 6.1.1 Lists defined

We can think of a list as satisfying exactly one of the following conditions:

- The list is empty, that is, there are no entries in the list.[1]
- The list has at least one entry, and we can think of it as being a created from a smaller list, say $l$, by adding an entry, say $s$.

[1]For example, if we have a program that calculates which entries should be added to the list we would like it to start from an empty list.

The list

$$[4, 3, 2, 1]$$

is the result of appending[2] the number

$$4 \qquad \text{to the list} \qquad [3, 2, 1],$$

which in turn is the result of appending the number

$$3 \qquad \text{to the list} \qquad [2, 1],$$

which in turn is the result of appending[3] the number

$$2 \qquad \text{to the list} \qquad [1],$$

which in turn is the result of appending the number

$$1 \qquad \text{to the list} \qquad [\,],$$

We can use this idea to give a formal definition of lists over a set $S$, and we can use the same idea to define operations on such lists.

**Definition 47: list**

A **list over a set $S$** is recursively defined as follows.

**Base case** list. There is an *empty list* $[\,]$.

**Step case** list. Given a list $l$ over $S$ and an element $s$ of $S$ there is a list $s : l$ where $s$ has been appended to $l$.

**Takeaway**

We collect elements from the set; the order is important, and elements can be added from the left only.

We use

$$\mathsf{Lists}_S$$

for the set of all lists over a given set $S$.

**Example 6.1.** What is written above regarding the list

$$[4, 3, 2, 1]$$

can now be written in the notation that is introduced in the formal definition of a list to read

$$\begin{aligned}
[4, 3, 2, 1] &= 4 : [3, 2, 1] \\
&= 4 : 3 : [2, 1] \\
&= 4 : 3 : 2 : [1] \\
&= 4 : 3 : 2 : 1 : [\,].
\end{aligned}$$

[2] We use terminology from the language Python for our lists. In MLthe *append* operation concatenates two lists, and the *cons* operation adds an entry to the list.

[3] We have made a somewhat arbitrary decision here to append entries to the left of the list. Instead we could have used $[1, 2, 3]$ to mean the list which arises from appending 1, 2 and 3 (in that order) to the empty list.

**Example 6.2.** Note that there is nothing in the definition of a list that stops us from adding the same item more than once, so it is perfectly fine to have the list

$$[3, 3, 3, 3],$$

and that this list is distinct from the lists

$$[3, 3, 3] \qquad \text{and} \qquad [3, 3, 3, 3, 3],$$

for example. Read on for a further discussion of the equality of lists.

Always having to deal with expressions like the one in the last row would be quite painful, but that is how a computer thinks of a list which is given by a list entry and a pointer to the remainder of the list. Human readability is improved, however, by using expressions like that in the top row.

These kinds of recursive definitions are very typical for functional programming languages such as Haskell or ML, but when programming with lists in C you will find a similar idea: An object of class List

- is empty or
- it consists of
  - an entry in the list and
  - a pointer to the remainder of the list (if there is one).

**Code Example 6.1.** Here's a class that implements this kind of list in Java, where the entries of the list are integers.

```
public class List {
    public int value;
    public List next;

    public List (int s, List l)
    {value = s; next = l;}
  }
```

We have to cheat slightly to deal with empty lists: We do this by using the value **null** to indicate that an empty list is being referenced.

So an object of the class List is

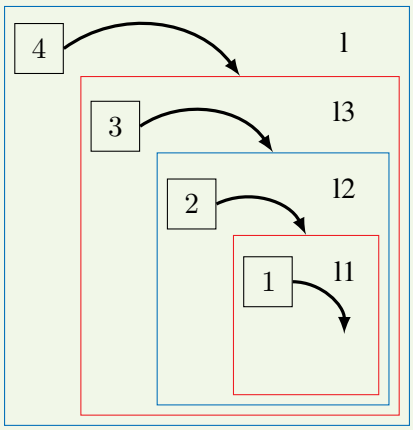
- the empty list if we get the value **null** when referring to it or
- it consists of an integer value and a List object next.

The fact that the class List is closely related to the lists defined in Definition 47 is not immediately obvious, but hopefully this explanation shows why they correspond to each other.

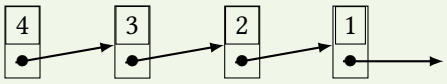
We give an idea of how to picture an object in this class. Assume that we have

- a List object l with l.value=4 and l.next=l3 and
- a List object l3 with l3.value=3 and l3.next=l2 and
- a List object l2 with l2.value=2 and l2.next=l1 and
- a List object l1 with l1.value=1 and l1.next=**null**.

You can picture these objects as follows:




Another picture that is sometimes used in this situation is the following.




Two lists are **equal** if and only if they have been created using exactly the same instances of base and step case. Since for lists there is only one base case, all lists are created using the same instance of the base case, so what is relevant here is that the instances of the step cases that follow are identical.

Note that 'using identical instances of the step case' means informally that lists contain the same entries added in the same order. In particular, the lists $[1, 2]$ and $[2, 1]$ are distinct.

### 6.1.2 Recursive definitions and proof by induction

Operations on such recursive datatypes are usually defined recursively. We give a number of examples for this particular construct.

**Example 6.3** (The sum operation). Assume that $N$ is a set of numbers $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ or $\mathbb{C}$. We want to define a function

$$\mathrm{sum}\colon \mathsf{Lists}_N \longrightarrow N$$

which adds up all the members of a given list. In other words we would like to have

$$\mathrm{sum}[4, 3, 2, 1] = 10.$$

**Tip**

Before defining an operation that is described verbally it can be a good idea to think about what that operation should do to a concrete example to make sure that you understand the instructions given.

**Example 6.4** (The sum operation ctd). Continuing this example how do we define such an operation recursively? We have to ask ourselves: What do we want to happen in the base case? So what should it mean to add up all the numbers that appear in the empty list? The answer is that we should get the unit[4] for addition, 0.

We then have to think about what we want to happen in the step case, so *knowing the result of* $\operatorname{sum} l$, what should the result of $\operatorname{sum}(s : l)$ be? It can sometimes help to write it like that, namely

$$\operatorname{sum}(s : l) = ??? \ \operatorname{sum} l.$$

Well, if we already know what happens if we add up all the numbers of the list $l$, then to add up all the numbers in the list $s : l$, we just have to add $s$ to the result. These considerations lead to the following definition.

**Base case** sum. $\operatorname{sum}[\,] = 0.$

**Step case** sum. $\operatorname{sum}(s : l) = s + \operatorname{sum} l.$

Note how this definition very closely follows the shape of the definition of a list. This is one of the hallmarks of definition by recursion. If you understand the definition of the underlying entity you know what shape an operation for the type will have.

The way this function works is to map

- the empty list to the number 0,
- and a non-empty list, which has an entry $s$ added to some list $l$, to the result of adding $s$ to the result applying that function to the remainder of the list $l$.

In general, defining a new operation for lists always takes the following shape:

**Base case** We have to say what the operation should do when given the base case of a list, that is, $[\,]$.

**Step case**. Assuming that we already know what the operation does for the list $l$ we define what it does when the argument is $s : l$.

**Tip**

When you have defined an operation recursively it is always a good idea to apply it to an example to make sure that your definition makes sense. We do this for the operation just defined.

[4] If you were wondering how to deal with the empty list then note that adding zero numbers is usually taken to describe the number 0. More generally, for an operation with unit $e$, applying the operation to 0 many elements should return $e$.

**Example 6.5** (The sum operation ctd). A sample calculation (this time without using our notation for lists to stay closer to the programming example below) looks as follows:

$$
\begin{aligned}
&\mathrm{sum}(4 : (3 : (2 : (1 : [\,]))))\\
&= 4 + \mathrm{sum}(3 : (2 : (1 : [\,])))\qquad && \text{step case sum}\\
&= 4 + 3 + \mathrm{sum}(2 : (1 : [\,]))\qquad && \text{step case sum}\\
&= 4 + 3 + 2 + \mathrm{sum}(1 : [\,])\qquad && \text{step case sum}\\
&= 4 + 3 + 2 + 1 + \mathrm{sum}\,[\,]\qquad && \text{step case sum}\\
&= 4 + 3 + 2 + 1 + 0.
\end{aligned}
$$

See Example 6.12 for an inductive proof of a property of this function.

**Code Example 6.2.** For our class List this method could be implemented by the following method add.

```
public static int sum (List l)
{
if (l == null)
    return 0;
else
    return l.value + sum(l.next);
}
```

We can work out the function calls for the calculation for the lists l, l3, l2, l1 as in Code Code Example 6.1, as follows:

```
sum(l)
  sum(l3)
    sum(l2)
      sum(l1)
        sum(null)
        return 0
      return 1 + 0
    return 2 + 1
  return 3 + 3
return 4 + 6
```

You can see why the computer has to create a stack which contains all the information needed to know what to do with the various return values and where to continue the computation.

Note the similarities between the mathematical function definition and the code implementing it.

**Base case** $\operatorname{sum}[\,] = 0.$

**Step case** $\operatorname{sum}(s : l) = s + \operatorname{sum} l.$

```
public static int sum (List l)
{
if (l == null)
    return 0;
else
    return l.value + sum(l.next);
}
```

The main difference is that in the mathematical definition, we assume that we already know whether a given list matches the base or the step case, whereas in the computer program we have to test which it is. As a consequence, in the step case of the mathematical definition we already know that the given list is built from the list $l$ by appending the entry $s$, whereas in the programming example we have an argument l which is known not to be in the base case and we have to find the integer and the list l is built from by looking at l.value and l.next.

**Example 6.6.** We define a second operation on lists.

**Base case** $\mathrm{m}_0$. $\mathrm{m}_0\,[\,] = [\,].$

**Step case** $\mathrm{m}_0$. $\mathrm{m}_0(s : l) = 0 : \mathrm{m}_0\, l.$

What does this operation do? We have not typed it, but we can see that its input might be an element of $\mathsf{Lists}_S$ for any set $S$, and that its output is another list, and that this list potentially contains the entry 0. We might, for example, type this assignment as

$$\mathrm{m}_0 \colon \mathsf{Lists}_S \longrightarrow \mathsf{Lists}_{\mathbb{N}}$$

for an arbitrary set $S$.

We may further see that in the base case, the empty list is mapped to the empty list, while in the step case, a list whose most recently added entry is $s$ is mapped to a list whose most recently added entry is 0. Since we then apply the function to the remainder of the list we may see that one by one, all the entries of the list are replaced by 0.

We are ready for a first proof by *induction*. We want to show that for all lists $l \in \mathsf{Lists}_S$ it is the case that

$$\operatorname{sum} \mathrm{m}_0\, l = 0.$$

Such a proof also follows the formal definition of the underlying data type. The pattern consists of a proof for the base case, a proof of the step case, and (optionally here) in between the statement of the *induction hypothesis*.

**Base case** list. We have to show that the statement holds for the empty list, that is $\operatorname{sum} \mathrm{m}_0\,[\,] = 0.$

**Induction hypothesis.** We assume the statement holds for the list[5] $l$, that is we have $\operatorname{sum} \mathrm{m}_0\, l = 0.$

[5]Note that sometimes we have to assume that the statement holds for all lists of a given length, or some other statement applying to more than one list.

**Step case** list. We have to show that given the induction hypothesis the statement holds for lists of the form $s : l$, that is we have

$$\operatorname{sum} \mathrm{m}_0(s : l) = 0.$$

**Example 6.7.** We show how to prove the statement given above.

**Base case** list. $\operatorname{sum} \mathrm{m}_0 [\,] = \operatorname{sum} [\,] = 0$. This is a simple application of the base cases of the definitions of the operations $\mathrm{m}_0$ and $\operatorname{sum}$.

**Induction hypothesis**. For the list $l$ we have

$$\operatorname{sum} \mathrm{m}_0\, l = 0.$$

**Step case** list. We check that

$$\begin{aligned} \operatorname{sum} \mathrm{m}_0(s : l) &= \operatorname{sum}(0 : \mathrm{m}_0\, l) && \text{step case } \mathrm{m}_0 \\ &= 0 + \operatorname{sum} \mathrm{m}_0\, l && \text{step case } \operatorname{sum} \\ &= 0 + 0 && \text{ind hyp} \\ &= 0. && \text{arithmetic} \end{aligned}$$

As is typical the base case is obvious, and it typically does not require many steps, while the step case has a little more substance.

Note that we were able to define this function without having to specify from which set $S$ our lists take their entries.

Let's pause a moment to think about how this proof works. The base case is fairly easy to understand.

It seems as if in the middle of the proof, where the induction hypothesis is stated, we are assuming the very same thing we aim to prove.

This is not so, however. An analogy that is often invoked is that of a line of dominoes. Assume that you have got a line of dominoes, standing on their short side, starting in front of you, and extending to the right (you may imagine infinitely many dominoes).

**Base case**. The first domino falls over, to the right.

**Induction hypothesis** The $n$th domino falls over to the right.

**Step case**. If the $n$th domino falls over to the right then the $(n+1)$th domino falls over to the right.

In this way of writing the proof the induction hypothesis is often skipped because it appears in the step case (*'If the $n$th domino falls over ...'*). If we do not state it explicitly we do not lose any information. After the first few examples I do not note the induction hypothesis explicitly if it is a precise copy of the statement we are proving; See Section 6.4 for examples where more sophisticated induction hypotheses appear.

The base and the step case together are sufficient to guarantee that if the first domino falls over to the right (the base case) then *all* dominoes fall over.

Another analogy you may find helpful is that of climbing a ladder.

**Base case**. I can climb the first rung of the ladder.

**Step case**. If I can get to the $n$th rung of the ladder then I can climb to the $(n+1)$th one.

If we have both these properties, then we can get onto the first, and all subsequent, rungs of the ladder.

Going back to the case of lists, the base case ensures that we know the desired result for the empty list, and the step case ensures that if we know the result for the list $l$ then we can prove it for the list where some entry $s$ had been added to $l$ to form $s : l$.

**Example 6.8**. We look at the proof that $\mathrm{sum}\,\mathrm{m}_0\,l = 0$ for a specific list $l$. Let

$$l = [s_3, s_2, s_1] = s_3 : (s_2 : (s_1 : [\,])).$$

We can see that in order to show that

$$\mathrm{sum}\,\mathrm{m}_0(s_3 : (s_2 : (s_1 : [\,]))) = 0$$

we need the step cases of $\mathrm{m}_0$ and $\mathrm{sum}$ three times:

$$\begin{aligned}
&\mathrm{sum}\,\mathrm{m}_0(s_3 : (s_2 : (s_1 : [\,]))) \\
&\quad = \mathrm{sum}(0 : \mathrm{m}_0(s_2 : (s_1 : [\,]))) && \text{step case } \mathrm{m}_0 \\
&\quad = 0 + \mathrm{sum}\,\mathrm{m}_0(s_2 : (s_1 : [\,])) && \text{step case } \mathrm{sum} \\
&\quad = 0 + \mathrm{sum}(0 : \mathrm{m}_0(s_1 : [\,])) && \text{step case } \mathrm{m}_0 \\
&\quad = 0 + 0 + \mathrm{sum}\,\mathrm{m}_0(s_1 : [\,]) && \text{step case } \mathrm{sum} \\
&\quad = 0 + 0 + \mathrm{sum}(0 : \mathrm{m}_0\,[\,])[\,] && \text{step case } \mathrm{m}_0 \\
&\quad = 0 + 0 + 0 + \mathrm{sum}\,\mathrm{m}_0\,[\,][\,] && \text{step case } \mathrm{sum} \\
&\quad = 0 + 0 + 0 + \mathrm{sum}\,[\,] && \text{base case } \mathrm{m}_0 \\
&\quad = 0 + 0 + 0 + 0 && \text{base case } \mathrm{sum}
\end{aligned}$$

The induction proof says that

- if we know the property for the base case and
- if we know that if the property holds for the list $l$, then via the step case it holds for every list of the form $s : l$

we know it holds for all lists. This example illustrates how for a concrete example we may keep applying the step case until we reach the base case. If we had allowed lists of infinite length (but these cannot be built with our method of recursion) then this principle would not work.

We can also see that if we have a list with five entries then we need the step case five times. A proof by induction takes a shortcut: by proving the base case, and that given a list with $n$ entries which has the desired property we can show that adding another entry to the list results in a list that also has the property in question, we can establish the property for all finite lists.

Note that sometimes in order to establish the step case (here $s : l$) we have to assume that the induction hypothesis for *all* entities which are somehow smaller than the current one. In the case of lists it might be that we assume the induction hypothesis for all lists we have built

on the way to reach the list $l$, or possibly even for all lists which have at most as many entries as $l$.

A proof by induction for lists always takes the following shape:

**Base case** list. The property is established for the base case for lists, that is: We show it works for $[\,]$. If the property we want to show is an equality then we get the statement we want to prove by inserting the empty list $[\,]$ for every occurrence of $l$ in the equality, so for example

$$\mathrm{sum}\,\mathrm{m}_0\,l = 0$$

becomes

$$\mathrm{sum}\,\mathrm{m}_0\,[\,] = 0.,$$

and[6]

$$\mathrm{sum}\,\mathrm{rev}\,l = \mathrm{sum}\,l$$

becomes

$$\mathrm{sum}\,\mathrm{rev}\,[\,] = \mathrm{sum}\,[\,].$$

**Ind hypothesis**. We assume that the statement holds for the list[7] $l$.

**Step case** list. Assuming the induction hypothesis we show that the property holds for the list $s : l$. We obtain the statement we have to show by replacing every occurrence of $l$ by $s : l$, , so

$$\mathrm{sum}\,\mathrm{m}_0\,l = 0$$

becomes

$$\mathrm{sum}\,\mathrm{m}_0(s : l) = [\,],$$

and

$$\mathrm{sum}\,\mathrm{rev}\,l = \mathrm{sum}\,l$$

becomes

$$\mathrm{sum}\,\mathrm{rev}(s : l) = \mathrm{sum}(s : l).$$

We summarize these ideas as follows.

**Tip**

A proof by induction for lists over a given set $S$ always has the following shape. Assume we are trying to show a statement given for all $l \in \mathsf{Lists}_S$.

**Base case** list. Prove the given statement for the case where all occurrences of $l$ have been replaced by $[\,]$.

**Ind hyp** Assume the given statement holds for the list $l$.[8]

**Step case** list. Prove the statement where all occurrences of $l$ have been replaced by $s : l$, where $s$ is an arbitrary element of $S$. The induction hypothesis is used as part of the proof.

Note that when proving something by induction for a recursively defined structure other than the natural numbers, people often speak of *structural induction*, because the structure of the data type gives the shape of the induction argument.

[6] Read on to find out how the operations used here are defined, but that is not required for understanding the point that is made here.

[7] Note that for advanced properties one may have to extend this, and, for example, assume the statement for all lists built on the way to reaching $l$, or that it holds for all lists that have at most the same number of entries as $l$, but on this course we have no examples like that.

[8] It may be necessary to assume it for $l$ and all sublists of $l$.

### 6.1.3 Operations on lists

We define an operation

$$\mathsf{Lists}_S \times \mathsf{Lists}_S \longrightarrow \mathsf{Lists}_S$$

for an example that takes two lists as input, raising an important question discussed below.

**Example 6.9.** We want to define an operation that takes as input two lists, and returns one list where the two have been stuck together, that is, *concatenated*, so

$$[1, 2, 3] +\!\!+ [4, 3, 2, 1] = [1, 2, 3, 4, 3, 2, 1].$$

We would like to have a formal definition of this operation. This formal definition can then be turned into code that implements this operation.

Now that we have two arguments we need to think about how to work that into the definition. For simple operations such as this one it is possible to consider one argument as a *parameter* and give a recursive definition in terms of the other argument. In this case we can leave the right argument alone an only have to look inside the left one, which is why we recurse over the *left* argument.

We define the operation

$$+\!\!+ \colon \mathsf{Lists}_S \times \mathsf{Lists}_S \longrightarrow \mathsf{Lists}_S$$

by setting

**Base case** $+\!\!+$. $\quad [\,] +\!\!+ l' = l'$ and

**Step case** $+\!\!+$. $\quad (s : l) +\!\!+ l' = s : (l +\!\!+ l')$,

where it is understood that $l$ and $l'$ are elements of $\mathsf{Lists}_S$.

Note that the only way to define this is by recursion over the left argument. If we try to do it the other way round we have no way of defining the list we want from

$$l +\!\!+ (s : l').$$

In EExercise 119 we look at a way of doing this recursing over the right argument, but that requires another operation and is considerably more complicated.

**Example 6.10.** We continue Example 6.9 by illustrating how this operation works in practice. We show how two lists are concatenated step by step by following the process defined by this definition.

$$\begin{aligned}
[4, 3] +\!\!+ [2, 1] &= (4 : [3]) +\!\!+ [2, 1] && \text{step case list} \\
&= 4 : ([3] +\!\!+ [2, 1]) && \text{step case } +\!\!+ \\
&= 4 : ((3 : [\,]) +\!\!+ [2, 1]) && \text{step case list} \\
&= 4 : 3 : ([\,] +\!\!+ [2, 1]) && \text{step case } +\!\!+ \\
&= 4 : 3 : [2, 1] && \text{base case } +\!\!+ \\
&= 4 : [3, 2, 1] \\
&= [4, 3, 2, 1].
\end{aligned}$$

**Code Example 6.3.** We give the code that corresponds to the mathematical definition.

```
public static List concat (List l, List l2)
{
if (l == null)
    return l2;
else
    return new List (l.value, concat(l.next,l2));
}
```

The code also illustrates why we have to use recursion over the left argument (try writing code which does it over the right argument if you can't see why—you just don't have access to the relevant part of the list).

From the point of view of programming we usually stop at defining operations, but when writing a compiler we have to worry about the properties of such operations.

We illustrate here how the proof principle of induction allows us to establish properties for operations that have been defined using the principle of recursion.

Such proofs once again follow the shape of the original definition of the operation, which in turn follows the shape of the definition of the original entity.

**Example 6.11.** We show that the empty list works like a unit on the right, that for all lists $l$ over a set $S$, and all $s \in S$, we have

$$l +\!\!+ [\,] = l.$$

We follow the shape of the definition of the concatenation operation, by first considering the base case.

**Base case** list. We note that

$$[\,] +\!\!+ [\,] = [\,] \qquad\qquad \text{base case } +\!\!+ .$$

**ind hyp** We assume that the statement holds for the list $l$, that is

$$l +\!\!+ [\,] = l.$$

We turn to the step case.

**Step case** list. We calculate

$$\begin{aligned} (s : l) +\!\!+ [\,] &= s : (l +\!\!+ [\,]) && \text{step case} +\!\!+ \\ &= s : l && \text{induction hypothesis.} \end{aligned}$$

Taken together these two cases cover all possibilities, and they form a proof by *induction*.

Note that we have immediately from the base case of the definition of $+\!\!+$ that $[\,]$ acts like a unit on the left, so overall we can see that it is the unit for this operation.

Below on we *do not write the induction hypothesis explicitly* if it consists of a statement that is identical to the one we are proving. In other words in examples of induction proofs which do

not have an induction hypothesis spelled out you may assume that it looks like the statement we are proving.

**CExercise 116.** Answer the following questions. Give an argument for your answer, either by providing a counter example or by giving a proof by induction. *Hint: If you want to see more examples for proofs by induction then read on to Examples 6.12, 6.16 and 6.17.*

*Compare this question with the one about concatenating strings, as part of CExercises 27 and 28 and eExercise 29.*

*Do not forget to justify each step of your proofs.*

(a) Is the $+\!\!+$ operation commutative?

(b) Is the $+\!\!+$ operation associative?

(c) Does the $+\!\!+$ operation have a unit? If yes, what is it? Justify your answer.

**Example 6.12.** The operations sum (see Examples 6.3 and 6.4) and $+\!\!+$ (see Example 6.9) are defined above. We show by induction[9] that for lists $l$ and $l'$ in $\mathsf{Lists}_N$ it is the case that

$$\operatorname{sum}(l +\!\!+ l') = \operatorname{sum} l + \operatorname{sum} l'.$$

**Base case** list.

$$\begin{aligned}
\operatorname{sum}([\,] +\!\!+ l') &= \operatorname{sum} l' && \text{base case } +\!\!+ \\
&= 0 + \operatorname{sum} l' && 0 \text{ unit for } + \\
&= \operatorname{sum}[\,] + \operatorname{sum} l' && \text{base case } \operatorname{sum}.
\end{aligned}$$

**Step case** list.

$$\begin{aligned}
&\operatorname{sum}((s : l) +\!\!+ l') \\
&\quad = \operatorname{sum}(s : (l +\!\!+ l')) && \text{step case } +\!\!+ \\
&\quad = s + \operatorname{sum}(l +\!\!+ l') && \text{step case } \operatorname{sum} \\
&\quad = s + (\operatorname{sum} l + \operatorname{sum} l') && \text{ind hyp} \\
&\quad = (s + \operatorname{sum} l) + \operatorname{sum} l' && + \text{ associative} \\
&\quad = \operatorname{sum}(s : l) + \operatorname{sum} l' && \text{step case } \operatorname{sum}.
\end{aligned}$$

**Tip**

Defining a function recursively can be quite tricky when you are not used to it. To get started you might as well make use of the fact that you know what such a definition needs to look like. Assume we intend to define a function

$$f : \mathsf{Lists}_{\mathbb{N}} \to S$$

that behaves in a particular way. Then we know we need to have two cases:

**Base case** $f$. $\quad f[\,] = ?$

**Step case** $f$. $\quad f(s : l) = ???\ fl.$

[9] As previously discussed we do not state the induction hypothesis explicitly since its statement is identical to the statement we are proving.

It should be easy to read off from your description of $f$ how to define it in the base case. For the step case you may require a bit of creativity. How does knowing $fl$ help us to calculate $f(s : l)$? Note that you are allowed to use operations from the target set of $f$, here $S$. If you can answer that question you should be able to give a correct definition. Then check your definition using an example.

Another useful operation is determining the length of a list. You are asked to define this operator yourself in the following exercise, but assume for the remainder of this section that there is a function

$$\mathrm{len}\colon \mathsf{Lists}_S \longrightarrow \mathbb{N}$$

which, given a list, returns the number of entries in the list.

**CExercise 117.** This exercise concerns the length function described in the preceding paragraph.

(a) Give a definition of this len function.

(b) Use your definition to calculate $\mathrm{len}[3, 2, 1]$ step by step.

(c) Give the code for the corresponding function for objects of type List.

(d) Show that for all lists $l$ and $l'$ over a set $S$ we have

$$\mathrm{len}(l +\!\!+ l') = \mathrm{len}\, l + \mathrm{len}\, l'.$$

Justify each step.

**Exercise 118.** For lists in $\mathsf{Lists}_{\mathbb{N}}$, give a recursive definition of a list being ordered,[10] that is, for example for the list

$$[k, l, m, n]$$

we demand $k \geq l \geq m \geq n$. *Hint: You want two base cases.*

We define additional operations for lists.

**Example 6.13.** We can also reverse a list, that is turn it back to front.

**Base case** rev. $\quad \mathrm{rev}\,[\,] = [\,]$

**Step case** rev. $\quad \mathrm{rev}(s : l) = \mathrm{rev}\, l +\!\!+ [s]$.

You are asked to work out how this definition works in the following exercise.

**Code Example 6.4.** We give the code that corresponds to the mathematical definition, using the concat method defined in Code Code Example 6.3.

```
public static List reverse (List l)
{
if (l == null)
    return l;
```

[10] We have chosen here to have the entries to get larger as they are added to the list—the opposite choice would also make sense.

```
else
    return concat(reverse(l.next), new List (l.value, null));
}
```

**EExercise 119.** Carry out the following for the $\mathrm{rev}$ operator.

(a) Calculate $\mathrm{rev}[1, 0]$ step by step.

(b) Show that for every list $l$ over an arbitrary set $\mathrm{len}\,\mathrm{rev}\,l = \mathrm{len}\,l$. *Hint: You may want to use a statement from a previous assessed exercise to help with this.*

(c) Show that $\mathrm{rev}(l +\!\!+ l') = \mathrm{rev}\,l' +\!\!+ \mathrm{rev}\,l$.

(d) Show that $\mathrm{rev}\,\mathrm{rev}\,l = l$ for all lists $l$ over $S$. *Hint: You may want to show* $\mathrm{rev}[s] = [s]$ *separately.*

(e) Use the $\mathrm{rev}$ operator to give an alternative definition of the concatenation operator. We are looking for a recursively defined operation $\oplus$ which recurses over *second* argument, and which does not use $+\!\!+$ at all. *Hint: Start by thinking about what the base and step case look like when recursing over the second argument. Then consider how you might use the* $\mathrm{rev}$ *operator to help you with the fact that you in the step case you are given an entry that will need to appear in the middle of the list being constructed.*

Give an argument that your definition agrees with the original. *Hint: You want to do this by induction over the* second *argument.*

*Note that finding a suitable definition and proving it has the required property is a hard problem—don't worry if you can't find a solution.*

Justify each step in your proofs.

**Example 6.14.** We give one more example where we look at code. When we have two lists it seems easy to decide whether they are 'the same'. By that we mean they consist of the same entries in the same order. A human being can check this for two elements of the set $\mathsf{Lists}_S$ by inspection, and below we give a recursive procedure that carries out this check.

When we have two List objects, however, our idea of having 'the same list' may not agree with that of the Java programming language. The boolean value

```
(l1 == l2)
```

evaluates to true precisely when l1 and l2 refer to the same object. If we want to check whether two such objects consist of the same entries in the same order we have to write code of the following kind.

```
public static boolean equal (List l1, List l2)
{
 if (l1 == null)
     return (l2 == null);
 else {
```

```
        if (l2 == null)
            return false;
        else
            return (l1.value == l2.value && equal(l1.next, l2.next));
    }
}
```

In our heads we are effectively defining a function that takes a List object and turns it into an element of $\mathsf{Lists}_{\mathbb{Z}}$, and we consider two lists equal if this function maps them to the same list.

The code above can be turned into a mathematical definition quite easily. We want to define a function, say

$$f_{=} \colon \mathsf{Lists}_S \times \mathsf{Lists}_S \longrightarrow \{0,1\} \ .$$

**Base case** $f_{=} : [\,], [\,]$. $\quad f_{=}([\,],[\,]) = 1$.

**Base case** $f_{=} : [\,], s' : l'$. $\quad f_{=}([\,], s' : l') = 0$.

**Base case** $f_{=} : s : l, [\,]$. $\quad f(s : l, [\,]) = 0$.

**Step case** $f_{=}$.

$$f_{=}(s : l, s' : l') = \begin{cases} f_{=}(l, l') & s = s' \\ 0 & \text{else.} \end{cases}$$

where it is understood that $l$ and $l'$ are elements of $\mathsf{Lists}_S$.

Why is the mathematical definition so much longer? Because in the code we make use of the equality function for integers and references.

Assume we have two functions

$$\begin{aligned} f_{=[\,]} \colon \mathsf{Lists}_S &\longrightarrow \{0,1\} \\ l &\longmapsto \begin{cases} 1 & l = [\,] \\ 0 & \text{else} \end{cases} \end{aligned}$$

and

$$\begin{aligned} f_{=_S} \colon S \times S &\longrightarrow \{0,1\} \\ (s, s') &\longmapsto \begin{cases} 1 & s = s' \\ 0 & \text{else.} \end{cases} \end{aligned}$$

then we can define instead

**Base case** $f_{=}$ $\quad f_{=}([\,], l') = f_{=[\,]} l'$.

**Step case** $f_{=} : [\,]$. $\quad f_{=}(s : l, [\,]) = 0$.

**Step case** $f_{=} : s' : l'$. $\quad f_{=}(s : l, s' : l') = f_{=_S}(s, s') \wedge f_{=}(l, l')$,

which is very close to the code given above.

**Exercise 120.** The aim of this exercise is to define a function

$$\begin{aligned} \text{search}\colon \mathsf{Lists}_S \times S &\longrightarrow \{0,1\} \\ (l,s) &\longmapsto \begin{cases} 1 & s \text{ occurs in } l \\ 0 & \text{else.} \end{cases} \end{aligned}$$

(a) Recursively define this function.

(b) If your definition is a definition by cases then change that, using the $f_{=_S}$ function from Example 6.14. *Hint: You may use boolean operations on the set* $\{0,1\}$.

(c) Write code that implements this function for objects of the class List.

**Example 6.15.** If we have a list $l$ over a set $S$, and a way of translating elements of $S$ to elements of some set $T$ via a function $f\colon S \to T$ we may transform our list to one over the set $T$ by using the function $\operatorname{map} f$ defined below. The type of this function is

$$\operatorname{map} f\colon \mathsf{Lists}_S \longrightarrow \mathsf{Lists}_T.$$

It is given by the following recursive definition.

**Base case** map. $\quad (\operatorname{map} f)[\,] = [\,]$

**Step case** map. $\quad (\operatorname{map} f)(s : l) = fs : (\operatorname{map} f)l.$

Assume we have the function $f\colon \mathbb{N} \to \mathbb{Z}$ given by

$$n \longmapsto -n.$$

Applying the map function to $f$ and the list $[3,2,1]$ we get

$$\begin{aligned} (\operatorname{map} f)[3,2,1] &= (\operatorname{map} f)(3 : [2,1]) && \text{Step case list} \\ &= f3 : (\operatorname{map} f)[2,1] && \text{step case map} \\ &= -3 : (\operatorname{map} f)(2 : [1]) && \text{def } f \text{ and step case list} \\ &= -3 : f2 : (\operatorname{map} f)[1] && \text{step case map} \\ &= -3 : -2 : (\operatorname{map} f)(1 : [\,]) && \text{def } f \text{ and step case list} \\ &= -3 : -2 : (f1 : \operatorname{map} f[\,]) && \text{step case map} \\ &= -3 : -2 : (-1 : [\,]) && \text{def } f\text{, base case map} \\ &= -3 : -2 : [-1] \\ &= -3 : [-2,-1] \\ &= [-3,-2,-1]. \end{aligned}$$

We use the map operator to give additional examples for proofs by induction.

**Example 6.16.** We show that for the identity function $\mathsf{id}_S$ on a set $S$ we have for all lists $l$ over the set $S$ that

$$(\operatorname{map} \mathsf{id}_S)l = l.$$

This is a proof by induction.

**Base case** list. We have $(\mathrm{map}\,\mathsf{id}_S)[\,] = [\,]$ by base case map.

**Step case** list. We note the following.[11]

$$\begin{aligned}(\mathrm{map}\,\mathsf{id}_S)(s : l) &= \mathsf{id}_S s : (\mathrm{map}\,\mathsf{id}_S)l && \text{step case map}\\ &= s : l && \mathsf{id}_S s = s;\ \text{ind hyp.}\end{aligned}$$

**Example 6.17.** We show another property for the map operator. Assume we have two functions $f : S \to T$ and $g : T \to U$ we have for all lists $l$ over $S$ that

$$(\mathrm{map}(g \circ f))l = (\mathrm{map}\,g)((\mathrm{map}\,f)l).$$

We carry out the proof by induction.

**Base case** list.

$$\begin{aligned}(\mathrm{map}(g \circ f))[\,] &= [\,] && \text{base case map}\\ &= (\mathrm{map}\,g)[\,] && \text{base case map}\\ &= (\mathrm{map}\,g)((\mathrm{map}\,f[\,])) && \text{base case map}.\end{aligned}$$

**Step case** list.

$$\begin{aligned}&(\mathrm{map}(g \circ f))(s : l)\\ &\quad= (g \circ f)s : (\mathrm{map}(g \circ f))l && \text{step case map}\\ &\quad= g(fs) : (\mathrm{map}\,g)((\mathrm{map}\,f)l) && (g \circ f)s = g(fs),\ \text{ind hyp}\\ &\quad= (\mathrm{map}\,g)(fs : (\mathrm{map}\,f)l) && \text{step case map}\\ &\quad= (\mathrm{map}\,g)((\mathrm{map}\,f)(s : l)) && \text{step case map}.\end{aligned}$$

**Exercise 121.** Use the map operator to solve the following problems. For each, define the function that you would like to apply the map operator to, and give an example of a list with at least three elements and show step-by-step how the required transformation is carried out.

(a) Turn a list of lengths in metres to one with the same lengths in kilometres.

(b) Turn a list containing numbers from 1 to 7 into a list of the corresponding days of the week.

(c) Turn a list of UK cities into a list of distances from the corresponding cities to Manchester (you don't have to describe the function for this case formally—just describe the idea and give some of the values).

(d) Turn a list of course units in the School into a list of academic staff who are the leaders for the corresponding course units (you don't have to describe the function for this case formally—just describe the idea and give some of the values).

[11] As discussed above we do not spell out the induction hypothesis explicitly since its statement is identical to the statement we are proving.

**CExercise 122.** Show the following by induction, justifying each step.

(a) If $l$ is a list over an arbitrary set $S$ and $f$ a function from $S$ to some set $T$ then $\operatorname{len}((\operatorname{map} f)l) = \operatorname{len} l$.

(b) If $l$ and $l'$ are elements of $\mathsf{Lists}_S$ and $f\colon S \longrightarrow T$ then

$$(\operatorname{map} f)(l +\!\!+ l') = (\operatorname{map} f)l +\!\!+ (\operatorname{map} f)l'.$$

(c) If $l$ is a list over an arbitrary set $S$ and $f$ a function from $S$ to some set $T$ then $(\operatorname{map} f)(\operatorname{rev} l) = \operatorname{rev}((\operatorname{map} f)l)$.

(d) Let $k_0$ be the function from $\mathbb{N}$ to $\mathbb{N}$ which maps every element to 0, that is

$$\begin{aligned} k_0\colon \mathbb{N} &\longrightarrow \mathbb{N} \\ n &\longmapsto 0. \end{aligned}$$

Assume that $l$ is an element of $\mathsf{Lists}_{\mathbb{N}}$. Show by induction that for the list

$$(\operatorname{map} k_0)l$$

every member is equal to 0.

*You may invoke previous parts even if you have not proved them.*

**Optional Exercise 24.** This exercise explores further the connection between sets, lists over that set, and the map operator.

(a) Define a function $i_S$ from a set $S$ to lists over the set by mapping each element of $S$ to the one-entry list containing just $s$, that is

$$i_S\colon s \longmapsto [s].$$

Show that for all functions $f\colon S \longrightarrow T$, and all $s \in S$ we have

$$(\operatorname{map} f)(i_S s) = i_T(fs).$$

(b) Note that given a set $S$ there is nothing stopping us from building lists over the set of lists over $S$, that is, lists whose entries are lists (over $S$).

So given a function $f\colon S \longrightarrow T$ we can use $\operatorname{map}(\operatorname{map} f)$ to map lists of lists over $S$ to lists of lists over $T$. Give some examples of how this operation works.

(c) Given a list of lists over $S$ we can 'flatten' this list into a list over $S$, for example by turning

$$[[2,3,4],[1,5],[\,],[6]]$$

into

$$[2,3,4,1,5,6].$$

Give a recursive definition of this 'flattening' operator, and let's call this $j_S$.

(d) Show that for all lists $l$ over the set $S$ it is the case that

$$j_S((\operatorname{map} i_S)l) = l.$$

(e) Show that for all functions $f\colon S \longrightarrow T$, and all lists $L$ of lists over $S$ it is the case that

$$(\operatorname{map} f)(j_S L) = j_T((\operatorname{map}(\operatorname{map} f))L).$$

We have sketched here some advanced structure from an area called category theory which tells us something about how lists over a set relate to the original set.

## 6.2 Trees

A datatype that appears in many programming languages is that of a binary tree. Binary trees do feature in the programming in Java course unit COMP16412, where you will also see them used to introduce notions of writing recursive code in that language.

### 6.2.1 Binary trees defined

We give a formal definition for trees where every node has 0 or two children.

**Definition 48: tree**

A **full binary tree with labels from a set $S$** is given by

**Base case** tree. For every element $s$ of $S$ there is a tree, $\texttt{tree}\, s$, consisting of just one node labelled with the element $s$.

**Step case** tree. Given two trees $t$ and $t'$ labelled over $S$, and an element $s$ of $S$, there is a tree $\texttt{tree}_s(t, t')$.

**Takeaway**

Such trees either consist of a single node, or they have a root node, and a left and a right subtree.

We use

$$\mathsf{FBTrees}_S$$

for the set of all binary trees with labels from $S$.

**Code Example 6.5.** Again we show how to define a corresponding class[12] in Java, again for integers as the entries. To emphasize that this is a bit different from the mathematical notion defined above we use a slightly different name for this class.

```
public class BTree {
    public int value;
    public BTree left, right;

    public BTree (int s, BTree t1, BTree t2)
    {value = s; left = t1; right = t2;}
```

```
}
```

Compare this to the following definition from a Java textbook::

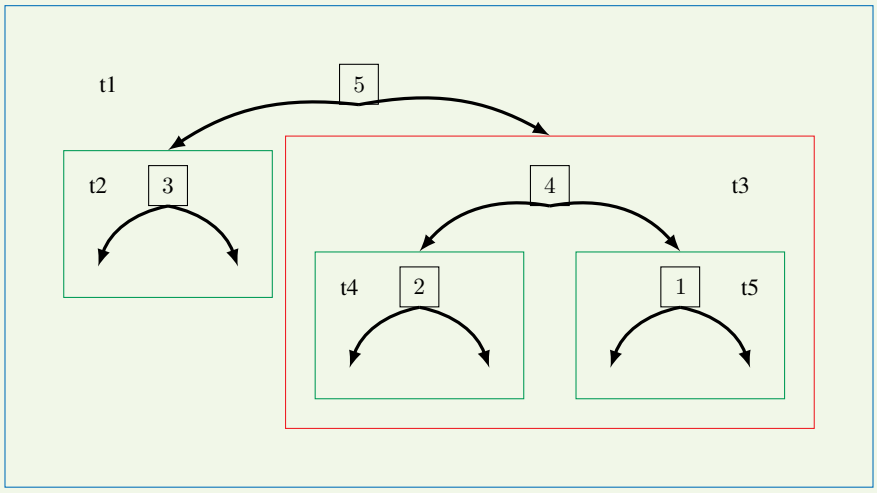
```
public class BinTreeNode
{
   private BinTreeNode left, right;
   private SomeClass nodeData;

   ...
}
```

Both these definitions of a class say that every binary tree node has two binary tree nodes, the left and the right one, and that it has a variable which contains data of some kind. In our case, the class SomeClass happens to be **int**, and with that instantiation we get our class BTree.

How can we picture an object of this class? We use the same idea as the visualization of an object of class List. Assume we have

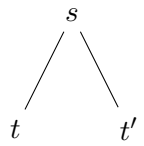
- a BTree object t with t.value=5 and t.left=t2 and t.right=t3 and
- a BTree object t2 with t2.value=3 and t2.left=**null** and t2.right=**null** and
- a BTree object t3 with t3.value=4 and t3.left=t4 and t3.right=t5 and
- a BTree object t4 with t4.value=2 and t4.left=**null** and t4.right=**null** and
- a BTree object t5 with t5.value=1 and t5.left=**null** and t5.right=**null**.

These objects may be pictured as follows,giving the whole tree in blue, the left subtree in red, and the right subtree (and its subtrees) in green.




See the following example for a less space-consuming way of drawing the corresponding full binary tree.

This kind of picture is a bit elaborate to draw, and in order to visualize instances of Section 6.2 most people draw pictures such as the following for a tree with label $s$ and left and right subtrees $t$ and $t'$ respectively.



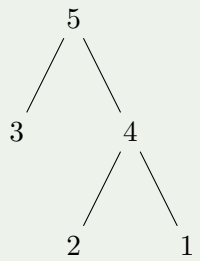

Note that $t$ and $t'$ are trees themselves, so the above does not give the full shape of the tree. To find that one has to look into how $t$ and $t'$ are defined.

**Example 6.18.** We look at the same tree as in Code Code Example 6.5, but this time we use the mathematical description.

If we take the complete description of a tree, such as[13]

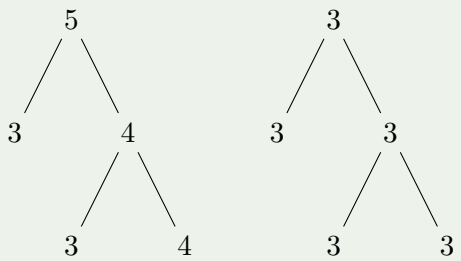
$$\texttt{tree}_5(\texttt{tree}\,3, \texttt{tree}_4(\texttt{tree}\,2, \texttt{tree}\,1))$$

over $\mathbb{N}$, we may draw the full shape.




Note that in our definition there is no such thing as an empty tree. Every tree has at least one node.

**Example 6.19.** Note that our definition of binary tree allows values to repeat, and so in particular trees like




are legitimate trees.

Two trees are **equal** if they are created using identical instances of base and step case in the same order. Informally this means that when we draw the trees they look identical.

We introduce some nomenclature for trees. For the tree $\texttt{tree}_s(t, t')$ the node labelled with the $s$ that appears in the description is the **root** of the resulting tree. In the above example that root has the label 5. The nodes labelled 3 and 4 are the **children** or the root node. Nodes that do

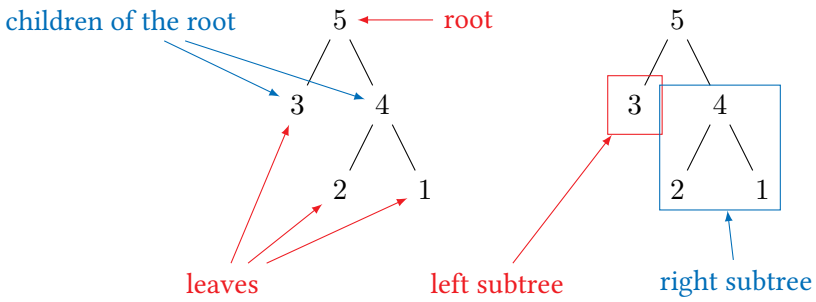
[12]There is one difference between the mathematical definition and the code below. Can you see what it is? Think about **null** references.

[13]Note that elements of $S$ may occur more than once in the tree—we give an example where this does not happen to make the connection with the picture clearer.

not have any children are the **leaves** of the tree. Note that when you define a tree recursively then the leaves of the final tree are built by invoking the base case at the beginning of the building process. The inner nodes (that is, non-leaves) of the tree are built by invoking the step case.

In our definition the two maximal subtrees of a tree are given one after the other (in an **ordered pair**), and so it makes sense to speak of the first of these as the left (rooted at the node labelled 3) and the second as the right (rooted at the node labelled 4) subtree.

We illustrate these definition with the following illustrations.




For completeness' sake we add two more definitions regarding trees. A binary tree is **perfect** if every leaf has the same distance to the root, namely the height of the tree. This is equivalent to demanding that on level $i$, counting down, with the root giving level 0, there are $2^i$ many nodes.

A binary tree is **complete** if and only if there are $2^i$ nodes on level $i$, with the possible exception of the final level, which must be 'filled' from left to right.

### 6.2.2 Operations on trees

Again we give examples for recursive definitions of operations for these recursively defined entities and give proofs by induction.

**Code Example 6.6.** For our Java class BTree we may want to know whether two objects describe the same tree structure. The following code does this for us.

```
public static boolean equal (BTree t1, BTree t2)
{
 if (t1 == null)
     return (t2 == null);
 else {
     if (t2 == null)
         return false;
     else
         return ((t1.value == t2.value)
         && equal (t1.left, t2.left)
         && equal (t1.right, t2.right));
   }
}
```

This method returns true precisely when the two trees providing its arguments have the

same structure as trees.

The **height of a tree** gives a (crude) notion of the size of the tree.[14] Compare the following definition with that of the length of a list, CExercise 117.

**Example 6.20.** We define the height function[15] recursively.

**Base case** hght. $\quad \text{hght}\ \texttt{tree}\ s = 0.$

**Step case** hght. $\quad \text{hght}\ \texttt{tree}_s(t, t') = \max\{\text{hght}\ t, \text{hght}\ t'\} + 1.$

We work out how this definition works for the sample tree from Example 6.18.

$$
\begin{aligned}
&\text{hght}\ \texttt{tree}_5(\texttt{tree}\ 3, \texttt{tree}_4(\texttt{tree}\ 2, \texttt{tree}\ 1)) \\
&= \max\{\text{hght}\ \texttt{tree}\ 3, \text{hght}\ \texttt{tree}_4(\texttt{tree}\ 2, \texttt{tree}\ 1)\} + 1 && \text{step case hght} \\
&= \max\{0, \max\{\text{hght}\ \texttt{tree}\ 2, \text{hght}\ \texttt{tree}\ 1\} + 1\} + 1 && \text{def hght} \\
&= \max\{0, \max\{0, 0\} + 1\} + 1 && \text{base case hght} \\
&= \max\{0, 1\} + 1 && \text{def max} \\
&= 1 + 1 && \text{def max} \\
&= 2
\end{aligned}
$$

Note that for this definition we have that the height of the tree is the integer part of the value of the logarithm (to base 2) of the number of nodes in the tree.

**Code Example 6.7.** We give the code that calculates the height of an object of class BTree. Note that this does something slightly different to our definition for trees in $\mathsf{FBTrees}_{\mathbb{Z}}$!

```
public static int height (BTree t)
{
if (t == nulll)
    return 0;
else
    return 1 + vMath.max(height(t.left), height(t.right));
}
```

What happens if this program is called for a tree whose left and right subtrees are null pointers, which is a tree corresponding to the base case of trees in $\mathsf{FBTrees}_{\mathbb{Z}}$?

**Example 6.21.** We give a recursive definition of the function

$$\text{no}\colon \mathsf{FBTrees}_S \longrightarrow \mathbb{N}$$

that counts the number of nodes in a tree in with labels from $S$.

[14] Some people call this the *depth* of the tree.

[15] Note that when defining the height of a tree it makes sense to either count the number of layers in a tree, or to count the number of connections between them. If we want to former we set the height of a one-node tree as 1, and the latter case is given here.

**Base case** no. We set $\mathrm{no}(\mathtt{tree}\, s) = 1$.

**Step case** no. We set $\mathrm{no}(\mathtt{tree}_s(t, t')) = 1 + \mathrm{no}\, t + \mathrm{no}\, t'$.

Compare this definition with that of the length of a list, CExercise 117. We can show by induction that the number of nodes in a tree in the set $\mathsf{FBTrees}_N$ is odd.

**Base case** tree. We calculate $\mathrm{no}(\mathtt{tree}\, s) = 1$, which is odd.

**Ind hyp.** We assume that $t$ and $t'$ are binary trees each with an odd number of nodes.[16]

**Step case** tree. We check this case.

$$\mathrm{no}\, \mathtt{tree}_s(t, t') = 1 + \mathrm{no}\, t + \mathrm{no}\, t' \qquad \text{step case } \mathrm{no}\,.$$

By the induction hypothesis we can find $k$ and $k'$ in $\mathbb{N}$ such that

$$\mathrm{no}\, t = 2k + 1 \qquad \text{and} \qquad \mathrm{no}\, t' = 2k' + 1.$$

Hence we may continue the above calculation as follows.

$$\begin{aligned} \mathrm{no}\, \mathtt{tree}_s(t, t') &= 1 + \mathrm{no}\, t + \mathrm{no}\, t' && \text{step case } \mathrm{no} \\ &= 1 + 2k + 1 + 2k' + 1 && \text{ind hyp} \\ &= 2(k + k' + 1) + 1 && \text{calcs in } \mathbb{N} \end{aligned}$$

This is an odd number which completes the proof.

Note that this is an example where proving the desired property requires us to do a bit more than write a sequence of equalities.

**Example 6.22.** We show by induction that for every tree $t$ in the set $\mathsf{FBTrees}_N$ it is the case that $\mathrm{hght}\, t \leq \mathrm{no}\, t$.

**Base case** tree.

$$\begin{aligned} \mathrm{hght}(\mathtt{tree}\, s) &= 0 && \text{base case } \mathrm{hght} \\ &\leq 1 \\ &= \mathrm{no}(\mathtt{tree}\, s) && \text{base case } \mathrm{no}\,. \end{aligned}$$

**Ind hyp.** For the trees $t$ and $t'$ we have

$$\mathrm{hght}\, t \leq \mathrm{no}\, t \qquad \text{and} \qquad \mathrm{hght}\, t' \leq \mathrm{no}\, t'.$$

We observe that for all natural numbers $m$ and $n$ we have that

$$m \leq m + n \qquad \text{and} \qquad n \leq m + n,$$

and so

$$\max\{m, n\} \leq m + n.$$

[16] This is the standard induction hypothesis for a data type with one step case which has two previously defined entities as inputs—we give it here because it's the first example of this kind.

For this reason the induction hypothesis implies

$$\max\{\text{hght}\, t,\ \text{hght}\} \leq \text{hght}\, t + \text{hght}\, t' \leq \text{no}\, t + \text{no}\, t',$$

which we use below.

**Step case** tree.

$$\begin{aligned}
&\text{hght}(\texttt{tree}_s(t, t')) \\
&\qquad = \max\{\text{hght}\, t, \text{hght}\, t'\} + 1 && \text{step case hght} \\
&\qquad \leq \max\{\text{no}\, t,\ \text{no}\, t'\} + 1 && \text{ind hyp} \\
&\qquad \leq \text{no}\, t + \text{no}\, t' + 1 && \max\{n, n'\} \leq n + n' \text{ in } \mathbb{N} \\
&\qquad = \text{no}\, \texttt{tree}_s(t, t') && \text{step case no}.
\end{aligned}$$

**Tip**

A proof by induction for trees in the set $\mathsf{FBTrees}_S$ always has the following shape. We are trying to prove a statement formulated in terms of the variable $t$ which is an element of $\mathsf{FBTrees}_S$.

**Base case** tree. Prove the given statement for the case where all occurrences of $t$ have been replaced by $\texttt{tree}\, s$, where $s$ is an arbitrary element of $S$.

**Ind hyp** Assume the given statement holds for the trees[17] $t$ and $t'$.

**Step case** tree. Prove the statement where all occurrences of $t$ have been replaced by $\texttt{tree}_s(t, t')$, where $s$ is an arbitrary element of $S$. The induction hypothesis is used as part of the proof.

Note how the general scheme derives from the shape of the definition of our trees.

**Code Example 6.8.** We give one last version of code for a recursive function. Note that once again the difference between our mathematical binary trees, which cannot be empty, and the trees of class BTree, which can, makes a difference.

```
public static int no (BTree t)
{
if (t == null)
    return 0;
else
    return 1 + no(t.left) + no(t.right);
}
```

[17] It may be necessary to assume it for $t$, $t'$ and all their subtrees.

**Example 6.23.** We give a recursive definition of the function

$$\mathrm{lvs}\colon \mathsf{FBTrees}_S \longrightarrow \mathbb{N}$$

that counts the number of leaves in a tree in $\mathsf{FBTrees}_S$.

**Base case** lvs. $\quad \mathrm{lvs}\,\mathtt{tree}\,s = 1.$

**Step case** lvs. $\quad \mathrm{lvs}\,\mathtt{tree}_s(t, t') = \mathrm{lvs}\,t + \mathrm{lvs}\,t'.$

Note how little this definition differs from that of the function no. A small change in a recursive definition can lead to a very different effect.

**Tip**

To get started on defining a recursive function for trees you might as well make use of the fact that you know what such a definition needs to look like. Assume we intend to define a function

$$f\colon \mathsf{FBTrees}_{\mathbb{N}} \longrightarrow S$$

that behaves in a particular way. Then we know we need to have two cases:

**Base case** $f$. $\quad f\,\mathtt{tree}\,n = ?$

**Step case** $f$. $\quad f(\mathtt{tree}_n(t, t')) = ???\ ft\ ?\ ft'.$

It should be easy to read off from your description of $f$ how to define it in the base case. For the step case you may require a bit of creativity. How does knowing $ft$ and $ft'$ help us to calculate $f(\mathtt{tree}_n(t, t'))$? Note that you are allowed to use operations from the target set of $f$, here $S$. When you have a definition check whether it does the right thing using an example.

**CExercise 123.** Assume that $N$ is a set of numbers, namely $\mathbb{Z}$, $\mathbb{Q}$ or $\mathbb{R}$.

(a) Draw the following tree: $\mathtt{tree}_3(\mathtt{tree}\,2, \mathtt{tree}_{-5}(\mathtt{tree}\,2, \mathtt{tree}\,-1))$

(b) Define a function

$$\mathrm{sum}\colon \mathsf{FBTrees}_N \longrightarrow N$$

which sums up all the labels that occur in the tree (compare Example 6.3 where the analogous operation for lists is defined).

(c) Apply your function to the tree from part (a).

(d) Give the code for a function that performs the same job for an object of class BTree.

(e) Justifying each step show that if the set of labels is $\mathbb{N} \setminus \{0\}$ then for all full binary trees $t$ we have

$$\mathrm{no}\,t \leq \mathrm{sum}\,t.$$

**Exercise 124.** Consider the function

$$\text{search}\colon \mathsf{FBTrees}_S \times S \longrightarrow \{0,1\}$$

which, for inputs $t$ and $s$, returns 1 precisely when $s$ occurs as a label in the tree $t$.

(a) Give a recursive definition for this function, You may use the function $f_{=_S}$ from Example 6.14 as well as the boolean operations on $\{0,1\}$.

(b) Give code that implements this function for objects of class BTree.

**CExercise 125.** Assume we have a function $f\colon S \longrightarrow T$.

(a) Recursively define a function $\text{map}\, f$ that takes a binary tree with labels from $S$ to a binary tree with labels from $T$, that is

$$\text{map}\, f\colon \mathsf{FBTrees}_S \longrightarrow \mathsf{FBTrees}_T \ .$$

*Hint: Consider the* map *function for lists as an example.*

(b) For the function

$$\begin{aligned} f\colon \mathbb{N} &\longrightarrow \mathbb{N} \\ n &\longmapsto 2n \end{aligned}$$

apply the function $\text{map}\, f$ you defined in part (a) step by step to the following tree.

$$\texttt{tree}_{17}(\texttt{tree}_5(\texttt{tree}\,3, \texttt{tree}\,19), \texttt{tree}_{25}(\texttt{tree}\,19, \texttt{tree}\,27))$$

(c) Show that for every such function $f$, and every tree $t$ in the set $\mathsf{FBTrees}_S$, we have

$$\text{hght}((\text{map}\, f)t) = \text{hght}\, t.$$

(d) Show that for every such function $f$ and every tree $t$ in the set $\mathsf{FBTrees}_S$ we have

$$\text{no}((\text{map}\, f)t) = \text{no}\, t.$$

(e) Assume that $N$ is a set of numbers between $\mathbb{N}$ and $\mathbb{R}$. Show that for the function

$$\begin{aligned} k_1\colon N &\longrightarrow N \\ n &\longmapsto 1 \end{aligned}$$

we have for all binary trees $t$ that $\text{no}\, t = \text{sum}((\text{map}\, k_1)t)$.

Justify each step in your proofs.

**EExercise 126.** This exercise is concerned with defining a mathematical entity of binary trees that corresponds to the BTree class. We refer to such a tree as a **binary tree with labels from a set $S$**, and use $\mathsf{BTrees}_S$ for the set of all these trees.

(a) Give a definition, similar to Definition 48, of a mathematical entity of *binary trees* which corresponds to the BTree class.

(b) Recursively define an operation that takes two binary trees and returns a binary tree where the second argument has been added below the right-most leaf of the tree.

(c) Write code which implements the operation from the previous part.

**Exercise 127.** Give the following recursive definitions.

(a) Trees with labels from $S$ such that every node has at most two children.*Hint: You may want to allow an empty tree. You may also want to draw some examples to give you an idea what you are looking for.*

(b) Trees with labels from $S$ such that every node can have an arbitrary finite number of children.

**Exercise 128.** This exercise is concerned with *perfect binary trees.*

(a) Recursively define the set of all those full binary trees with labels from a set $S$ that are perfect. Do so by describing, for each height, those elements of $\mathsf{FBTrees}_S$ which are perfect. *Hint: Have a look ahead to Example 6.45, and for each $n \in \mathbb{N}$ define a set $\mathsf{PTrees}_S^n$ of those elements of $\mathsf{FBTrees}_S$ which are perfect and have height $n$.*

(b) Show that for such a tree $t$ we have

$$\operatorname{lvs} t = 2^{\operatorname{hght} t}.$$

(c) Show that it is indeed the case that, for $t \in \mathsf{PTrees}_S^n$, we have $\operatorname{hght} t = n$.

(d) Show that for a perfect tree $t$ we have $\operatorname{lvs} t = 2^{\operatorname{hght} t}$.

(e) Show that for a perfect tree $t$ we have

$$\operatorname{no} t = 2^{1+\operatorname{hght} t} - 1,$$

Conclude that

$$\operatorname{hght} t = \log(1 + \operatorname{no} t) - 1.$$

### 6.2.3 Ordered binary trees

Trees are useful structures when it comes to keeping data in a way that makes it easy to search. In Example 4.99 we looked at searching through an array, here we see in EExercise 130 how one can search through entries that are kept in a tree in an ordered way, rather than in an array. Note that the ideas that follow are usually employed for trees in binary trees rather than full binary trees.

**Example 6.24.** Sometimes we require the set of all the labels that occur in a tree, for example in order to define *ordered binary trees*, see below. The **set of labels** $\operatorname{lab} t$ **that occur in a tree** $t$ with labels from $S$ is given as follows:

**Base case** lab. We have $\operatorname{lab}(\texttt{tree}\, s) = \{s\}$.

**Step case** lab. We have $\operatorname{lab}(\texttt{tree}_s(t, t')) = \{s\} \cup \operatorname{lab} t \cup \operatorname{lab} t'$.

Note that this defines a function

$$\mathrm{lab}\colon \mathsf{FBTrees}_S \longrightarrow \mathcal{P}S,$$

since it maps a binary tree with labels from $S$ to a subset of $S$.

If we have trees over a set of numbers $N$ with $\mathbb{N} \subseteq N \subseteq \mathbb{R}$ then that set comes with an *order*[18] then we can recursively define what we mean by a binary tree over $N$ being ordered.

**Definition 49: ordered binary tree**

A binary tree with labels from $N$ is **ordered** under the following conditions.

**Base case** tree. A tree of the form $\mathtt{tree}\, n$ is ordered.

**Step case** tree. A tree of the form $\mathtt{tree}_n(t, t')$ is ordered if and only if

- For every $m \in \mathrm{lab}\, t$ we have $m \leq n$,
- for every $m' \in \mathrm{lab}\, t'$ we have $n \leq m'$ and
- $t$ and $t'$ are ordered.

Ordered binary trees are sometimes called *binary search trees*.

**Takeaway**

A binary tree with rules regarding where the labels may appear; used to store values we may search for.

We use $\mathsf{OFBTrees}_S$ for the set of ordered full binary trees with labels from the set $S$.

In Java there the TreeSet class is used to store values in trees in ascending order, so you don't have to program your own data structure for this kind of thing. Note that there is a difference between that class and our mathematical definition.

Ordered binary trees are used to keep an unknown number of data in a structure that allows for the binary search algorithm to be defined. Typically one would like to be able to insert new nodes, or to delete them. For this application one cannot use ordered *full* binary trees because after an insertion or deletion the tree will no longer be full, and one really should stick to ordered binary trees. Binary trees are mathematically defined in EExercise 126.

**Exercise 129.** Which of the following trees are ordered over the obvious set of numbers? *Hint: You may find it easier to answer this question if you draw the tree first.*

(a) $\mathtt{tree}_3(\mathtt{tree}\, 2, \mathtt{tree}_{17}(\mathtt{tree}\, 5, \mathtt{tree}\, 19))$

(b) $\mathtt{tree}_5(\mathtt{tree}_2(\mathtt{tree}\, 0, \mathtt{tree}\, 1), \mathtt{tree}\, 149))$.

(c) $\mathtt{tree}_{17}(\mathtt{tree}_5(\mathtt{tree}\, 3, \mathtt{tree}\, 19), \mathtt{tree}_{25}(\mathtt{tree}\, 19, \mathtt{tree}\, 27))$.

(d) $\mathtt{tree}_{25}(\mathtt{tree}_9(\mathtt{tree}\, 4, \mathtt{tree}\, 1), \mathtt{tree}_{49}(\mathtt{tree}\, 36, \mathtt{tree}\, 64))$.

(e) $\mathtt{tree}_{-3}(\mathtt{tree}_{-5}(\mathtt{tree}\, -7. \mathtt{tree}\, -4), \mathtt{tree}_5(\mathtt{tree}\, 4, \mathtt{tree}\, 7))$.

[18] See Section 7.5.1 for a formal definition of this concept.

**EExercise 130.** Let $N$ be a set of numbers between $\mathbb{N}$ and $\mathbb{R}$. Consider the function

$$\text{search}\colon \mathsf{OFBTrees}_N \times N \longrightarrow \{0,1\}$$

that for an input consisting of a ordered[19] binary tree $t$ with labels from $N$ and an element $n$ of $N$ gives 1 if $n$ occurs as a label in $t$, and 0 otherwise.

(a) Give a recursive definition of search. We may think of this function as performing a binary search.

(b) Write code that implements this function, assuming we have a class OBTree which restricts BTree to ordered binary trees only.

(c) Draw a picture of the ordered binary tree

$$\texttt{tree}_{-3}(\texttt{tree}_{-5}(\texttt{tree}\,-7, \texttt{tree}\,-4), \texttt{tree}_5(\texttt{tree}\,3, \texttt{tree}\,7)).$$

Imagine you want to add a node with label 4 to the tree. Where should that go? Draw your solution. Do you still have a full binary tree?

(d) Write code that takes as inputs an object of the class OBTree and an integer and returns an object of class OBTree in which the second argument has been inserted if it is not already present. Use the previous part to guide you.

*Hint: Your definition should take advantage of the fact that the tree is ordered and not look at every label.*

## 6.3 Syntax

Many formal languages are recursively defined. Examples of these are computer languages, or the language of logical propositions, or the language of regular expressions that appears in COMP11212.

This allows us to recursively define operations as well as to give inductive proofs of properties of such operations. This is one of the reasons why understanding recursion and induction is very useful in computer science.

We describe some examples of this kind in this section.

### 6.3.1 Strings

The simplest example of this kind is that of a **string** over a set $S$. We also speak of a **word over the alphabet** $S$.

**Base case** string. The empty string[20] $\epsilon$ is a string.

**Step case** string. If $s \in S$ and $w$ is a string then $ws$ is a string.

One can now give a recursive definition of concatenation for strings, and one can define the length of a string recursively, just as these operations are defined for lists. Indeed, lists and strings are closely related, see the following exercise for the precise connection.

[19] This exercise assumes you have a solution to EExercise 126.

[20] Since it's a bit difficult to indicate something empty we use a symbol here. This is standard practice.

**Exercise 131.** Carry out the following tasks. *This is a really nice exercise to work out whether you've understood all the concepts from this chapter.*

(a) Give a recursive definition for the concatenation $+$ of strings over a set $S$. Ditto reusing $\mathrm{len}$ and in particular $+$

(b) Give a recursive definition of the length $\mathrm{len}$ of a string over a set $S$.

(c) Show that if $w$ and $w'$ are strings over a set $S$ then $\mathrm{len}\, w + \mathrm{len}\, w' = \mathrm{len}(w + w')$.

(d) Recursively define a function from lists over $S$ to strings over $S$.

(e) Recursively define a function from strings over $S$ to lists over $S$.

(f) Show by induction that your two functions are mutual inverses of each other.

(g) What can you say about your two functions regarding how they behave with respect to the $\mathrm{len}$ functions for strings and lists? What about these functions and concatenating strings, versus concatenating lists?

Justify each step in your proofs.

### 6.3.2 Logical propositions

We now turn to the language of logical propositions that is studied in Chapter 3. We recall the definition of the boolean interpretation of a propositional formula[21] from Section 3.2.1.

**Base case** prop. For every propositional variable[22] $Z$, $Z$ is a proposition.

**Step cases** prop. Assume that $A$ and $B$ are propositions. Then the following are propositions.

**Step case** $\neg$. $(\neg A)$,
**Step case** $\wedge$. $(A \wedge B)$,
**Step case** $\vee$. $(A \vee B)$ and
**Step case** $\rightarrow$. $(A \rightarrow B)$.

**Example 6.25.** The definition of an interpretation relative to a valuation of a proposition is also via recursion. Assume that we have a valuation $v$ that gives a value to each propositional variable in the boolean algebra $\{0, 1\}$. We give the formal definition of the boolean interpretation of propositions. This is given by a function $I_v$ which maps a proposition to its boolean interpretation relative to $v$. The source of this function is the set of all propositions, and its target is $\{0, 1\}$.

**Base cases** $I_v$. For a propositional variable $Z$ we define $I_v Z = vZ$.

**Step cases** $I_v$. We define:

**Step case** $\neg$. $I_v(\neg A) = \neg I_v A$,
**Step case** $\wedge$. $I_v(A \wedge B) = I_v A \wedge I_v B$,

[21] Below we use the shorter 'proposition' instead of 'propositional formula'.
[22] In this section we use this letter for propositional variables.

**Step case** $\vee$. $I_v(A \vee B) = I_v A \vee I_v B$ and

**Step case** $\rightarrow$. $I_v(A \rightarrow B) = I_v A \rightarrow I_v B$.

Note that to argue about the properties of semantic equivalence studied in the material on logic we did not use induction. To show, for example, that the boolean interpretation of $\neg\neg A$ is the same as that of $A$ for all valuations is, when fully spelled out, an induction proof but it can be made plausible without that. See the following exercise for an idea of how this works formally.

**Tip**

A proof by induction for propositions usually takes the following shape:

**Base cases** prop. We show that the statement holds for every proposition of the form Z, where Z is a propositional variable.

**Ind hyp.** We assume that the statement holds for formulae A and B.

**Step cases** prop. We show that, given the induction hypothesis, the statement holds for formulae of the form

- $\neg A$,
- $A \wedge B$,
- $A \vee B$,
- $A \rightarrow B$.

**CExercise 132.** Give the formal definition of the powerset interpretation for the set of all propositions (see Section 3.2.2) relative to a valuation.

**EExercise 133.** Solve the following problems for propositional logic.

(a) Give a recursive definition of the function var which takes as its argument a propositional formula and returns the set of propositional variables that occur in that formula.

(b) Give a recursive definition of the subformula construction as a function subf which takes as its argument a propositional formula and returns the set of all its subformulae,[23] see Section 3.1.

(c) Justifying each step, show by induction that for all propositional formulae $A$ we have

$$\text{var } A \subseteq \text{subf } A.$$

Note that the definition of a formal derivation in a natural deduction system is also recursive: The base cases are given by the axiom rules and the step cases by the other derivation rules.

### 6.3.3 Formal languages

We look at elements of formal languages as found in the course unit COMP11212.

[23] You will note that some people use formulas, and some people formulae, for the plural. The latter is the original Latin form, but in modern English some prefer to form an English plural.

**Regular expressions**

The following explains a recursive definition from Part 1 of COMP11212.

**Example 6.26.** Let $\Sigma$ be a set of symbols. A **pattern** or **regular expression over** $\Sigma$ is generated by the following recursive definition.

**Base case** regexp $\emptyset$. The character $\emptyset$ is a pattern;

**Base case** regexp $\epsilon$. the character $\epsilon$ is a pattern;

**Base case** regexp $\Sigma$. every symbol from $\Sigma$ is a pattern;

**Step case** regexp $+$. if $p_1$ and $p_2$ are patterns then so is[24] $(p_1 p_2)$;

**Step case** regexp $|$. if $p_1$ and $p_2$ are patterns then so is $(p_1|p_2)$;

**Step case** regexp $*$. if $p$ is a pattern then so is $(p*)$.

It is typical for formal languages that there are several base cases as well as several step cases. This is due to the fact that we have several notions of a 'primitive' term in our language (that is one that is not built from other terms), and we have several ways of putting terms together to get new terms.

**Example 6.27.** There is a related recursive definition which tells us when a string made up of symbols from $\Sigma$ *matches* a given regular expression.

Let $p$ be a pattern over a set of symbols $\Sigma$ and let $s$ be a string consisting of symbols from $\Sigma$. We say that $s$ **matches** $p$ if one of the following cases holds:

**Base case** regexp $\epsilon$. The empty word $\epsilon$ matches the pattern $\epsilon$.

**Base case** regexp $\Sigma$. A character $x$ from $\Sigma$ matches the pattern $p = x$.

**Step case** regexp $+$. The pattern $p$ is a concatenation $p = (p_1 p_2)$ and there are words $s_1$ and $s_2$ such that $s_1$ matches $p_1$, $s_2$ matches $p_2$ and $s$ is the concatenation of $s_1$ and $s_2$.

**Step case** regexp $|$. The pattern $p$ is an alternative $p = (p_1|p_2)$ and $s$ matches $p_1$ or $p_2$ (it is allowed to match both).

**Step case** regexp $*$. The pattern $p$ is of the form $p = (q*)$ and $s$ can be written as a finite concatenation $s = s_1 s_2 \cdots s_n$ such that $s_1, s_2, \ldots, s_n$ all match $q$; this includes the case where $s$ is empty (and thus an empty concatenation, with $n = 0$).

Noticeably there is no entry for Base case regexp $\emptyset$, and the reason for this is that no string matches the pattern $\emptyset$. Consequently there is no need to include that base case.

**Example 6.28.** A result that is given in the notes for COMP112, but not proved, is covered in the following (somewhat extended) example.

[24] The name of this step case mentions an operator that does not appear in the term constructed, but it's not clear what better name there is for concatenation.

Given a regular expression $p$, we recursively define

$$\mathcal{L}(p)$$

as follows:

**Base case** regexp $\emptyset$. $\mathcal{L}(\emptyset) = \emptyset$.

**Base case** regexp $\epsilon$. $\mathcal{L}(\epsilon) = \{\epsilon\}$.

**Base case** regexp $\Sigma$. $\mathcal{L}(x) = \{x\}$ for all $x \in \Sigma$.

**Step case** regexp $+$. $\mathcal{L}(p_1 p_2) = \mathcal{L}(p_1) \cdot \mathcal{L}(p_2)$.

**Step case** regexp $|$. $\mathcal{L}(p_1 | p_2) = \mathcal{L}(p_1) \cup \mathcal{L}(p_2)$.

**Step case** regexp $*$. $\mathcal{L}(p*) = (\mathcal{L}(p))^*$.

The operations $\cdot$ and $(-)^*$ for sets of strings are defined in the COMP112 notes.

Note that we can think of $\mathcal{L}$ as a function from the set of all regular expressions over the alphabet $\Sigma$ to the powerset of the set of strings over $\Sigma$, $\mathcal{P}\Sigma^*$.

The claim made, but not proved, in the COMP112 notes is

**Proposition**

For all regular expressions $p$ over the alphabet $\Sigma$ we have that

$$\mathcal{L}(p) = \{s \in \Sigma^* \mid s \text{ matches } p\}.$$

**Proof.** One may prove this by induction. Note that the induction proof has the number of base and step cases indicated by the recursive definition of a regular expression.

**Base case** regexp $\emptyset$. $\mathcal{L}(\emptyset) = \emptyset$. Since no string matches the pattern $\emptyset$ the set of strings that match this pattern is indeed the empty set.

**Base case** regexp $\epsilon$. $\mathcal{L}(\epsilon) = \{\epsilon\}$. By definition, $\epsilon$ is the only string that matches the pattern $\boldsymbol{\epsilon}$.

**Base case** regexp $\Sigma$. $\mathcal{L}(x) = \{x\}$ for all $x \in \Sigma$. By definition the string $x$ is the only string that matches the pattern $x$.

**Step case** reg-ex $+$. $\mathcal{L}(p_1 p_2) = \mathcal{L}(p_1) \cdot \mathcal{L}(p_2)$. By definition of $\cdot$ for sets of words we have that,

$$\mathcal{L}(p_1) \cdot \mathcal{L}(p_2) = \{s_1 s_2 \mid s_1 \in \mathcal{L}(p_1), s_2 \in \mathcal{L}(p_2)\},$$

By the induction hypothesis, this latter set is the set of all concatenations of strings $s_1 s_2$ such that

$$s_1 \text{ matches } p_1 \qquad \text{and} \qquad s_2 \text{ matches } p_2,$$

and by definition, a string $s$ matches the pattern $p_1 p_2$ if and only if there exist strings $s_1$ and $s_2$ such that

$$s_1 \text{ matches } p_1 \qquad \text{and} \qquad s_2 \text{ matches } s_2$$

$$\text{and} \qquad s = s_1 s_2,$$

which gives the claim.

**Step case** regexp $|$. $\mathcal{L}(p_1|p_2) = \mathcal{L}(p_1) \cup \mathcal{L}(p_2)$. By definition a string $s$ matches $p_1|p_2$ if and only if at least one of

$$s \text{ matches } p_1 \qquad \text{or} \qquad s \text{ matches } p_2,$$

which by the induction hypothesis means that this is the case if and only if

$$s \in \mathcal{L}(p_1) \cup \mathcal{L}(p_2).$$

**Step case** regexp $*$. $\mathcal{L}(p*) = (\mathcal{L}(p))^*$. By definition the pattern $p*$ is matched by the string $s$ matches if and only if there is

$$n \in \mathbb{N} \text{ and } s_1, s_2, \ldots s_n \in \Sigma^*$$

with

$$s = s_1 s_2 \cdots s_n$$
$$\text{and} \qquad s_i \text{ matches } p \ (0 \leq i \leq n).$$

But by the induction hypothesis, this means that

$$s_i \in \mathcal{L}(p) \text{ for } 0 \leq i \leq n,$$

and this is equivalent to

$$s = s_1 s_2 \cdots s_n \in (\mathcal{L}(p))^*$$

as required.

In the notes for COMP11212 no formal proof is given that for every regular expression there is a finite state automaton defining the same language—what is described there is effectively a sketch of such an argument. Such a proof would proceed by induction over the above definition, including all the given cases. If you look at the text that explains how to turn a regular expression into a finite state automaton you should be able to see how each base case, and each step case, is treated there. As a consequence the essence of the proof is given in those notes, and all you would have to do to complete it is to introduce the formal induction structure.

### Context-free grammars

There are a number of recursive definitions connected with context-free grammars.[25] If we look at the definition of a string being generated by a grammar we can see that these are instructions that follow recursive rules, using the following pattern:

**Base case** $\Gamma$. This is provided by the start symbol $S$.

**Step cases** $\Gamma$. We have a step case for each production rule.

[25] Please note that this material is not taught on COMP11212 until Week 5, so you may want to skip reading this section until you have seen the concept on the other unit.

If we generalize the definition of a string being generated by a grammar to strings consisting of both, terminal and non-terminal, symbols the recursive nature of the construction becomes obvious.

**Example 6.29.** The definition of when a string is *generated by a grammar* is one example:

**Base case** gen. by $\Gamma$. The string $S$ is generated by $\Gamma$.

**Step cases** gen. by $\Gamma$. If $R \to Y$ is a rule of the grammar and the string $X$ is generated by $\Gamma$ then the string that results from replacing one occurrence of $R$ in $X$ by $Y$ is a string generated by $\Gamma$.

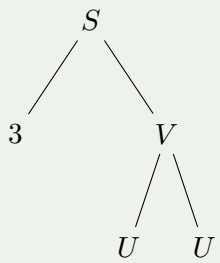
**Example 6.30.** Similarly the notion of a *parse tree*[26] for a grammar has an underlying recursive definition:

**Base case** parse tree for $\Gamma$. The tree consisting of only one node, which is labelled $S$, is a parse tree for $\Gamma$.

**Step cases** parse tree for $\Gamma$. If $R \to Y$ is a rule of the grammar and the tree $t$ is generated by $\Gamma$ then the tree that results from the following process is a parse tree for $\Gamma$:

- Identify a leaf labelled $R$.
- Add new children to that leaf, one for every symbol that occurs in $Y$ in that order.

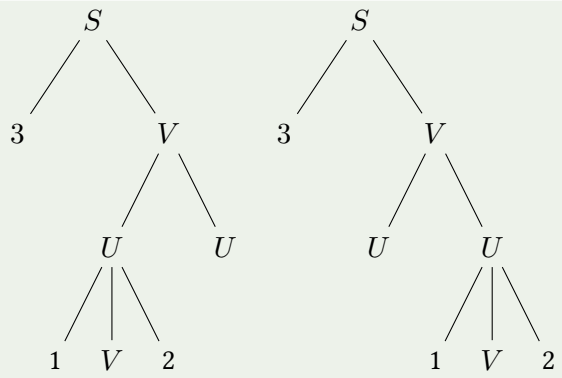
For example assume we have a grammar with non-terminal symbols $S$, $U$ and $V$, and terminal symbols digits from 0 to 9, if we have a parse tree as follows.




and a rule

$$U \to 1V2$$

then both of the following trees are parse trees for $\Gamma$:




### 6.3.4 Elements of programming languages

**Example 6.31.** To give another example, assume that we have a set of numbers $N$, on which we have various operations and properties, see below for a concrete example. For such a set of numbers we may express calculations by forming terms that consist of various elements of $N$ connected by the available operations. This gives us a language of such expressions. Many programming languages have such a notion of *arithmetic expressions* built in.

One might try to specify an arithmetic expression over the set $N$ by the following recursive definition:

**Base cases** arex. For every $n \in N$ we have that $n$ is an arithmetic expression.

**Step cases** arex. Assume that $a$ and $a'$ are arithmetic expressions. Then the following are arithmetic expressions:

| | |
|---|---|
| **Step case** $()$. | $(a)$, |
| **Step case** $-$. | $-a$, |
| **Step case** $^{-1}$. | $a^{-1}$, |
| **Step case** $+$. | $a + a'$ and |
| **Step case** $\cdot$. | $a \cdot a'$. |

This is the kind of definition (see the While language below) that appears as part of a larger definition of a programming language.[27] If the set of numbers in question is some approximation to the real numbers then one might also have built-in operations such as the trigonometric functions, and there is probably support for exponentiation of numbers.

Typically the idea is that the programmer, as part of the code, uses arithmetic expressions to specify calculations to be carried out by the computer at run-time.

There's one problem with the definition from Example 6.31: Not all arithmetic expressions defined in this way work as intended: This is because we have done nothing to ensure that the multiplicative inverse is only applied to expressions which are different from 0. Some languages (such as Java) have mechanisms to deal with situations like this, for example by raising an exception, while in other cases one might actually get a result, but there are no guarantees that this result makes any sense.

[26] In the COMP11212 notes the only parse trees drawn are those whose leaves are labelled with terminal symbols, but for a formal definition one needs to first allow a more general version.

[27] Typically there are more expressions, for example the programmer is allowed to write $a - a'$ as a shortcut for $a + (-a')$.

This means that at run-time (or in some cases at compile-time) the machine will find itself being asked to carry out a multiplication by an inverse that does not exist, such as a division by 0 in the integers.

One would like to change the above definition by specifying a changed step case:

- If $a$ is an arithmetic expression with $a \neq 0$ then $a^{-1}$ is an arithmetic expression.

But in order to carry out the test to check whether an arithmetic expression is equal to 0, either the programmer, or the machine, would effectively have to do all the required calculations *before* the program is compiled. Clearly it would be pointless to expect the programmer to carry out this check when writing the program. After all, the point of writing code is to have the computer deal with such calculations. But it turns out that one can not ask the computer to do these calculations prior to run-time either. This is an issue related to the *Halting Problem*, which is explained in more detail in COMP11212.

Every programming language that supports arithmetic expressions has a *recursively defined evaluation procedure* for these expressions, which tell the machine how to calculate the *value* of such an arithmetic expression (or throw up an error if a division by 0 occurs).

For most programming languages the valid programs are defined recursively, typically using grammars. For most languages this definition is very large. Typically it involves defining more than one kind of entity. Below we give an example of a language that has a short definition to given an idea of how this might work. Note that this language is no less powerful (in a formally definable way) than, say, Java.

**Example 6.32.** We give the definition[28] of the While programming language as it appears in the notes for COMP11212.

First it is necessary to define *arithmetic expressions* for While. This assumes that there is a notion of number predefined, and that we have a notion of variable. To keep this definition short we do not cover those here.[29]

**Base case** AExp $n$. For every number $n$ we have that $n$ is a While arithmetic expression.

**Base case** AExp var. For every variable $x$ we have that $x$ is a While arithmetic expression.

**Step case** AExp $+$. If $a_1$ and $a_2$ are While arithmetic expressions then so is $a_1 + a_2$.

**Step case** AExp $-$. If $a_1$ and $a_2$ are While arithmetic expressions then so is $a_1 - a_2$.

**Step case** AExp $\times$. If $a_1$ and $a_2$ are While arithmetic expressions then so is $a_1 \times a_2$.

These arithmetic expressions are used in the following definition of While boolean expression:

**Base case** BExp `True`. There is a While boolean expression `True`.

**Base case** BExp `False`. There is a While boolean expression `False`.

**Step case** BExp $=$. If $a_1$ and $a_2$ are While arithmetic expressions then $a_1 = a_2$ is a While boolean expression.

**Step case** BExp $\leq$. If $a_1$ and $a_2$ are While arithmetic expressions then $a_1 \leq a_2$ is a While boolean expression.

**Step case** BExp $\neg$. If $b$ is While boolean expressions then so is $\neg b$.

**Step case** BExp $\wedge$. If $b_1$ and $b_2$ are While boolean expressions then so is $b_1 \wedge b_2$.

Finally we can define While statements.

**Base case** Stm $:=$. If $x$ is a variable and $a$ is a While arithmetic expression then $x := a$ is a statement.

**Base case** Stm `skip`. There is a statement `skip`.

**Step case** Stm ;. If $S_1$ and $S_2$ are statements then so is $S_1; S_2$.

**Step case** Stm `if`. If $b$ is a While boolean expression and $S_1$ and $S_2$ are statements then `if` $b$ `then` $S_1$ `else` $S_2$ is a statement.

**Step case** Stm `while`. If $b$ is a While boolean expression and $S$ is a statement then `while` $b$ `do` $S$ is a statement.

This is not how a computer scientist would write down the definition. You can see that it is very verbose, and that many of the words are not really required to understand what is being defined. For this reason computer scientists have created their own notations for making such definitions more compact, and you will see some of those in COMP11212.

But it is only the notation that differs—mathematically speaking, the corresponding definitions are still examples of recursive definitions.

When we have a formal definition of a programming language we can then reason about all programs for that language. One area of computer science where this happens is the *semantics of programming languages*.

Also note that the definition from Example 6.32 only tells us what the valid While programs are—it does not tell us anything about how those should be executed, and what the result of any computation should be. You will see how one can do that, again making use of a recursive definition, in the second part of COMP11212.

## 6.4 The natural numbers

The classic example of a recursively defined set is that of the natural numbers, and it is the only way of formally defining what this set should be. Very little has to be assumed for this definition.

### 6.4.1 A formal definition

**Definition 50: natural numbers**

The elements of the **set of natural numbers** $\mathbb{N}$ are given by the following recursive definition.

**Base case** $\mathbb{N}$. There is a natural number 0.

**Step case** $\mathbb{N}$. For each natural number $n$ there is a natural number $Sn$, the *successor of n*.

[28] Note that the definition given does not involve brackets but these are introduced by the backdoor in the paragraph following the definition, and you might argue that it would be cleaner to make them explicit from the start.

[29] Also, symbols used in such definitions are usually restricted to those that are readily available from a keyboard. In order to make our definition human readable we stick to symbols used in this course unit.

**Takeaway**

A natural number is either 0, or it is the successor of a natural number.

It is more customary to write $n+1$ for $Sn$, but strictly speaking this has to be justified (see CExercise 135 (b)), and we use $Sn$ for the time being. We introduce

$$1 \text{ as a shortcut for } S0.$$

**Code Example 6.9.** We give a Java class that corresponds to this definition.

```
public class Nat {

    public Nat next;

    public Nat (Nat n)
    {next=n;}
}
```

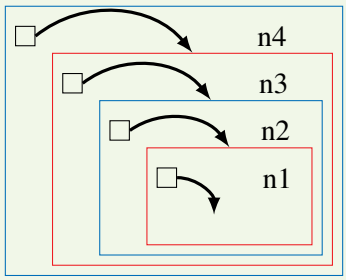
This is a very odd class that one wouldn't want to use in practice, but the example here is to show you that this can be done. Note that this class does not have a built-in upper bound for the numbers so defined (unlike the Java class List). In practice the size of the numbers you can use in this way is limited by the machine's memory. Note also the way the class is constructed the number you get from using the argument encoding $n$ is the number encoding $n+1$.

You can picture an object of this class as shown below.

Assume that we have

- a Nat object n4 with n.next=n3 and
- a Nat object n3 with n3.next=n2 and
- a Nat object n2 with n2.next=n1 and
- a Nat object n1 with n1.next=**null**.

You may picture these objects as follows:




Just as we need to count the number of symbols $S$ to know which number is given by

$$SSSSSSSSSSSSSSSSSSSSSSSSSS0,$$

so we have to check how many references we have to follow until we reach the **null** reference to know which number is given by some object of class Nat.

Previously most of you will have assumed that the natural numbers exist, and that you know what they are, and how their operations work, without having seen a formal definition. For now you are expected to work with this formal definition.

### 6.4.2 Formal definitions for basic operations on the natural numbers

In what follows we illustrate how the familiar operations may be formally defined. You could also think of the following as something that could be implemented on a fairly primitive computer.

**Example 6.33.** An example of a recursively defined operation[30] is addition. The **sum** $m + n$ of two natural numbers $n$ and $m$ is defined as follows. For all $m$ in $\mathbb{N}$:

**Base case** +. $m + 0 = m$ and

**Step case** +. $m + Sn = S(m + n)$.

We have defined $m + n$ *by recursion over* the argument $n$, covering the base case and the step case for natural numbers. Note that in the step case we are using the fact that $m + n$ may be assumed to be already defined when giving a definition for $m + Sn$. Further note that nothing is assumed about the nature of $+$, but, of course, our definition here gives the common notion of addition.

**Example 6.34.** We could just as well have used [31] an alternative definition, for which we use a different symbol to avoid confusion.

**Base case** $+\!\!+$. $0 +\!\!+ n = n$ and

**Step case** $+\!\!+$. $Sm +\!\!+ n = S(m +\!\!+ n)$

as our definition.[32] Since we are aiming to define the usual addition operator, we expect the result to be commutative, and so it should not matter which argument we recurse over. We show below that this definition does indeed give a commutative operator.

Note that the official definition of addition is the first one given—this is the one you must use in the exercises.

**Code Example 6.10.** We give code for this operation for the class defined above.[33]

```
public static Nat plus (Nat n, Nat n2)
{
    if (n2 == null)
        return n;
    else
        return new Nat (plus(n,n2.next));
}
```

[30] Compare this with the definition of concatenating two lists, Example 6.9.
[31] This is where the situation here differs from the $+\!\!+$ operation for lists.
[32] You may want to compare this definition with that of concatenating two lists, Example 6.9.

Now assume that we had a class nat in Java like **int**, but restricted to the natural numbers. Then you could write the following code to implement addition.

```
public static nat plus (nat n, nat n2)
{
    if (n == 0)
        return n2;
    else
        return 1 + (plus(n−1,n2));
}
```

Obviously one wouldn't write this in practice, but it does work and implements addition.

**Exercise 134.** Assume you would prefer your natural numbers to start with 1 rather than 0.

(a) Give a definition for the natural numbers starting with 1.

(b) Give code for a corresponding class.

(c) Give a definition for addition.

(d) Give the code for an addition operation for your class from part (b).

**Example 6.35.** Once again we may use *induction* to prove properties of the operation just defined. For a very simple example let us prove that for all natural numbers $n$ we have

$$0 + n = n.$$

Here the *base case* is for $n = 0$, and the *step case* is for $Sn$, and we may use the *induction hypothesis* $0 + n = n$ for the latter. This is what the formal proof looks like.

**Base case** $\mathbb{N}$. We note $0 + 0 = 0$ by base case +.

**Ind hyp.** We assume that we have $0 + n = n$.

**Step case** $\mathbb{N}$. We calculate

$$\begin{aligned} 0 + Sn &= S(0 + n) && \text{step case } + \\ &= Sn && \text{induction hypothesis.} \end{aligned}$$

**Tip**

The standard structure for an inductive proof over the natural numbers is the following. Assume we have a statement given in terms of the variable $n$ denoting an element of $\mathbb{N}$.

**Base case.** We show the statement for some number $b$. The base case is the case of the smallest number $b$ for which the given statement holds, and it is obtained by replacing every occurrence of $n$ by that smallest number.

**Ind hyp** We assume that the statement holds for $n$ (in which case it coincides with the

[33] Compare this with the code for concatenating two lists, Code Code Example 6.3.

given statement), or possibly all numbers that are less than or equal to $n$, and greater than or equal to the number $b$ from the base case.

**Step case.** We show the statement for $Sn$ by proving the statement where every occurrence of $n$ has been replaced by $Sn$.

Below we do not give the induction hypothesis explicitly if it coincides with the statement we are proving.

**Example 6.36.** As another example we show that

$$m + Sn = Sm + n. \qquad (*)$$

Often in a statement with two variables where induction is required it is sufficient to carry out the induction over *one* of the variables. Usually the better choice is to *follow the pattern given by the recursive definition*, here the definition of $+$, and so we carry out an induction over the variable $n$.

Note that for more complicated statements it may be necessary to carry out a *double induction* proof, that is, a proof where induction has to be carried out over both arguments.

We begin the proof of statement $(*)$ with the base case, $n = 0$.

**Base case** $\mathbb{N}$. We have

$$\begin{aligned} m + S0 &= S(m + 0) && \text{step case } + \\ &= Sm && \text{base case } + \\ &= Sm + 0 && \text{base case } + . \end{aligned}$$

Next one has to establish the step case, where $Sn$ appears in the place of $n$, and where one may use the *induction hypothesis*, that is the claim $Sm + n = m + Sn$.

**Step case** $\mathbb{N}$. We have

$$\begin{aligned} m + SSn &= S(m + Sn) && \text{step case } + \\ &= S(Sm + n) && \text{induction hypothesis} \\ &= Sm + Sn && \text{step case } + . \end{aligned}$$

What is the point of property $(*)$? It turns out to be very useful, see below, and it will be helpful for you in CExercise 135. It takes a statement where we cannot apply the step case of the definition of $+$ and turns it into a statement where we can. This is how it is used below.

**Example 6.37.** As a more complicated example we show that addition is commutative, that is

$$\text{for all } m, n \in \mathbb{N} \qquad \text{we have} \qquad m + n = n + m.$$

**Base case** $\mathbb{N}$. This is established as part of CExercise 135 (a).

**Step case** $\mathbb{N}$. We have

$$\begin{aligned} m + Sn &= S(m + n) && \text{step case } + \\ &= S(n + m) && \text{induction hypothesis} \end{aligned}$$

$$
\begin{aligned}
&= n + Sm && \text{step case } + \\
&= Sn + m && \text{property } (*).
\end{aligned}
$$

**CExercise 135.** This exercise is concerned with the properties of addition. For this exercise use the definition from Example 6.33—do not switch to the alternative mentioned in Example 6.34. Justify each step.

(a) Show that 0 is a unit for addition on the natural numbers.

(b) Show that $m + 1 = Sm = 1 + m$ for all $m \in \mathbb{N}$, where 1 is a shortcut for $S0$.

(c) Show that addition on the natural numbers is associative.

**Exercise 136.** We have given two potential definitions for addition on the natural numbers in Examples 6.33 and 6.34. Show that they agree. *Hint: You will need properties of addition established in the notes and in the exercises above.*

Defining further operations for the natural numbers, for example multiplication, is now possible. The more complicated the operation, the more complicated is the formal definition likely to be, and the same goes for proving properties of such operations. However, it is possible to use operations previously defined, and so one can build up complicated operations step by step.

**Example 6.38.** The **product** $m \cdot n$ of two natural numbers $m$ and $n$ is defined as follows. For all $m \in \mathbb{N}$ we have

**Base case** $\cdot$. $\quad m \cdot 0 = 0$

**Step case** $\cdot$. $\quad m \cdot Sn = m \cdot n + m.$

Again we could have instead recursed over the first variable instead of doing so for the second.

**Exercise 137.** This exercise is concerned with properties of multiplication. Justify each step.

(a) Show that 1 is a unit for multiplication on the natural numbers.

(b) Show that for all $m \in \mathbb{N}$ we have

$$m \cdot 0 = 0 = 0 \cdot m.$$

(c) Show that for natural numbers $m$ and $n$ we have $Sm \cdot n = m \cdot n + n$.

(d) Show that multiplication for natural numbers is a commutative operation.

(e) Show that we have the following distributivity law for natural numbers:

$$k \cdot (m + n) = k \cdot m + k \cdot n.$$

(f) Show that multiplication for natural numbers is associative.

**CExercise 138.** Carry out the following tasks, justifying each step in your proofs.

(a) Define a function $d$ from $\mathbb{N}$ to $\mathbb{N}$ that doubles its argument, without referring to the operations $+$ or $\cdot$.

(b) Give code that implements this operation for the class nat from Code Code Example 6.9.

(c) Show that for all $n \in N$ we have $dn = SS0 \cdot n$, where the definition of multiplication is given in Example 6.38. *Hint: You may find property* $(*)$ *useful, and you will definitely require addition in this proof.*

(d) Consider the function $\mathrm{mod}_2$ defined as follows:

**Base case** $\mathrm{mod}_2 : 0.$ $\quad \mathrm{mod}_2\, 0 = 0.$

**Base case** $\mathrm{mod}_2 : S0.$ $\quad \mathrm{mod}_2\, S0 = S0.$

**Step case** $\mathrm{mod}_2.$ $\quad \mathrm{mod}_2\, (SSn) = \mathrm{mod}_2\, n.$

How would you describe the action of this function? Why do we need two base cases to define this function? Show that

$$\text{for all } n \in \mathbb{N} \text{ we have} \qquad \mathrm{mod}_2\, dn = 0.$$

**Exercise 139.** For natural numbers $m$ and $n$ define $m^n$. You are allowed to use operations defined above in this chapter. Then prove that for all $n$, $k$ and $l$ in $\mathbb{N}$ we have $n^{k+l} = n^k \cdot n^l$, justifying each step. You may use results from previous exercises as needed.

**Example 6.39.** We can define the *predecessor* $Pn$ of a natural number $n$ as follows.

**Base case** $P.$ $\quad P0 = 0$

**Step case** $P.$ $\quad P(Sn) = n.$

**Code Example 6.11.** Again we give code that implements this operation for the class Nat.

```
public static Nat pred (Nat n)
{
if (n == null)
    return n;
else
    return n.next;
}
```

**Example 6.40.** The notion of a predecessor allows us to define an operation related to subtraction for the natural numbers, which satisfies

$$m \dot{-} n = 0 \qquad \text{if} \qquad m < n,$$

that is we use the value 0 wherever subtraction in the integers would lead to a negative number. In other words we give a recursive definition of the function

$$m \dot{-} n = \begin{cases} m - n & m \geq n \\ 0 & \text{else} \end{cases}$$

**Base case** $\dot{-}$. $\quad m \dot{-} 0 = m$

**Step case** $\dot{-}$. $\quad m \dot{-} Sn = P(m \dot{-} n)$.

Note that there is no sensible way of defining this operation by recursing over the first argument: The reason our definition works is that the result of $m \dot{-} Sn$ is related to that of $m \dot{-} n$: The result of $m \dot{-} Sn$ should be one below that of $m \dot{-} n$, unless the result of $m \dot{-} n$ is 0 already, in which case $m \dot{-} Sn$ must be 0 as well. This is exactly how the predecessor function $P$ works. There is no such simple relation between $Sm \dot{-} n$ and $m \dot{-} n$.

**Optional Exercise 25.** Think about what would be required to define the $\dot{-}$ operator by recursing over the first argument. You may find that you need to define $\leq$ for natural numbers, see Exercise 140.

**Example 6.41.** Recall integer division from page 6. This is more complicated to define formally since we require case distinctions. For natural numbers $m$ and $n$, where $m \neq 0$, we make the following definition.[34]

**Base case** mod. $\quad 0 \bmod m = 0$ and

**Step case** mod.

$$Sn \bmod m = \begin{cases} 0 & S(n \bmod m) = m \\ S(n \bmod m) & \text{else.} \end{cases}$$

**Base case** div. $\quad 0 \operatorname{div} m = 0$.

**Step case** div.

$$Sn \operatorname{div} m = \begin{cases} S(n \operatorname{div} m) & (Sn \bmod m) = 0 \\ n \operatorname{div} m & \text{else.} \end{cases}$$

Again there is a reason we recurse over the first argument: The result of dividing $Sn$ by $m$ is closely related to that of dividing $n$ by $m$, but the result of dividing $n$ by $Sm$ is not close to that of dividing $n$ by $m$. Working with these definitions is quite delicate and proofs of simple statements are quite involved. Note that giving a definition by cases may not be what one would like, and you are asked to find an alternative in EExercise 141.

[34] Note that a special case of this operation appears in CExercise 138.

**Example 6.42.** Note that in the preceding example we are assuming that we know when two elements of $\mathbb{N}$ are equal. We describe here how one may define equality formally. We do this by thinking of it as function which takes two arguments from $\mathbb{N}$ and returns 0 or $S0 = 1$. We want to give a recursive definition of a function which for input $m$ and $n$ gives 1 if and only if $m = n$, and 0 else. This can be done as follows (compare Example 6.14).

| | |
|---|---|
| **Base case** $f_=$. | $f_=(0,0) = S0$. |
| **Step case** $f_=$:$m, 0$. | $f_=(Sm, 0) = 0$. |
| **Step case** $f_=$:$0, n$. | $f_=(0, Sn) = 0$. |
| **Step case** $f_=$:$m, n$. | $f_=(Sm, Sn) = f_=(m, n)$. |

Here you can see how recursion over two arguments works, but note that this is a particularly simple case. The code from Code Example 6.14 can be adjusted to implement this function for the class Nat.

Note that we have not said which source and target is intended for this function. Clearly the source is meant to be $\mathbb{N} \times \mathbb{N}$, but there are at least two sensible target sets: One might pick $\{0, 1\}$, or one might pick $\mathbb{N}$ again. It is the latter option that you are expected to use in EExercise 141.

**Exercise 140.** Similar to the definition of $f_=$ above give a definition of $f_\leq$, with $f_\leq(m, n) = 1$ if and only if $m \leq n$.

It is possible to combine $f_=$ with the previously defined operations $\dot{-}$ and $\cdot$ to give a definition of $\mathrm{mod}$ which does not require the use of a definition by cases, see the following exercise.

**EExercise 141.** Justifying each step show the following statements by induction.

(a) $0 \dot{-} n = 0$ for all $n \in \mathbb{N}$.

(b) $P(Sn) = n$ for all $n \in \mathbb{N}$.

(c) $P(Sn \dot{-} m) = n \dot{-} m$ for all $m, n \in \mathbb{N}$.

(d) $n \dot{-} n = 0$ for all $n \in \mathbb{N}$.

(e) Give a definition of $\mathrm{mod}$ which does not use a case distinction, but which instead uses the function $f_=$ from Example 6.42, viewed as a function from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$. Argue that your definition agrees with the one from the notes.

(f) Show that $n \bmod S0 = 0$ for all $n \in \mathbb{N}$. *Hint: Use the definition of* $\mathrm{mod}$ *from the previous part.*

*Hint: You may use statements from previous parts even if you have not proved them.*

**Optional Exercise 26.** If you are looking for a more challenging proof, try to show that $n \bmod n = 0$ for all $n \in \mathbb{N}$, or that $n \operatorname{div} 1 = n$ for all $n \in \mathbb{N}$.

The material in this section to this point gives us a rigorous definition of the natural numbers, as well as definitions of commonly used operations on them. This may appear somewhat tedious,

covering only things you already knew. But mathematics is all about building everything from first principles, and doing so in a rigorous way. One might continue the development sketched above until one has assembled all the information about the operations on the natural numbers that is summarized in Chapter 0, but we don't have the time for this. Hopefully the above gives you a flavour of how this might work.

### 6.4.3 Advanced operations for natural numbers

From here on we go back to using the usual names for natural numbers, so we write 1 instead of $S0$, $n+1$ instead of $Sn$, and so on. The step case in an induction proof is now the more familiar step from $n$ to $n+1$. Note that in some of the examples below we use induction hypotheses which are more complicated than assuming that the given statement holds for some number $n$ and then proving that this implies it holds for $n+1$. In these cases we make the induction hypothesis explicit.

From here on we also use the usual operations for natural numbers without referring to their formal recursive definitions. But you should still think about the properties you use, and make them explicit in your proofs.

**Example 6.43.** A standard example given to introduce the idea of a recursively defined function is that of the *factorial function*. We define this operation as follows.

**Base case** !. $0! = 1$.

**Step case** !. $(n+1)! = (n+1) \cdot n!$.

Turned into Java code (for integers) this looks something like this:

```
public static int factorial (int n);
{
    if (n==0)
        return 1;
    else
        return n * factorial(n−1);
}
```

Note that we are explicitly carrying out a test:

- If we are in the base case then we return the appropriate number,
- else we know that we are in a case were $n$ is of the form $m+1$, and so it is safe to subtract one from $n$, and stay in $\mathbb{N}$.

Also note that this code makes no attempt to determine whether the argument is a negative number. You may want to think about what happens if it is called for a negative number n.

But apart from these minor differences the code is a very straightforward translation of the mathematical definition. Understanding how the mathematics works therefore can help you write recursive programs.

You might note that $1! = 1$, and so if $n = 1$ there's really no need to do a recursion step. You can therefore make the code more efficient (but maybe less clear) by writing instead:

```
public static int factorial (int n);
{
    if (n <= 1)
        return 1;
    else
        return n * factorial(n−1);
}
```

When it comes to calculating the result of reasonably complex operations on natural numbers recursion can be a useful tool.

### Euclid's algorithm

Given two natural numbers $m$ and $n$ we can find the largest number dividing both of them, their *greatest common divisor*.

**Example 6.44.** The following is known as *Euclid's algorithm*. It appears in COMP26120 as an example algorithm.

Assume that $a \leq b$ are natural numbers. We set

$$r_0 = b \qquad \text{and} \qquad r_1 = a.$$

By Fact 2 from Chapter 0 we can find natural numbers $k_1$ and $r_2 < r_1$ with the property that

$$b = r_0 = k_1 r_1 + r_2 = k_1 a + r_2.$$

We keep applying this idea and, invoking Fact 2, we define

$$r_{i+2} \text{ with } 0 \leq r_{i+2} < r_{i+1} \qquad \text{and} \qquad k_{i+1}$$

to be the unique numbers with the property that

$$r_i = k_{i+1} r_{i+1} + r_{i+2}.$$

In other words, we have

$$k_{i+1} = r_i \operatorname{div} r_{i+1} \qquad \text{and} \qquad r_{i+2} = r_i \bmod r_{i+1}.$$

Note that we have that

$$r_0 > r_1 > \cdots$$

Any strictly descending sequence of natural numbers must be finite. We may apply Fact 2 and construct new elements for the sequence until we get the number 0, let's say when we reach $r_{n+1}$, so that

$$r_{n-1} = k_n r_n + 0.$$

The number $r_n$ plays a particular role for $a$ and $b$.

We claim that $r_n$ is the *greatest common divisor* of $a$ and $b$. More specifically we show that

- the number $r_n$ divides both, $a$ and $b$ and
- if $c$ divides both, $a$ and $b$ then $c$ divides $r_n$.

These proofs are carried out by induction. To prove the first claim we show that

$$\text{for all } 0 \leq i \leq n \qquad r_n \quad \text{divides} \quad r_i.$$

This means that in particular, $r_n$ divides both, $r_1 = a$ and $r_0 = b$.

Note that this proof is a bit different from other induction proofs we have seen in that

- It proceeds backwards from $n$ down to 0.
- It requires two base cases since the step case requires that the claim holds for the two previously considered numbers.

For this reason we state the induction hypothesis explicitly.

**Base case** $n$. Obviously $r_n$ divides $r_n$.

**Base case** $n-1$. Since $r_{n-1} = k_n r_n$ we have that $r_n$ divides $r_{n-1}$.

**Ind hyp**. We know that $r_n$ divides $r_{i+1}$ and $r_{i+2}$.

**Step case**. We show that $r_n$ divides $r_i$. By construction we have

$$r_i = k_{i+1} r_{i+1} + r_{i+2}.$$

Since by induction hypothesis we know that

$$r_n \text{ divides } r_{i+1} \qquad \text{and} \qquad r_n \text{ divides } r_{i+2}$$

we can find natural numbers $l_{i+1}$ and $l_{i+2}$ with the property that

$$r_{i+1} = l_{i+1} r_n \qquad \text{and} \qquad r_{i+2} = l_{i+2} r_n,$$

and so

$$\begin{aligned} r_i &= k_{i+1} r_{i+1} + r_{i+2} \\ &= k_{i+1} l_{i+1} r_n + l_{i+2} r_n \\ &= (k_{i+1} l_{i+1} + l_{i+2}) r_n, \end{aligned}$$

which means that

$$r_n \text{ divides } r_i.$$

We leave the proof of the second claim as an exercise.

In COMP26120 you will think about the complexity of such algorithms, that is, you will worry about how many $r_i$ have to be calculated until the greatest common divisor has been reached.

**Exercise 142.** Show the second claim from the preceding example.

Recursion can be used beyond defining operations on natural numbers.

**Example 6.45.** We can also use recursion to define subsets of sets already defined. For example, the set of even numbers (compare Examples 0.10 and 0.18) $M_2$ can be defined to be the smallest set satisfying the following conditions.

**Base case** $M_2$. $0 \in M_2$.

**Step case** $M_2$. $n \in M_2$ implies $2 + n \in M_2$.

The base case tells us that $2 \cdot 0 = 0$ is in $S$, and using the step case we get

$$0 + 2 = 2 = 2 \cdot 1 \in S,$$

and applying the step case again we get

$$2 + 2 = 4 = 2 \cdot 2 \in S,$$

and so on. Note that we do not have to refer to multiplication in this definition—all we have to do is keep adding 2.

**Example 6.46.** For another example, to define a set $P_2 \subseteq \mathbb{N}$ containing exactly the powers of 2 we can use the following.

**Base case** $P_2$. $1 \in P_2$.

**Step case** $P_2$. $n \in P_2$ implies $2 \cdot n \in P_2$.

The base case tells us that $2^0 = 1$ is in $S$, and using the step case we get

$$1 \cdot 2 = 2 = 2^1 \in S,$$

and applying the step case again we get

$$2 \cdot 2 = 4 = 2^2 \in S,$$

and so on. Note that we do not have to refer to exponentiation in this definition—all we have to do is keep multiplying with 2.

**Exercise 143.** Give a recursive definition for each of the following sets.

(a) The set of odd natural numbers as a subset of the natural numbers.

(b) The set of all non-empty lists over $\mathbb{N}$ for which the $n$th element (from the right), for $n \geq 2$, if it exists is equal to twice the previous entry.

(c) The set of all non-empty lists over $\mathbb{N}$ for which the $n$th symbol (from the right), if it exists,

- for $n = 2$ is twice the first element and

- for $n \geq 3$ is is equal to the sum of the previous entries.

(d) The set of full binary trees such that each label of a node that is not a leaf is the sum of the labels of its children.

(e) The set of full binary trees such that each label of a node that is not a leaf is the sum of the leaves below it.

*Hint: You may use previously defined operations. If you aren't sure what the step case might look like, construct some lists or trees that satisfy the given criteria.*

**Example 6.47.** We give a definition of the the $\sum$ operator which makes an appearance in Chapter 4. Assume that $N$ is a set of numbers. Assume that $n$ is a natural number and that $a_i \in N$ for $1 \leq i \leq n$, The operation

$$\sum_{i=1}^{n} a_i$$

is recursively defined as follows.

**Base case** $\sum$. $\sum_{i=1}^{0} a_i = 0$

**Step case** $\sum$. $\sum_{i=1}^{n+1} a_i = (\sum_{i=1}^{n} a_i) + a_{n+1}$.

This definition says that the base case consists of summing up no elements at all, in which case the definition needs to return the unit 0 of the addition operation in order for the step case to work correctly.

For some sums it is possible to find a simplification, and in those cases a proof that this works is usually by induction.

**Example 6.48.** We show that for all $n \in \mathbb{N}$,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

**Base case** $\mathbb{N}$. We have

$$\begin{aligned} \sum_{i=1}^{0} i &= 0 && \text{base case } \sum \\ &= \frac{0 \cdot 1}{2} && 0 \cdot n = 0 \text{ in } \mathbb{N}. \end{aligned}$$

**Step case** $\mathbb{N}$. For this case we have

$$\begin{aligned} \sum_{i=1}^{n+1} i &= \sum_{i=1}^{n} i + (n+1) && \text{step case } \sum \\ &= \frac{n(n+1)}{2} + (n+1) && \text{ind hyp} \\ &= \frac{n(n+1)}{2} + \frac{(n+1)2}{2} && 2/2 = 1, 1 \text{ unit for mult} \\ &= \frac{(n+1)n + (n+1)2}{2} && \text{distr and comm of mult} \end{aligned}$$

$$= \frac{(n+1)(n+2)}{2} \qquad \text{distr for mult.}$$

Note that sometimes sums start at 0 instead of at 1, such as in the exercise below. In that case you have

$$\sum_{i=0}^{n} a_i = \sum_{i=1}^{n+1} a_{i-1},$$

which is equivalent to the definition below:

**Base case** $\sum$. $\sum_{i=0}^{0} a_i = a_0$

**Step case** $\sum$. $\sum_{i=0}^{n+1} a_i = (\sum_{i=0}^{n} a_i) + a_{n+1}$.

In general, it is always possible to use a base case where instead of adding up no elements at all, we take the first element of our sum. We call this the **alternative base case**. The reason we give the more general definition above is that sometimes, when one derives a formula, the base case of the empty sum does occur naturally.

**Example 6.49.** For the same expression as in the previous example, it is also possible to construct a proof by induction as follows:

**Base case.** We have

$$\begin{aligned} \sum_{i=1}^{1} i &= 1 && \text{alt base case } \sum \\ &= \frac{1 \cdot 2}{2} && \text{arithmetic.} \end{aligned}$$

**Step case.** For this case we have

$$\begin{aligned} \sum_{i=1}^{n+1} i &= \sum_{i=1}^{n} i + (n+1) && \text{step case } \sum \\ &= \frac{n(n+1)}{2} + (n+1) && \text{ind hyp} \\ &= \frac{n(n+1)}{2} + \frac{(n+1)2}{2} && 2/2 = 1, 1 \text{ unit for mult} \\ &= \frac{(n+1)n + (n+1)2}{2} && \text{distr and comm of mult} \\ &= \frac{(n+1)(n+2)}{2} && \text{distr for mult.} \end{aligned}$$

**CExercise 144.** Show the following statements by induction.

(a) For $r \in \mathbb{R} \setminus \{1\}$ and $n \in \mathbb{N}$ we have $\sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}$.

(b) For $n \in \mathbb{N}$ we have $\sum_{i=0}^{n} i(i+1) = \frac{n(n+1)(n+2)}{3}$.

(c) For $n \in \mathbb{N}$ we have $\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$.

(d) For $n \in \mathbb{N}$ we have $\sum_{i=0}^{n} \frac{1}{2^i} = \frac{2^{n+1} - 1}{2^n}$.

(e) For $n \in \mathbb{N}$ we have $\sum_{i=0}^{n} i!i = (n+1)! - 1$.

(f) For $n \in \mathbb{N}$ we have $1 + \sum_{i=0}^{n} 2^i = 2^{n+1}$.

### 6.4.4 Combinatorial rules

Looking back to the formulae for counting combinations that appear in Section 4.1.2 we can see that these can be shown to do what they are supposed to do by using inductive arguments.

**Selection with return.**

Consider a situation where we have $n$ items to randomly pick from, returning them each time. The claim we have from Section 4.1.2 is that there are

$$n^i$$

combinations when drawing $i$ times. Can we use induction to create a formal argument that this is indeed so?

**Base case** $N$. If there is no draw there is one possible outcome.[35]

**Step case** $N$. If there are $i+1$ draws then by the induction hypothesis there are $n^i$ many outcomes from the first $i$ draws. Each of those has to be combined with the $n$ possible outcomes of the $(i+1)$th draw, which means there are

$$n^i \cdot n = n^{i+1}$$

many outcomes for $i+1$ draws.

**Selection without return**

If we have $n$ items to pick from then according to the formula from Section 4.1.2 for $i$ draws without return there are

$$\frac{n!}{(n-i)!}$$

many combinations. Again we want to give a proof.

**Base case** $N$. If there is no draw there is one possible outcome.[34]

**Step case** $N$. If there are $i+1$ draws then by the induction hypothesis there are

$$\frac{n!}{(n-i)!}$$

many outcomes from the first $i$ draws. Since $i$ items have been removed there are $n-i$ possibilities for the $(i+1)$th draw, each of which has to be combined with every outcome from the previous draws, giving

$$\frac{n!}{(n-i)!} \cdot (n-i) = \frac{n!}{(n-i-1)!} = \frac{n!}{(n-(i+1))!}$$

many outcomes

**Unordered selection without return**

We leave the remaining case as an exercise.

[35] If you prefer you can start with 'If there is one draw there are $n$ possible outcomes by assumption'.

**Exercise 145.** For unordered selection without return prove by induction that the number of possible outcomes is

$$\frac{n!}{(n-i)!i!}.$$

### 6.4.5 Functions given via recursive specifications

When trying to compute the complexity of a recursive algorithm, that is trying to see how the number of steps changes as the problem size grows, one often ends up having to solve a *recurrence relation*. This means that one has a recursive specification of a function, and one would like to find a simpler description of that function (using known functions).

**Example 6.50.** Assume we have a program whose complexity we are trying to work out as a function $f$ which takes as its input the problem size $n$, and gives as its output the number of steps (or an approximation thereof) the program will take when given an input of size $n$. Further assume that by studying the program we have worked out that the function $f$ has to satisfy the following specification:

**Base case** $f$. $f0 = 1$, that is, if the problem size is 0 the program takes one step and

**Step case** $f$. $f(n+1) = kfn$, for some $k \in \mathbb{N}$, that is, if the problem size grows by one the program needs $k$ times the number of steps from before.

Working with this specification is unwieldy. In this case it's quite easy to give an alternative description of $f$, but let's see how one might arrive there. It's probably easiest to first of all compute the first few values.

| $n$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $fn$ | 1 | $k$ | $k^2$ | $k^3$ | $k^4$ |

We might now guess that $fn = k^n$, but we really should verify that this definition is mandated by the original description.

We show that if $f$ is a function satisfying the given specification then it has to be defined by the formula given. As with most recursive statements the obvious way of proving this is by induction.

**Base case** $f$. $f0 = 1 = k^0$, so we have a match.

**Step case** $f$.

$$\begin{aligned} f(n+1) &= kfn && \text{specification } f \\ &= k \cdot k^n && \text{induction hypothesis} \\ &= k^{n+1} && \text{calcs in } \mathbb{N}, \end{aligned}$$

and this gives another match.

**Example 6.51.** Let's do a more complicated example.

**Base case** $g$:0. $g0 = 0$.

**Base case** $g$:1. $g1 = 1$.

**Step case** $g$. $g(n+2) = 2g(n+1) - gn + 2$.

Note that we have two base cases here.They are required because the step case relies on *two* previously calculated values. Again we work out the first few values.

| $n$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $gn$ | 0 | 1 | 4 | 9 | 16 |

Once more there's an obvious guess: We guess that $gn = n^2$. Again we have to provide a proof. Note that we have to cover two base cases now, and that the induction hypothesis is now: $gk = k^2$ for all $k < n+2$. In particular we use below the induction hypothesis that

$$g(n+1) = (n+1)^2 \qquad \text{and} \qquad gn = n^2.$$

**Base case** $g$:0. $g0 = 0 = 0^2$.

**Base case** $g$:1. $g1 = 1 = 1^2$.

**Ind hyp** $g$. We have

$$gn = n^2 \qquad \text{and} \qquad g(n+1) = (n+1)^2.$$

**Step case** $g$.

$$\begin{aligned} g(n+2) &= 2g(n+1) - gn + 2 && \text{spec } g \\ &= 2(n+1)^2 - n^2 + 2 && \text{ind hyp} \\ &= 2n^2 + 4n + 2 - n^2 + 2 && \text{calcs in } \mathbb{N} \\ &= n^2 + 4n + 4 && \text{calcs in } \mathbb{N} \\ &= (n+2)^2 && \text{calcs in } \mathbb{N}. \end{aligned}$$

You might think that above we only show that a function that satisfies the given specification must be given by the formula provided, but our proof also includes an argument that a function given by that formula satisfies the specification:

- For the base case(s) this is part of what we establish.
- For the step case the proof takes the form that
  - we use the specification to rewrite the function in terms of its values at smaller inputs,
  - we use the induction hypothesis to replace those with the appropriate instances of the given formula, and
  - we show then that this gives the formula for the appropriate input.

  The part from the expression derived using the induction hypothesis to the end establishes the desired equality.

For this reason you do not have to prove separately that your formula satisfies the given specification if you prove that it is the only formula that does so.

In general, if our recursive specification is such that to calculate the value of our function $f$ at $n$, we only rely on the values $f$ takes for inputs smaller than $n$, there is a unique function specified. However, it is not hard to write down specifications that have more than one solution,

or no solution at all, and this idea is explored a bit in the '*Looking for more to do*' part of Practice Sheet 14.

**CExercise 146.** For the following recursive specifications, work out an alternative *non-recursive* representation of the given function and prove by induction that it satisfies the given specification.

(a) **Base case** $f$. $f0 = 0$.

**Step case** $f$. $f(n+1) = fn + 2$.

(b) **Base case** $f$. $f0 = 1$.

**Step case** $f$. $f(n+1) = (n+1)fn$.

(c) **Base case** $f$. $f0 = 2$.

**Step case** $f$. $f(n+1) = (fn)^2$.

(d) **Base case** $f$:0. $f0 = 0$.

**Base case** $f$:1. $f1 = 3$.

**Step case** $f$. $f(n+2) = 2f(n+1) - fn$.

(e) **Base case** $f$:0. $f0 = 1$.

**Base case** $f$:1. $f1 = 1$.

**Step case** $f$. $f(n+2) = 2f(n+1) - fn$.

(f) **Base case** $f$:0. $f0 = 1$.

**Base case** $f$:1. $f1 = 2$.

**Step case** $f$. $f(n+2) = f(n+1) + 2fn$.

(g) **Base case** $f$:0. $f0 = 1$.

**Base case** $f$:1. $f1 = 3$.

**Step case** $f$. $f(n+2) = 2f(n+1) + 3fn$.

Note that the examples given in these notes have been carefully chosen so that it is possible to guess a *closed form* for the underlying function. Not all recurrence equations that appear in real-world programs are as easy as this. An example is provided by the following optional exercise. Typically recursive specifications arise when one wants to compute the complexity of a recursive program, and often those occurring in practice are much harder to solve than our examples.

**Optional Exercise 27.** A popular exercise in writing recursive programs is to compute the Fibonacci sequence.[36] Consider the following recursive specification for a function.

**Base case** $f$:0. $f0 = 0$.

**Base case** $f$:1. $f1 = 1$.

**Step case** $f$. $f(n+2) = f(n+1) + fn$.

(a) Calculate the first few values of $f$.

(b) Show that for

$$\alpha = \frac{1+\sqrt{5}}{2} \qquad \text{and} \qquad \beta = \frac{1-\sqrt{5}}{2}$$

we have

$$\alpha^{n+2} = \alpha^{n+1} + \alpha^n \qquad \text{and} \qquad \beta^{n+2} = \beta^{n+1} + \beta^n,$$

so these numbers satisfy the equations defining $f$.

(c) Show that the function

$$\begin{aligned} fn &= \frac{\alpha^n - \beta^n}{\alpha - \beta} \\ &= \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}} \\ &= \frac{(1+\sqrt{5})^n - (1-\sqrt{5})^n}{2^n\sqrt{5}} \end{aligned}$$

satisfies the original condition for $f$.

Would you have guessed this definition for $f$?

**Optional Exercise 28.** Note that it is possible to have recurrence relations with more than one variable. A typical example are the *binomial coefficient*, which can be thought of as given by the recurrence relation

**Base case** binom. $\binom{n}{0} = 1$.

**Step case** binom. $\binom{n+1}{i+1} = \binom{n}{i} + \binom{n}{i+1}$.

Show that this gives the same definition as the more common

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}.$$

Note that the recursive definition does not require multiplication, just addition. You may have seen this definition at work in *Pascal's triangle*.

## 6.5 Further properties with inductive proofs

Many properties involving natural numbers can be shown by induction. The following example and exercises gives a taste of those.

**Example 6.52.** Here is an inductive proof of a property of complex numbers, but invoking natural numbers where it counts. We show that

$$|z| = 1 \qquad \text{implies} \qquad \text{for all } n \in \mathbb{N},\ |z^n| = 1.$$

[36] Check out https://en.m.wikipedia.org/wiki/Fibonacci_number to read up on where this sequence originates and some of its interesting properties.

**Base case** $\mathbb{N}$. We have $|z^0| = |1| = 1$.

**Step case** $\mathbb{N}$. For this case we have

$$\begin{aligned} |z^{n+1}| &= |z^n z| && \text{def } (-)^{n+1} \\ &= |z^n||z| && |zz'| = |z||z'|, \text{ Ex 16} \\ &= 1 \cdot 1 && \text{ind hyp, assumptn} \\ &= 1 && 1 \text{ unit for mult} \end{aligned}$$

**Exercise 147.** Show the statements from Fact 8 on page 44.

**EExercise 148.** The following statements can be shown by induction but some creativity is required to see how the proof might work.

(a) Show that if we have 4p and 5p stamps then they can be combined to get any amount from 12p up. In other words we want to establish that every natural number greater than or equal to 12 is the sum of multiples of the numbers 4 and 5, where we only allow natural numbers as multipliers (the question gets very easy when we allow integers). *Hint: Work out solutions for smaller numbers, up to 24 or so. A pattern should emerge. You will require a number of case distinctions.*

(b) Show that for all natural numbers $n \neq 0$ we have that $7^n - 1$ is divisible by 6, as is $n^3 - n$.

(c) Assume that $n \in \mathbb{N}$ and $n \geq 2$. Show that a set of size $n$ has $n(n-1)/2$ two-element subsets. *Hint: This is an exercise that wants to be solved by reasoning in English, rather than manipulating some formula.*

(d) Assume that $n, m \in \mathbb{N}\backslash\{0\}$ and that $n \geq m$. Show that a set of size $n$ has $n!/(n-m)!m!$ many $m$-element subsets. *This is a generalization of the previous part.*

After all this you may wonder what '*recursion theory*' is all about. It is the study of those functions $\mathbb{N}^n \longrightarrow \mathbb{N}$ which can be defined using recursion. This is more involved than you may think—because one may use previously defined functions to define more sophisticated ones the functions that can arise in this way are considerably more complicated than those that appear in this section.

## 6.6 More on induction

The examples given above are comparatively simple cases of proofs by induction. We briefly discuss situations where a proof becomes more complicated.

- The induction hypothesis in most of our examples is very straightforward in the sense that for each step case, we merely require the statement for each 'ingredient'. For example, in the case of a propositional formula, assuming the statement holds for the formula $A$ and the formula $B$ we show that it holds for $A \wedge B$. Sometimes one has to require

the statement for 'all ingredients built so far'. In the above example this would mean assuming the statement for $A$, $B$, and all their subformulae.

- In some cases it is not sufficient to assume the given claim for all entities built so far'. Instead one finds that when trying to prove the step case that one needs a *stronger* statement than the one that is at issue. This often gives some idea of an alternative, stronger statement that can be shown by induction. This is sometimes called *strengthening the induction hypothesis*. We have not seen any examples of this.

- It is perfectly possible to *nest* induction proofs, that is, inside a proof by induction one requires another proof, also by induction, of an unrelated statement. We have not seen examples of this.

- If one tries to show a statement which contains more than one variable one of the following cases will apply if the statement is provable by induction at all:

  - We may prove the statement by induction over one of the variables, treating the others as parameters. Examples of this are Examples 6.12 and 6.36.
  - We may prove the statement by giving an induction proof for one of the variables, which inside contains induction proofs for the other variables (where one has to be careful to state the various induction hypotheses carefully to ensure that they do not assume anything unwarranted). In the case where there are two variables this is known as *double induction*. We have not seen examples of this.

# Chapter 7

# Relations

So far when trying to connect two sets we have only looked at functions. This assumes that we have a mechanism for turning elements from the source set into elements of the target set. However, there are other ways of making connections.

**Example 7.1.** In a database we typically want to think of tables, and indeed people often talk about 'relational databases'. These can be viewed as relations of a general kind. For a simple example, think of a library which has a table of members (uniquely determined by their membership numbers), a table of books (uniquely determined by catalogue numbers), and a table which keeps track of which book is currently on loan to which member. We can think of this as connecting the set of all members $M$ (represented by the set of all valid membership numbers) to the set of all books $B$ (represented by the set of all current catalogue numbers).

This connection cannot be thought of as a function: A given member may have no, or several, books on loan, so we cannot produce a unique output for each input. Instead we can think of this table as a subset $L$ of the product $M \times B$ with the property that

$$(m, b) \in L \qquad \text{if and only if} \qquad \text{member } m \text{ currently has book } b \text{ on loan.}$$

This is an example of a relation from $M$ to $B$.

In terms of a database one would describe this as a *relation schema*, and would give a type for it along the following lines:

OnLoan(member:**int**,book:**int**),

assuming that membership and catalogue numbers are implemented as integers. The entries in the database are exactly the members of the relation.

In these notes we largely restrict ourselves to relations connecting two sets, which means relation schema with two entries. Databases often have relation schemas with more entries, for example a database that keeps track of members of the university might have a relation schema including

(title,name,building,office number,phone number).

The general ideas regarding relations that are introduced below apply to this kind of situation as well.

## 7.1 General relations

We use relations all the time, even though you may not have been aware of that.

It is often convenient to think of relations as generalizations of functions, and sometimes a similar notation is used.

A **relation** $R$ **from a set** $S$ **to a set** $T$ is given by a subset of $S \times T$. This is sometimes written as

$$R\colon S \longrightarrow\!\!\!\!\!\!+\;\; T,$$

but this is not universal.

A function is a special kind of relation: Given a function $f\colon S \longrightarrow T$ we have its graph

$$\{(s, fs) \in S \times T \mid s \in S\},$$

which is a subset of $S \times T$. It is standard to identify $f$ with its graph to view it as a relation. Proposition 2.2 tells us which relations are the graphs of functions, namely those relations $R \subseteq S \times T$ where for every $s \in S$ there is a unique $t \in T$ such that $(s, t) \in R$.

Examples of relations are abundant.

**Example 7.2.** We give a few examples of relations from a number of areas.

(a) The relation from the set of students in the Department to the set of academics where a student is related to an academic if during the current academic year the student is enrolled on a course unit on which the academic teaches.

(b) The relation from the set of students in the Department to the set of COMP course units offered which relates a student to all the course units he or she is enrolled on.

(c) The relation from the set of real numbers to the set of real numbers where $x$ is related to $y$ if $x = y^2$.

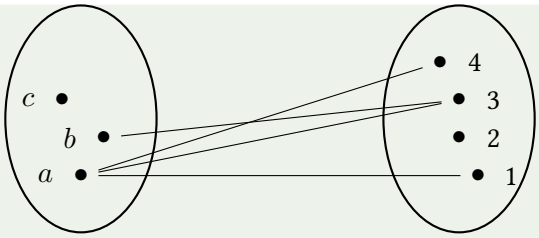
(d) The relation between Java programs where one program is related to another if they can be viewed as computing the same thing.[1]

(e) The relation of equality for a number of entities, for example, equality of numbers, equality of fractions, and more generally the equality of 'arithmetic expressions', that is, expressions written using numbers and operations such as addition, multiplication and inverses with respect to these. We usually consider two such expressions to be equivalent (or equal) if they evaluate to the same number.

We can picture relations between small finite sets in a picture, similar to how we draw functions between such sets.

**Example 7.3.** We show how to picture a small relation between two different sets. For small relations on the same set see Section 7.3.1.

[1]Defining this in general in a rigorous way is non-trivial, but it is fairly easy if one only looks at Java programs which can be thought of as having a natural number as input and a natural number as output.




This relation goes from the set $\{a, b, c\}$ to the set $\{1, 2, 3, 4\}$. It relates

- $a$ to 1, 3 and 4,
- $b$ to 3, and
- $c$ to no element.

We typically write this relation as the collection of pairs from the set

$$\{a, b, c\} \times \{1, 2, 3, 4\}$$

which it contains, which in the case of the example above is

$$\{(a, 1), (a, 3), (a, 4), (b, 3)\}.$$

### 7.1.1 Important notions

There are two common notations for relations. One is to rely on the idea that a relation $R$ from $S$ to $T$ is a subset of $S \times T$ and so to denote the fact that $R$ relates an element $s$ of $S$ to an element $t$ of $T$ by

$$(s, t) \in R.$$

Sometimes infix notation is preferred, and instead of $(s, t) \in R$ one might write

$$s \; R \; t.$$

Where infix notation is used it is not unusual to see symbols, rather than letters, to denote a relation; for example you may find a relation $\sim$ from $S$ to $T$ where the same fact is written as

$$s \sim t.$$

**Example 7.4.** An example of a relation that is typically written in this manner this is *equality of arithmetic expressions.* To denote that the two expressions $2/4$ and $1/2$ denote the same number in $\mathbb{Q}$ or $\mathbb{R}$ we write

$$2/4 = 1/2;$$

the equal symbol $=$ being written in infix notation.

**Example 7.5.** Another example is the notion of *semantic equivalence* of propositions from the material on logic, where the notation

$$A \equiv B$$

is used to denote the fact that with respect to every valuation $A$ has the same boolean

interpretation as $B$.

Both notations are routinely used and you should become comfortable with both.

Each set has a special relation: Given a set $S$ the **identity relation** $I_S$ **on** $S$ is given by

$$\{(s, s) \in S \times S \mid s \in S\}.$$

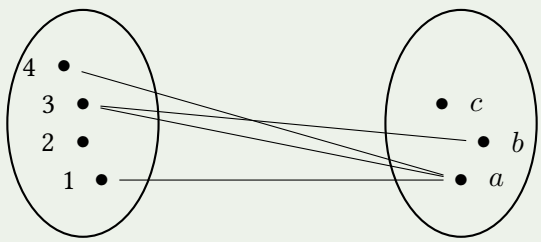
In other words, every element is related to itself, and to nothing else. The identity relation on $S$ is the graph of the identity function $\mathsf{id}_S$ on $S$.

Given a relation $R$ from $S$ to $T$ there is an easy way of turning it into a relation from $T$ to $S$: The **opposite relation** $R^{\mathsf{op}}$ **of** $R$ is given by

$$R^{\mathsf{op}} = \{(t, s) \in T \times S \mid (s, t) \in R\}.$$

In other words we 'turn the relation around' by changing the order of the pairs, and also switch the 'source' and 'target'.

**Example 7.6.** For the relation given in Example 7.3 above the opposite relation is given by the following picture.




**Example 7.7.** Consider the relation 'is a child of' as a relation on the product with itself of the set of all people, where

$$(a, b)$$

being an element of the relation means that $a$ is a child of $b$. The opposite of the 'is a child of' relation is the 'is a parent of' relation, where

$$(a, b)$$

is in the relation if and only if $a$ is a parent of $b$.

Note that since relations are sets we can apply set operations to them. In particular, given a relation $R$ from $S$ to $T$ there is its complement,

$$(S \times T) \setminus R,$$

and given two relations $R$ and $R'$ from $S$ to $T$ we may form

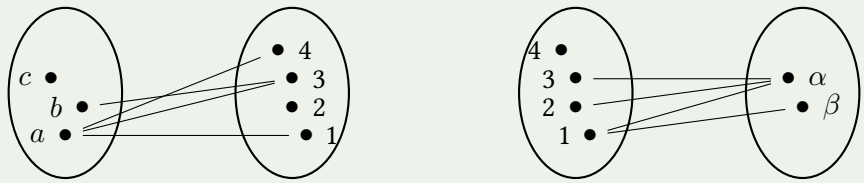
$$\text{their union } R \cup R' \qquad \text{and their intersection } R \cap R'.$$

Relations are a little like functions in that one can define their composition.

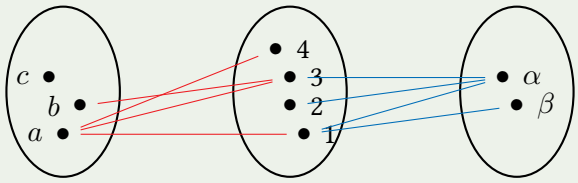
Let $R$ be a relation from $S$ to $S'$, and let $R'$ be a relation from $S'$ to $S''$. The **relational composite** $R \,;\, R'$ is given by

$$R \,;\, R' = \{(s, s'') \in S \times S'' \mid \exists s' \in S'. ((s, s') \in R \text{ and } (s', s'') \in R')\}.$$

**Example 7.8.** Consider the two relations given by the following picture.

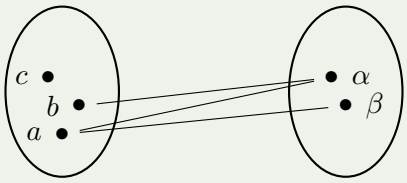



If we 'overlay' the two we can more easily see what the composite of the two relations is.




The composite connects

- a node in the left-most set with
- a node in the right-most set if and only if
- they are connected by a line through the set in the middle (that is a red line followed by a blue one).




**Example 7.9.** Consider the 'is a child of' relation from Example 7.7. We may form the relational composite of this relation with itself, and the result is the 'is a grandchild of' relation.

**Example 7.10.** Assume we have two relation schema (compare Example 7.1) in a database,

- one connecting members (given by their membership number) with borrowed books (given by their catalogue number) and
- one connecting books (given by catalogue number) and their titles.

If one wants to create a new relation scheme that connects members (given by their membership number) with the titles of the books they have on loan one has to form the relational composite of the two underlying relations.

On the left we have a table describing the relations between members and the catalogue numbers of the books they have on loan, and on the right a table describing the title of the book corresponding to a catalogue number.

| | |
|---|---|
| Member 1 | 00002 |
| Member 1 | 00003 |
| Member 3 | 00005 |
| Member 3 | 00007 |
| Member 3 | 00011 |

| | |
|---|---|
| 00002 | *The Joys of Java* |
| 00003 | *Maths for Dummies* |
| 00005 | *Higher Category Theory* |
| 00007 | *Topoi and Theories* |
| 00011 | *Multiversal Algebra* |

The relational composite of the underlying relations, written again as a table, is the following:

| | |
|---|---|
| Member 1 | *The Joys of Java* |
| Member 1 | *Maths for Dummies* |
| Member 3 | *Higher Category Theory* |
| Member 3 | *Topoi and Theories* |
| Member 3 | *Multiversal Algebra* |

But note that this is a special example, which makes it look as if every entry in the first table gives rise to an entry in the resulting table. This is the case because our second table defines a very special relation, namely one that is functional: For every catalogue number there exists exactly one book title (namely the title of the corresponding book). Moreover, since every catalogue number can be on loan to at most one person, the first relation is also special.

**Example 7.11.** Assume we have a different database. We have a relation schema connecting applicants to the Department with their computer science interests, and another relation schema which suggests related interests.

| | |
|---|---|
| Wong | AI |
| Wong | HCI |
| Kim | Maths |
| Anna | AI |
| Anna | Hardware |

| | |
|---|---|
| AI | Machine Learning |
| AI | Logic Programming |
| AI | Knowledge Representation |
| Security | Encryption |
| Maths | Logic |
| Maths | Probabilities |
| Hardware | Circuits |

The relational composite of the underlying relations, written again as a table, is the following:

| | |
|---|---|
| Wong | Machine Learning |
| Wong | Logic Programming |
| Wong | Knowledge Representation |
| Kim | Logic |
| Kim | Probabilities |
| Anna | Machine Learning |
| Anna | Logic Programming |
| Anna | Knowledge Representation |
| Anna | Circuits |

It connects students with potential interests.

**Example 7.12.** In COMP11212 the notion of a *simulation* between finite state automata is introduced. This is a relation that connects the states of the two automata. Only relations with particular properties are valid simulations. This is an example where relations specifically appear in a computer science context.

**Optional Exercise 29.** Show that if you have a relation $R$ which is a simulation from $A$ to $A'$, and if $R'$ is a simulation from $A$ to $A''$ then their composite as relations, $R\,;R'$, is a simulation from $A$ to $A''$.

**Exercise 149.** Show the following for relational composition:

(a) Given a relation $R$ from $S$ to $T$, show that for the identity relation $I_S$ on $S$ we have $I_S\,;R = R$ and similarly that for the identity relation $I_T$ on $T$ we have $R\,;I_T = R$.

(b) Assume that $R$, $R'$ and $R''$ are relations that can be composed. Show that $(R\,;R')\,;R'' = R\,;(R'\,;R'')$.

We begin by looking at a generalization of the notion of a function, and then move to other cases of relations with particular properties.

## 7.2 Partial functions

Sometimes we would like to consider assignments that behave like functions, but which are not defined on the whole source set.

**Example 7.13.** Division is such an operation from $\mathbb{Q} \times \mathbb{Q}$ to $\mathbb{Q}$.

It is defined everywhere with the exception of the subset

$$\mathbb{Q} \times \{0\} \qquad \text{of} \qquad \mathbb{Q} \times \mathbb{Q}.$$

This gives us the choice of defining division

- as a function with source and target

$$\mathbb{Q} \times (\mathbb{Q} \setminus \{0\}) \longrightarrow \mathbb{Q}$$

- or as a *partial function*

$$\mathbb{Q} \times \mathbb{Q} \rightharpoonup \mathbb{Q},$$

  which is undefined for all those pairs whose second component is 0, that is, pairs of the form $(q, 0)$.

In this example there is a choice regarding what to do, but it can be difficult, or even impossible, to calculate the domain where a particular partial function is defined. Examples of this appear in COMP11212.

**Example 7.14.** A more interesting example is that of subtraction for the natural numbers. We may define a partial function

$$\mathbb{N} \times \mathbb{N} \rightharpoonup \mathbb{N}$$

which maps

$$(n, m) \longmapsto \begin{cases} n - m & n \geq m \\ \text{undefined} & \text{else.} \end{cases}$$

Alternatively a total function would have be[2]

$$\text{from } \{(n, m) \in \mathbb{N} \times \mathbb{N} \mid n \geq m\} \qquad \text{to } \mathbb{N}.$$

**Example 7.15.** Nothing stops us from defining, for example, for natural numbers $n$,

$$fn = \begin{cases} n & \text{the no of atoms in the universe is divisible by } n \\ \text{undefined} & \text{else} \end{cases}$$

Calculating where this function is defined is impossible (beyond repeating the definition). If we use this function together with a recursive definition we can define

**Base case** $g$. $\quad g0 = 0.$

**Step case** $g$.

$$g(n+1) = \begin{cases} gn + f(n+1) & gn,\ f(n+1) \text{ both defined} \\ fn & fn \text{ defined, } gn \text{ undefined} \\ \text{undefined} & \text{else.} \end{cases}$$

Here it is even harder to work out whether $g$ is defined for a given $n$, and there is no simple predicate which tells us whether $gn$ is defined for a given $n$.

**Example 7.16.** In COMP11212 you will study the idea of a *partially decidable partial function.* This is a partial function for which we can find an algorithm to tell us where it is defined, but we demand the termination of the algorithm only in the case where the given partial function is defined at the given element. Specifically, the question is whether there is a While program which, for input $n$, will terminate and give the answer 1 if the function is defined at $n$. Such a program is known as a *partial decision procedure.*

In a computer science context a typical application of this idea is to have a program $P$ that takes some input, for example a natural number, and produces an output, say another natural number. We may then define a partial function

$$n \longmapsto \begin{cases} \text{output of } P \text{ on input } n & P \text{ produces output for input } n \\ \text{undefined} & \text{else.} \end{cases}$$

The reason why $P$ might not produce an output for the given input could be that it attempts a division by 0, or that it runs into an infinite loop on certain inputs. You will study this idea in more detail in COMP11212.

We give a formal definition for the concept used in the examples above.

[2] Or we would have to turn it into a total function such as $\dot{-}$ from the previous section.

**Definition 51: partial function**

Let $S$ and $T$ be sets. A **partial function from $S$ to $T$** is an assignment where

$$\text{for every } s \in S \qquad \text{there is at most one } t \in T \qquad \text{with} \quad s \longmapsto t.$$

**Takeaway**

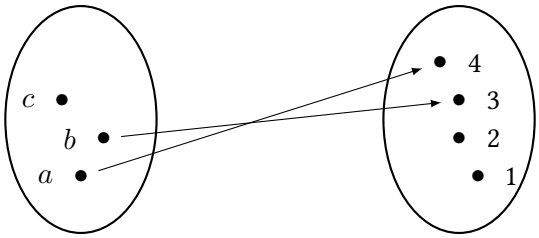
The assignment may not produce an output for some, or all, of its inputs.

We use a different kind of arrow, one with only 'half a tip'

$$f\colon S \rightharpoonup T$$

to indicate that the function described is partial.

When we have partial functions between small sets we can draw pictures of partial functions, similar to those used for (total) functions in Section 0.3. Here for every element of the source set we may have at most one element of the target set which it is connected with.




If $f$ is a partial function then when we write $fs$ we cannot be sure whether this defines an element of $T$, or whether this is undefined. This can be awkward when one tries to argue about partial functions. For this reason it is fairly customary to use an extra symbol[3] $\perp$.

If $f\colon S \rightharpoonup T$ is a partial function then we write

$$fs = \perp$$

in the case[4] where $f$ is not defined at $s$.

**Example 7.17.** Every function from $S$ to $T$ is also a partial function from $S$ to $T$.

When both, proper and partial functions are around people sometimes talk of **total functions** to distinguish proper functions, which are defined everywhere, from partial ones.

**Example 7.18.** Recall the notion of a list over a set $S$ from the previous chapter. We may want to define a function that returns the most recently added element of the list (if it exists). This should be a partial function $\mathrm{head}$ from the set of lists over $S$ to $S$. It is partial because it is undefined for the empty list. There is a recursive definition which says

$$\mathrm{head}(s : l) = s.$$

By not providing a definition for $\mathrm{head}$ at $[\,]$ we are implicitly stating that this function is not defined for the empty list, which means that it is partial.

[3] This looks like the symbol we used in Chapter 3 for propositions whose boolean interpretation relative to every valuation is 0, and that is no coincidence. People usually pronounce it 'bottom'—see Definition 64 for the reason why.

[4] This only works if $\perp$ is not an element of the target set $T$.

There really is no sensible way of extending this function to the empty list—how would we pick an element of $S$ to return?

It is for reasons such as the function in this example (and the fact that when programming, illegal operations such as division by 0 cannot be prevented from happening in some automated way) that computer scientists have to consider partial functions.

Above we discussed the idea that we can make a partial function total by restricting its source to include only those elements for which the function is defined. This idea has a name.

The **domain of definition of a partial function** $f\colon S \rightharpoonup T$ is defined as

$$\operatorname{dom} f = \{s \in S \mid fs \text{ is defined}\}.$$

**Proposition 7.1**

For every partial function $f\colon S \rightharpoonup T$ there is a unique total function

$$g\colon \operatorname{dom} f \longrightarrow T$$

with the property that for all $s \in \operatorname{dom} f$ we have

$$gs = fs.$$

**Proof.** We can use the desired equality as a definition for $g$. This clearly gives a total function with the required source and target and the condition ensures that for every element of $\operatorname{dom} f$ there is a unique element of $T$ to which to map it.

This means it is possible to work with total functions if we prefer to do so. However in theoretical computer science there exist a number of concepts around computability theory (and also in recursion theory in mathematics) which are more easily explained using partial functions. Examples occur in COMP11212.

The previous result has a counterpart where we move from total to partial functions.

**Proposition 7.2**

If $f\colon S \longrightarrow T$ is a (total) function from $S$ to $T$ and $S'$ is a superset of $S$ then there is a unique partial function

$$g\colon S' \rightharpoonup T$$

with the property that

$$\operatorname{dom} g = S \qquad \text{and} \qquad \forall s \in \operatorname{dom} g.\, gs = fs.$$

**Exercise 150.** Prove Proposition 7.2.

Partial functions can be composed just is the case for total functions, but we have to be careful about for which arguments the corresponding composite function is defined.

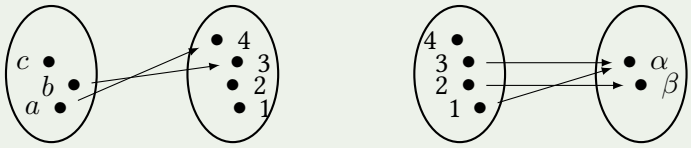
The **composite of partial functions** $f\colon S \rightharpoonup T$ and $g\colon T \rightharpoonup U$ is defined as:

$$(g \circ f)s = \begin{cases} g(fs) & fs \text{ and } g(fs) \text{ are both defined} \\ \bot & \text{else.} \end{cases}$$

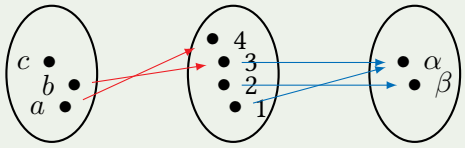
Alternatively one may say that $(g \circ f)s$ is defined if and only if $s$ is in the domain of definition of $f$ and $fs$ is in the domain of definition of $g$.

Note that this subsumes the definition of the composition of functions (defined in Section 0.3).

**Example 7.19.** We illustrate the notion of a composite of two partial functions by using a small example which can be described using a picture.

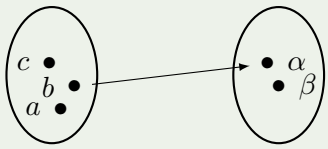



If we 'overlay' the two pictures we can more easily see what the composite has to look like.




The composite of the two partial functions can be drawn by the following procedure:

- Pick an element of the left hand set. If there is an outgoing arrow, follow that arrow. If there is an outgoing arrow from that element, follow that arrow and connect the original element with the resulting element.
- If at any point in the above procedure there is no outgoing arrow for one of the elements then the original element is not connected to anything.




**CExercise 151.** Consider the following partial functions with source $\mathsf{FBTrees}_S$.

(a) Give the definition of a (possibly partial) function that takes a full binary tree and returns the label of the root of the tree. What is the domain of definition of your function?

(b) Give the definition of a (possibly partial) function that takes a full binary tree and returns its left subtree. What is the domain of definition of your function?

(c) What happens if you apply your second function, and then the first, to a tree? Describe the domain of definition of this composite.

**EExercise 152.** Let $f\colon S \rightharpoonup T$ and $g\colon T \rightharpoonup U$ be partial functions.

(a) Show that the domain of definition of $g \circ f$ is a subset of the domain of definition of $f$, that is

$$\operatorname{dom}(g \circ f) \subseteq \operatorname{dom} f.$$

(b) Give a description of the domain of definition of $g \circ f$ as a subset of $\operatorname{dom} f$, without mentioning $g \circ f$, nor the expression $g(fs)$ for some $s \in S$. *Hint: You may use* $\operatorname{dom} f$ *and* $\operatorname{dom} g$.

The *graph of a partial function* $f\colon S \rightharpoonup T$ is very similar to that of a (total) function (compare

page 39). Its definition is

$$\{(s, fs) \in S \times T \mid f \text{ is defined at } s\}.$$

We can characterize those subsets of $S \times T$ which appear as the graph of a partial function (compare Proposition 2.2).

**Proposition 7.3**

A relation $R \subseteq S \times T$ is the graph of a partial function from $S$ to $T$ if and only if

$$\text{for all } s \in S \qquad \text{there exists at most one } t \in T \qquad \text{with} \qquad (s, t) \in R.$$

**Proof.** It is possible to make minor changes to the proof of Proposition 2.2 to obtain a proof of this result.

In the same way that we can view a function as a relation by considering its graph we may view a partial function that way, and this proposition tells us which relations are the graphs of partial functions.

**Exercise 153.** Show that for partial functions

$$f\colon S \rightharpoonup T \qquad \text{and} \qquad g\colon T \rightharpoonup U$$

we have that

$$\mathrm{gr}(g \circ f) = \mathrm{gr}\, f \,;\, \mathrm{gr}\, g,$$

where ; is the relational composite defined on page 355, and gr applied to a function gives its graph. Note that this means in particular that composition of (total) functions is subsumed by relational composition.

## 7.3 Binary relations

For the remainder of this chapter we only consider relations from one set *to itself*. Instead of saying that $R$ is a relation $S \rightharpoonup S$ one typically says that $R$ is a **(binary) relation on** $S$. Often it is convenient to drop the 'binary' in this case.

**Example 7.20.** Typical examples of binary relations on a set are the following.

(a) Sharing some property, such as having the same size, the same colour, the same nationality, speaking the same language, having the same digits after the decimal point, having a common divisor, evaluating to the same number,

(b) Having the same value under some function (some of the examples from above can also be viewed from this perspective, for example, one could map people to their height, or a real number to its sequence of digits after the decimal point, but one cannot do this for all these examples since a person may have more than one nationality or speak more than one language),

(c) Other kinds of connections between two members of the same set, for example one person having the contact details for another, or a number being less than or equal to another, or two subsets of a given set being included in each other.

(d) If we look at two objects of class List then we think of them as implementing the same

list if the method equal (see Example 6.14) returns true. This defines a binary relation on objects of this class.

A different way of thinking of the same relation is as follows: Every object of class List can be thought of as implementing an element of $\mathsf{Lists}_{\mathbb{Z}}$ in the obvious way. This defines a function

$$\text{Objects of class List} \longrightarrow \mathsf{Lists}_S,$$

and two such objects are in the relation if and only if this function maps them to the same list.[5]

Note that for binary relations the 'source' and 'target' are identical, which means we may always form the relational composite of a binary relation with itself.

### 7.3.1 Picturing binary relations

If we have a binary relation on a small finite set $S$ it is possible to draw[6] it in the form of a *directed graph*.

**Definition 52: directed graph**

A **directed graph on a set** $S$ is given[7] by a binary relation on $S$. In this context the elements of $S$ are often called the *nodes* of the graph[8] and the pairs in the relation the *edges*.

**Takeaway**

We may picture a directed graph by drawing the nodes and arrows between those.

Such graphs are often drawn as pictures when the set $S$ is small.

**Example 7.21.** Consider the following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, b), (b, a), (b, c), (c, c), (c, d), (d, e), (e, a), (b, d)\}.$$

Its picture as a binary relation is as follows.

$$\begin{tikzcd}
 & c \arrow[loop above] \arrow[r] & d \arrow[d] \\
b \arrow[ur] \arrow[urr] \arrow[dr, leftrightarrow] & & e \arrow[dl] \\
 & a &
\end{tikzcd}$$

In this picture the arrow with tips at both ends is a shortcut to having arrows going each way.

[5] Compare Example 6.14.

[6] Above we show how to generally draw relations from a small set $S$ to a small set $T$, but this method makes use of the fact that the relation goes from a set to the same set.

[7] Some definitions of directed graph exclude connections between an element of $S$ and itself. In that case the graph is given by a collection of pairs of the form $(s, s')$ where $s$ and $s'$ are distinct elements of $S$. When that definition is used, our directed graphs become *directed graphs with loops*.

[8] Some people call the nodes of the graph the vertices.

**Exercise 154.** Draw the following binary relations.

(a) The powerset $\mathcal{P}\{a, b, c\}$ with the relation which relates a subset $S$ of $\{a, b, c\}$ to the subset $S'$ if and only if $S \subseteq S'$.

(b) The set $\{0, 1, 2, 3, 4, 5, 6\}$ with the relation which relates an element $m$ to an element $n$ if and only if they both leave the same remainder when divided by 3, that is, $m \bmod 3 = n \bmod 3$.

(c) The set $\{0, 1, 2, 3, 4, 5, 6\}$, with the relation which relates an element $m$ to an element $n$ if and only if $m \leq n$.

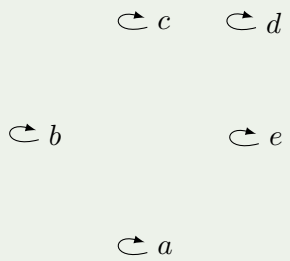
(d) The set $\{0, 1, 2, 3, 4, 5, 6\}$ with the relation which relates an element $m$ to an element $n$ if and only if $m$ divides $n$.

(e) The compulsory course units for a student on your programme of studies, where one unit is related to another if it is a prerequisite for that unit (ignore corequisites).

(f) The students in your tutorial group where one student is related to another if they have the same gender.

The identity relation on a set can be pictured easily using these ideas. It is the relation which connects every element of the set with itself, and nothing else.

**Example 7.22.** For the set $\{a, b, c, d, e\}$ the identity relation is pictured as follows.



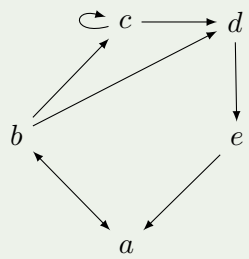

When we have a binary relation we picture its opposite slightly differently from the way illustrated in Example 7.6.

**Example 7.23.** For the relation on the set $\{a, b, c, d, e\}$ from Example 7.21,

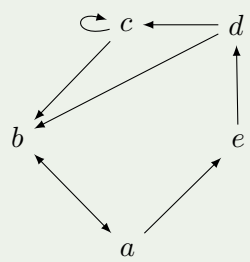
$$\{(a, b), (b, a)(b, c), (c, c), (c, d), (d, e), (e, a), (b, d)\},$$

we show both, the given relation and its opposite.

Given relation




Its opposite




The opposite relation, described as a set of pairs, is

$$\{(b,a),(a,b)(c,b),(c,c),(d,c),(e,d),(a,e),(d,b)\},$$

We can see how the opposite relation arises from the original by turning around all the arrows—which means that for loops, or for arrows with tips on both ends, nothing changes.

### 7.3.2 Further relations on a set

Very occasionally one may want to connect more than two elements of the same set, in which case one may speak of a

- ternary relation on $S$ (a subset of $S \times S \times S$),
- a quarternary relation on $S$ (a subset of $S \times S \times S \times S$),
- or, more generally, an $n$-ary relation on $S$ (a subset of the $n$-fold product of $S$).

## 7.4 Equivalence relations

Sometimes when we look at all the elements of a set we don't necessarily wish to distinguish all the elements of the set. When you are building a specific structure from building blocks, in order to complete the design you don't have to consider the colour of a given block, just its shape. So from the point of view of the design, all blocks of the same shape are equivalent.[9] In COMP11212 you have met the notion of a bisimulation between two finite state automata. You can think of a bisimulation as a way of demonstrating that two given automata are equivalent, as far as the language they define is concerned.

More generally whenever we want to think of various entities as equivalent we have to make sure we do this in a safe way. Relations that allow us to do this are known as *equivalence relations*. These are relations satisfying a number of properties, and we introduce these properties one at a time.

[9] But, of course, for aesthetic reasons, or to match an existing edifice, you may want to consider the colour when building a structure.

### 7.4.1 Reflexivity

**Definition 53: reflexive**

A binary relation $R$ on a set $S$ is **reflexive** if and only if it is the case that we have

$$\text{for all } s \text{ in } S \qquad (s, s) \in R.$$

**Takeaway**

Every element is related to itself.

If we want to express properties of binary relations using first order logic we can express the relation as a two-placed predicate, where we take $R(x, y)$ to mean that $R$ relates $x$ and $y$, which we usually write as $(x, y) \in R$. Hence the first order logic formula that describes reflexivity of $R$ is

$$\forall x.\, R(x, x).$$

We may express reflexivity in a very brief way by noting that it means that

$$I_S \subseteq R,$$

where $I_S$ is the identity relation on the underlying set $S$.

This means that if our relation is reflexive then every element of the underlying set is related to itself. If we can draw a picture of the relation we can check whether the relation is reflexive by checking that every element has a connection from itself to itself, usually drawn as a little loop.

Typical examples are relations involving some kind of equality.

**Example 7.24.** We provide examples and non-examples for this concept.

(a) Consider the relation between first year students in the Department where two students are related if and only if they are in the same tutorial group. Since every student is in the same tutorial group as him- or herself, this relation is reflexive.

(b) Look at the relation between two members of the human population where two people are related if and only if they have the same height in centimetres. Since everybody has the same height as him- or herself, this relation is reflexive.

(c) The relation between two non-zero natural numbers where $m$ is related to $n$ if and only if $m$ divides $n$ is reflexive since every natural number other than 0 divides itself.

(d) We define the relation between first year students in the Department where student $A$ is related to student $B$ if $A$'s student id number is below that of student $B$. This relation is not reflexive since a number is not below itself.

(e) Consider the relation between members of the human population which relates person $A$ to person $B$ if and only if they are siblings. This relation is not reflexive since nobody is their own sibling. On the other hand, relating two people if and only if they have a parent in common is a reflexive relation.

(f) Consider the relation between two natural numbers where $m$ is related to $n$ if $n = 2m$. Since, for example $1 \neq 2 \cdot 1$ this relation is not reflexive.

(g) The relation between elements of $\mathsf{Lists}_S$ which relates two lists if and only if they have the same number of elements is reflexive.

(h) The relation which relates two objects of class List if the method equal returns true (see Example 6.14) is reflexive.

(i) The relation of semantic equivalence for propositions is reflexive.

**Exercise 155.** Which of the following relations are reflexive? Justify your answer.

(a) The relation where two first year students in the Department are related if their last name starts with the same letter.

(b) Two first year students in the Department being related if there is a university society they both belong to.

(c) The following relation on the set $\{a, b, c, d\}$:

$$\{(a, a), (b, c), (c, b), (b, b), (c, d), (d, c), (d, d)\}$$

(d) The relation on $\mathbb{N}$ where two numbers are related if and only if they have a common divisor greater than 1.

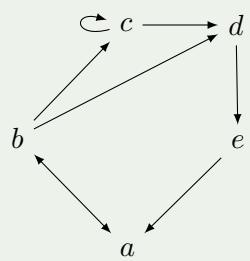
It is easy to make a relation reflexive in a unique and minimal way. Given a binary relation $R$ on $S$ the **reflexive closure of** $R$ is given by

$$R \cup I_S = R \cup \{(s, s) \mid s \in S\}.$$

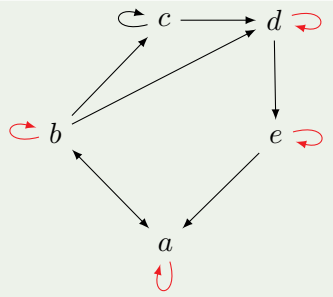
Since we add precisely those pairs to the relation which have to be present for it to satisfy reflexivity this is clearly the smallest relation we can form which contains $R$ as a subset and which is reflexive. Optional OExercise 30 asks you to think about how to prove this.

When we consider a binary relation on a small finite set then checking whether the relation is reflexive, and drawing the reflexive closure, is easy. All we have to do is to make sure that every element has an arrow from itself to itself.

**Example 7.25.** If we go back to the relation from Example 7.21, we see that the relation is not reflexive since, with the exception of $c$, no element has a connection to itself.




The reflexive closure of this relation is pictured below.[10]




### 7.4.2 Symmetry

The next important property we consider for binary relations on a set is concerned with directedness: If we can go from one element to another, can we always go back?

**Definition 54: symmetric**

A binary relation $R$ on a set $S$ is **symmetric** if and only if we have

$$\text{for all } s, s' \in S \qquad (s, s') \in R \qquad \text{implies} \qquad (s', s) \in R.$$

**Takeaway**

Being related works both ways.

The first order logic formula that describes this property for a binary predicate symbol $R$ is

$$\forall x. \forall y. \, (R(x, y) \to R(y, x)).$$

Relations built around the idea of equality of a property are usually symmetric, but there are plenty of relations which are not symmetric.

If we have a picture of a relation we can check whether it is symmetric by checking that every connection has an arrow at each end.

**Example 7.26.** We give some examples for relations which are symmetric, and some which are not.

(a) Consider the relation between first year students in the Department where two students are related if and only if they are in the same tutorial group. If student $A$ is in the same tutorial group as student $B$ then student $B$ is in the same tutorial group as student $A$, and so this relation is symmetric.

(b) Look at the relation between two members of the human population where two people are related if and only if they have the same height in centimetres. Since $A$ having the same height as $B$ implies that $B$ has the same height as $A$ this relation is symmetric.

(c) The relation between two non-zero natural numbers where $m$ is related to $n$ if and only if $m$ divides $n$ is not symmetric: 1 divides 2 but 2 does not divide 1.

(d) The relation between first year students in the Department where student $A$ is related to student $B$ if $A$'s student id number is below that of student $B$ is not symmetric:[11] Indeed, if the id number for student $A$ is below that of student $B$ then that for student $B$ cannot be below that for student $A$.

[10]The new edges are drawn in red.

(e) The relation that relates two people if and only if they are siblings is symmetric, whereas the relation that relates two people if the first is the child of the second is not.

(f) Consider the relation between two natural numbers where $m$ is related to $n$ if $n = 2m$. This relation is not symmetric since 2 is related to 4 but 4 is not related to 2.

(g) The relation between elements of $\mathsf{Lists}_S$ which relates two lists if and only if they have the same number of elements is symmetric.

(h) The relation which relates two objects of class List if the method equal returns true (see Example 6.14) is symmetric.

(i) The relation of semantic equivalence for propositions is symmetric.

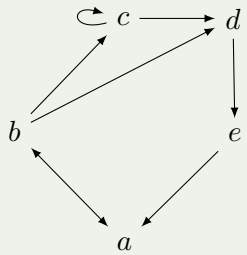
Again we can make a relation symmetric in a unique and minimal way.
If $R$ is a binary relation on a set $S$ then the **symmetric closure of** $R$ is given by

$$R \cup R^{\mathsf{op}}.$$

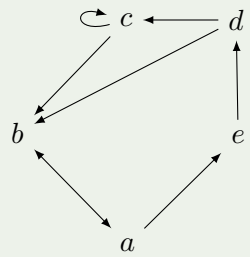
This is the smallest relation on $S$ that contains $R$ and is symmetric. Optional OExercise 30 asks you to think about how to prove this.

**Example 7.27.** If we return to Example  we can see that taking the union of the given relation and its opposite we get a relation where every connection has arrows pointing both ways.

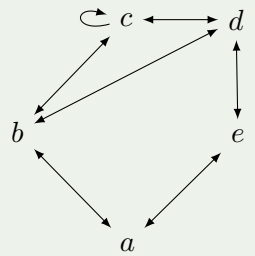
Given relation $R$




Its opposite $R^{\mathsf{op}}$




Their union $R \cup R^{\mathsf{op}}$




This gives us a concise way of saying what it means for a relation $R$ to be symmetric, namely

$$R = R \cup R^{\mathsf{op}}.$$

This can be simplified further by the observation that the non-trivial part of this equality is that

$$R \supseteq R \cup R^{\mathsf{op}},$$

[11]This example seems to be the opposite of symmetric—to make that idea precise look at the notion of *anti-symmetry* defined in Section 7.5.1.

and since $R$ is always a superset of itself, symmetry is equivalent to demanding that

$$R \supseteq R^{\mathsf{op}}.$$

Another way of expressing symmetry using these ideas is to demand that

$$R = R^{\mathsf{op}}.$$

Above there is an argument that symmetry is equivalent to $R \supseteq R^{\mathsf{op}}$, and by applying the $(-)^{\mathsf{op}}$ operator on both sides this implies

$$R^{\mathsf{op}} \supseteq (R^{\mathsf{op}})^{\mathsf{op}} = R,$$

so the two must be equal.

An alternative way of describing the symmetric closure of $R$ is given by

$$\{(s, s') \in S \times S \mid (s, s') \in R \text{ or } (s', s) \in R\}.$$

In other words, we add exactly those pairs to the relation which have to be present for the relation to become symmetric. If the relation is defined on a small finite set then we can once again look at the graph.

**Example 7.28.** Looking once again at the relation from Example 7.21 we can perform the check suggested above to see that it is not symmetric.

$$\begin{tikzcd}
 & c \arrow[loop above] \arrow[r] & d \arrow[d] \\
b \arrow[ur] \arrow[urr] \arrow[dr, leftrightarrow] & & e \arrow[dl] \\
 & a &
\end{tikzcd}$$

For example, $(b, c)$ is in the relation but $(c, b)$ is not. We may think of its symmetric closure as being constructed by adding all the arrow tips missing, and picture it[12] as follows. If we think of it this way we don't have to draw the opposite relation as in Example 7.27.

$$\begin{tikzcd}
 & c \arrow[loop above] \arrow[r, leftrightarrow] \arrow[dl, leftrightarrow] & d \arrow[d, leftrightarrow] \arrow[dl] \\
b \arrow[dr, leftrightarrow] & & e \arrow[dl, leftrightarrow] \\
 & a &
\end{tikzcd}$$

Again if we can draw the corresponding graph it is easy to see whether the relation is symmetric: We just have to check that every arrow that is not a loop has a tip at both ends.

**Exercise 156.** Which of the following relations is symmetric? Justify your answer.

(a) The relation where two first year students in the Department are related if their last name starts with the same letter.

(b) The relation on first year students in the Department where $A$ is related to $B$ if $A$ can

[12] New arrow tips drawn in red.

name student $B$ when shown a picture.

(c) The following relation on the set $\{a, b, c, d\}$:

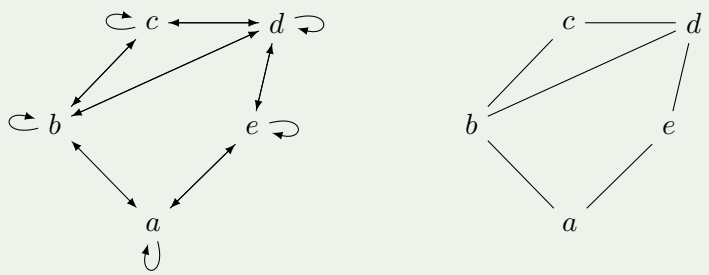
$$\{(a, a), (b, c), (c, b), (b, b), (c, d), (d, c), (d, d)\}$$

(d) The relation on $\mathbb{N} \setminus \{0\}$ where $m$ is related to $n$ if and only if $m$ and $n$ have a common divisor other than 1.

(e) The relation on $\mathbb{N} \setminus \{0, 1\}$ where $m$ is related to $n$ if and only if $m$ divides a power of $n$.

Whenever we know a relation to be reflexive and symmetric we can picture it using an *undirected graph*. This is a graph where we only record which of the elements are connected, without worrying about the direction of that connection. We do not record an element being connected with itself, we know they all are and so there's no reason to include that information in the picture.

**Example 7.29.** The symmetric closure of the reflexive closure of our example relation is drawn on the left, and on the right we draw the corresponding undirected graph where redundant information (for relations known to be reflexive and symmetric) has been removed.




### 7.4.3 Transitivity

We require one additional property of relations to define the concept we are aiming for.

**Definition 55: transitive**

A binary relation $R$ on a set $S$ is **transitive** if and only if we have that

$$\text{for all } s, s', s'' \in S$$
$$(s, s') \in R \text{ and } (s', s'') \in R \qquad \text{implies} \qquad (s, s'') \in R.$$

**Takeaway**

Being related transfers along multiple steps of the relation.

This definition means that whenever we have a situation in the picture below, given the two black arrows we must have the red one.

$$\begin{tikzcd}
 & s'' \\
s' \arrow[ur] & \\
 & s \arrow[ul] \arrow[uu, red]
\end{tikzcd}$$

The corresponding first order formula describing this property is

$$\forall x.\forall y.\forall z.\,((R(x,y) \wedge R(y,z)) \to R(x,z)).$$

Again, relations based on equality of a property are often transitive, but not always.

**Example 7.30.** (a) Consider the relation between first year students in the Department where two students are related if and only if they are in the same tutorial group. If student $A$ is in the same tutorial group as student $B$ and student $B$ is in the same tutorial group as student $C$ then student $A$ is in the same tutorial group as student $C$, and so this relation is transitive.

(b) Look at the relation between two members of the human population where two people are related if and only if they have the same height in centimetres. Since $A$ having the same height as $B$ and $B$ having the same height as $C$ implies that $A$ has the same height as $C$ this relation is transitive.

(c) The relation between first year students in the Department where student $A$ is related to student $B$ if $A$'s student id number is below that of student $B$ is transitive: Indeed, if the id number for student $A$ is below that of student $B$ and that for student $B$ is below that for student $C$ then that for student $A$ is below that for student $C$.

(d) The relation on the set of all humans which relates person $A$ to person $B$ if and only if $A$ is a child of $B$ is not transitive, but the relation that relates $A$ to $B$ if and only if $B$ is an ancestor of $A$ is. (See also Example 7.32.)

(e) Consider the relation between two natural numbers where $m$ is related to $n$ if $n = 2m$. This relation is not transitive since 2 is related to 4 and 4 is related to 8 but 2 is not related to 8.

(f) Consider the relation between human beings where person $A$ is related to person $B$ if there is a language they both speak. This relation is reflexive and symmetric but not transitive: Person $A$ may speak English and Urdu, Person $B$ may speak English and Spanish, and Person $C$ may speak Spanish. The relation is

$$\{(A,B),(B,C)\}.$$

So $A$ is related to $B$, and $B$ is related to $C$, but $A$ is not related to $C$.

(g) The relation between elements of $\mathsf{Lists}_S$ which relates two lists if and only if they have the same number of elements is transitive.

(h) The relation which relates two objects of class List if the method equal returns true (see Example 6.14) is transitive.

(i) The semantic equivalence relation between propositions is transitive.

We may define the **transitive closure of a relation $R$ on a set $S$** as adding all those pairs

$$(s_1, s_n) \qquad \text{to} \qquad R$$

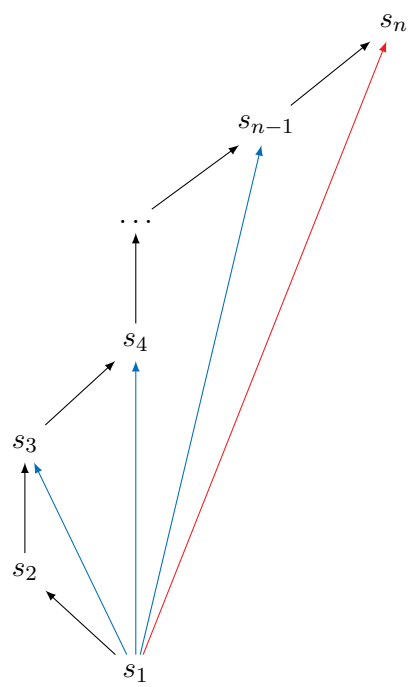
for which we can find a list of elements

$$s_1,\ s_2,\ \ldots,\ s_n \text{ in } S, \qquad\qquad \text{with } n \geq 2,$$

such that

$$\text{for all } 1 \leq i \leq n-1 \qquad \text{we have} \qquad (s_i, s_{i+1}) \in R.$$

We may picture this situation as follows: Whenever we have the black arrows, we must have the blue arrows, and that means we must have the red arrow. You may understand the definition above as going straight from the black arrows to the red one.




If we look at a relation on a small finite set then transitivity is more difficult to check than reflexivity and symmetry. What we need to check here is that for all $s$, $s'$ in $S$,

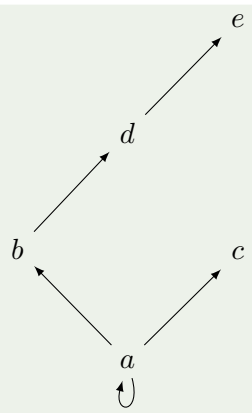
if there is a **path** from $s$ to $s'$ then there is an **edge** from $s$ to $s'$.

In the relation from Example 7.21 the transitive closure requires us to connect *every pair of elements* since there is a path from every element to every element. This is messy to draw, so we consider a different example here.

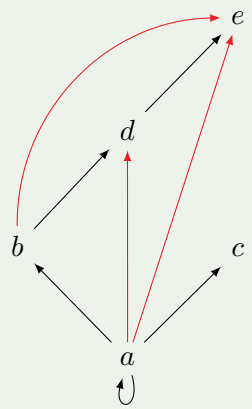
**Example 7.31.** Assume we have the relation

$$R = \{(a,a), (a,b), (a,c), (b,d), (d,e)\}.$$

The picture of this relation is as follows:




The transitive closure of this relation is given by[13]




**Example 7.32.** If we look at the relation 'is a parent of' (see Example 7.7) then its transitive closure gives us the 'is an ancestor' relation.

Instead of calculating the transitive closure of a relation in one go we can do it stepwise. For this we need to define the powers (with respect to composition of relations) of a binary relation. Consider he definition of the relational composite ; from page 355, and note that since a binary relation goes from a set to the same set we may compose such a relation with itself.

We may therefore give the following recursive definition. Let $R$ be a binary relation on a set $S$.

**Base case** ˆ. $R^0 = I_S$.

**Step case** ˆ. $R^{n+1} = R^n \,;\, R$.

An intuitive explanation of $R^n$ is the following:

$s$ and $s'$ in $S$ are related by $R^n$

if and only if

one can get from $s$ to $s'$ by following the relation $R$ exactly $n$ times.

The transitive closure of $R$ consists of all those pairs $(s, s')$ for which it is possible to get from $s$ to $s'$ by following $R$ any finite number greater than 0 times. This motivates a second recursive definition.

[13] New edges drawn in red.

**Base case** $R^{\leq n}$. $\quad R^{\leq 0} = \emptyset$.

**Step case** $R^{\leq n}$. $\quad R^{\leq (n+1)} = R^{\leq n} \cup R^{n+1}$.

$$\begin{array}{c} s \text{ and } s' \text{ in } S \qquad \text{are related by } R^{\leq n} \\ \text{if and only if} \\ \text{one can get from } s \text{ to } s' \text{ by following the relation } R \\ \text{at least once and at most } n \text{ times.} \end{array}$$

Note that $R^0$, the identity relation on $S$, is not involved in computing the transitive closure of a relation.

**Example 7.33.** Assume the relation $R$ describes railway journeys, so that two stations are in the relation if and only if one can travel from the first to the second without having to change trains. Then $R^2$ gives us pairs of stations for which it is possible to travel from the first to the second by changing exactly once, $R^3$ gives us pairs of stations where one has to change exactly twice, whereas $R^{\leq 3}$ gives those where one has to change at most twice.

We define

$$R^{<\infty} = \bigcup_{n \in \mathbb{N}} R^{\leq n} = \bigcup_{n \in \mathbb{N} \setminus \{0\}} R^n.$$

This is the relation which can be described as follows:

$$\begin{array}{c} s \text{ and } s' \text{ in } S \qquad \text{are related by } R^{<\infty} \\ \text{if and only if} \\ \text{one can get from } s \text{ to } s' \\ \text{by following the relation } R \text{ a finite number of times, but at least once.} \end{array}$$

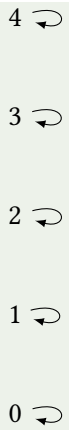
This gives us a way of stating that a relation $R$ is transitive, since this is the case if and only if

$$\forall n \in \mathbb{N} \setminus \{0\} \qquad \text{we have} \qquad R^n \subseteq R.$$

**Example 7.34.** If, for example, we have the set $\{0, 1, 2, 3, 4\}$ with the relation

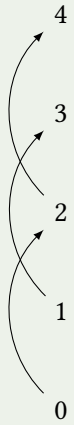
$$R = \{(0, 1), (1, 2), (2, 3), (3, 4)\}$$

then we have the following.

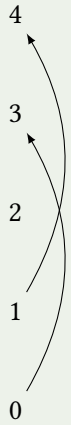



$$R^0 = \{(0,0),(1,1),(2,2),(3,3),(4,4)\}$$

$$R^1 = R$$




$$R^2 = \{(0,2),(1,3),(2,4)\}$$




$$R^3 = \{(0,3),(1,4)\}$$

$$R^4 = \{(0,4)\}$$

All $R^n$, for $n \geq 5$, are empty.

We draw the relations $R^{\leq n}$ for this example.

$$\begin{tikzcd}
4 \\
3 \\
2 \\
1 \\
0
\end{tikzcd}$$

$$R^{\leq 0} = \emptyset$$

$$\begin{tikzcd}
4 \\
3 \arrow[u] \\
2 \arrow[u] \\
1 \arrow[u] \\
0 \arrow[u]
\end{tikzcd}$$

$$R^{\leq 1} = R^1 \cup R^{\leq 0} = R$$

$$\begin{tikzcd}
4 \\
3 \arrow[u] \arrow[ddu, bend left] \\
2 \arrow[u] \arrow[uu, bend left] \\
1 \arrow[u] \arrow[uu, bend left] \\
0 \arrow[u] \arrow[uu, bend left] \arrow[uuu, bend left]
\end{tikzcd}$$

$$R^{\leq 2} = R^2 \cup R^{\leq 1}$$

$$\begin{tikzcd}
4 \\
3 \arrow[u] \arrow[ddu, bend left] \\
2 \arrow[u] \arrow[uu, bend left] \arrow[ddu, bend left] \\
1 \arrow[u] \arrow[uu, bend left] \arrow[uuu, bend left] \\
0 \arrow[u] \arrow[uu, bend left] \arrow[uuu, bend left] \arrow[uuuu, bend left]
\end{tikzcd}$$

$$R^{\leq 3} = R^3 \cup R^{\leq 2}.$$

$$\begin{tikzcd}
4 \\
3 \arrow[u] \arrow[ddu, bend left] \\
2 \arrow[u] \arrow[uu, bend left] \arrow[ddu, bend left] \\
1 \arrow[u] \arrow[uu, bend left] \arrow[uuu, bend left] \arrow[ddu, bend left] \\
0 \arrow[u] \arrow[uu, bend left] \arrow[uuu, bend left] \arrow[uuuu, bend left] \arrow[uuuuu, bend left]
\end{tikzcd}$$

$$R^{\leq 4} = R^4 \cup R^{\leq 3}$$

All remaining $R^{\leq n}$, for $n \geq 5$, are equal to $R^{\leq 4}$. For this reason the last picture, showing $R^{\leq 4}$, shows the transitive closure of $R$.

**Exercise 157.** Show that $R^{<\infty}$ is the transitive closure of $R$.

**CExercise 158.** Which of the following relations are transitive? Justify your answers. For those which are not transitive describe the transitive closure.

(a) The relation on the set of first year students within the Department which relates student $A$ to student $B$ if they have at least one course unit in common.

(b) The relation on the set of first year students within the Department which relates student $A$ to student $B$ if they have the same nationality.

(c) The relation on $\mathbb{N}$ where $m$ is related to $n$ if and only if $m$ divides $n$.

(d) The relation on $\mathbb{N}$ where $m$ is related to $n$ if and only if $m + n$ is even.

(e) The relation on $\mathbb{N}$ where $m$ is related to $n$ if and only if $m$ and $n$ have a common divisor greater than 1, or if $m = n = 1$.

(f) The following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, a), (a, b), (b, e), (e, c)\}$$

(g) The following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, a), (a, b), (c, d), (d, e)\}$$

(h) The following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, a), (b, b), (c, c), (e, e)\}$$

If we want to create the transitive closure of the symmetric closure of the reflexive closure of $R$ all we have to do to this procedure is to change what we do at the start. We have to change the relation we use

$$\tilde{R} = I_S \cup R \cup R^{\mathsf{op}}$$

This achieves two objectives:

- By adding all elements of the identity relation we add all pairs $(s, s)$. where $s \in S$, we make sure the relation we produce is reflexive.
- By adding all elements of the relation $R^{\mathsf{op}}$ we add all the pairs $(s', s)$ for which $(s, s') \in R$, which ensures that we produce a symmetric relation.

**Proposition 7.4**

The transitive closure of the symmetric closure of the reflexive closure of a relation $R$ on a set $S$ is given by $\tilde{R}_\infty$.

**Proof.** By Exercise 157 we know that given a relation $R'$ the relation $R'_\infty$ is transitive. We observe that forming $I_S \cup R$ is the reflexive closure of $R$, and that the symmetric closure of

the result is

$$\begin{aligned}(I_S \cup R) \cup (I_S \cup R)^{\mathsf{op}} &= (I_S \cup R) \cup ({I_S}^{\mathsf{op}} \cup R^{\mathsf{op}}) \\ &= I_S \cup R \cup R^{\mathsf{op}} \qquad\qquad {I_S}^{\mathsf{op}} = I_S.\end{aligned}$$

The remainder follows from the following exercise.

Note that the procedure of forming the reflexive symmetric transitive closure is important. If somebody tells you about particular instances they want you to consider as equivalent then this allows you to generate an equivalence relation which makes all the specified entities equivalent, but does not identify anything unnecessarily. This procedure is used in Section 7.4.4.

**EExercise 159.** The various closures of relations defined above work well together.

(a) Show that the symmetric closure of a reflexive relation is reflexive.

(b) Show that the transitive closure of a reflexive relation is reflexive.

(c) Show that the transitive closure of a symmetric relation is symmetric.

Conclude that the transitive closure of the symmetric closure of a reflexive relation is reflexive, symmetric and transitive. It is usually referred to as the *reflexive symmetric transitive closure. Hint: You may want to use Exercise 157, but you don't have to.*

This exercise shows that if we form the transitive closure of the symmetric closure of the reflexive closure of a relation then the result will be reflexive, symmetric, and transitive.

**Optional Exercise 30.** Above we talk about how to add a minimal number of elements to a relation to make it reflexive, symmetric or transitive. In this exercise we make these ideas precise.

(a) Show that if $R'$ is a relation on a set $S$ which is reflexive and which contains the relation $R$ as a subset then $R'$ contains the reflexive closure of $R$.

(b) Show that if $R'$ is a relation on a set $S$ which is symmetric and which contains the relation $R$ as a subset then $R'$ contains the symmetric closure of $R$.

(c) Show that if $R'$ is a relation on a set $S$ which is transitive and which contains the relation $R$ as a subset then $R'$ contains the transitive closure of $R$.

(d) Conclude that the reflexive/symmetric/transitive closure of a relation $R$ is the smallest reflexive/symmetric/transitive relation which contains $R$ as a subset.

**Optional Exercise 31.** There is another way of defining the three closure operations for relations. Instead of adding elements to the give relation one may think about starting with a large relation and then removing all those pairs of elements which are not required.

(a) Show that the intersection of arbitrarily many reflexive relations is reflexive.

(b) Show that the intersection of arbitrarily many symmetric relations is symmetric.

(c) Show that the intersection of arbitrarily many transitive relations is transitive.

(d) Prove that the intersection of all reflexive/symmetric/transitive relations containing a

relation $R$ is the smallest reflexive/symmetric/transitive relation containing $R$ and conclude (with the help of the previous optional exercise) that this intersection is equal to the reflexive/symmetric/transitive closure of $R$.

### 7.4.4 Equivalence relations defined

**Definition 56: equivalence relation**

A binary relation on a set is an **equivalence relation** if it is reflexive, symmetric and transitive.

**Takeaway**

These properties give a sensible notion of items we don't want to distinguish between by treating them as equivalent.

A number of examples are given above, but we put them together here:

**Example 7.35.** We give examples of equivalence relations.

(a) From every function we get an equivalence relation by relating those elements of the source set which are mapped to the same element in the target set, see Exercise 161.

A number of the relations mentioned above fall under this idea.

(i) Considering building blocks of the same shape equivalent has the underlying function which maps a block to its shape.

(ii) Mapping people to their height in centimetres leads to identifying those that have the same height.

(iii) Mapping students to their tutorial group allows us to identify the members of the same group.

(iv) Mapping people to their nationality[14] allows us to talk about the nationalities represented in a particular group.

(v) Another example taken from programming is the following: Given a specific algorithm there are many programs which implement that algorithm in a particular programming language.[15] Typically we don't care which particular program is used, only that it implements the chosen algorithm correctly. The underlying function here maps each program to the algorithm it implements.

(b) Relating elements of a set which have a particular property is usually a special case of the previous example, because we can map the elements of the set to the corresponding property (for example nationality, or tutorial group). But one has to be careful here in cases where there is no underlying function: Speaking a common language is not an equivalence relation since this need not be a transitive relation, see above on page 373.

(c) One might want to identify all the sets which have the same size, which leads to the notion of *cardinal* in mathematics.[16]

(d) When we consider which algorithm to use to solve a particular problem we may be worried about its complexity only. In that case we typically don't worry about distinguishing between algorithms that are in the same *complexity class*.[17]

(e) In COMP11212 and COMP26120 you will learn about the notion of 'big O'. There is an underlying equivalence relation on functions where $f$ and $g$ are equivalent if and only if

- there exists $m \in \mathbb{N}$ such that $nf$ eventually dominates $g$ and
- there exists $n \in \mathbb{N}$ such that $ng$ eventually dominates $f$.

This is an equivalence relation.

(f) The notion of semantic equivalence from Chapter 3 defines an equivalence relation on propositions, where we only care about the boolean interpretations of a given proposition.

(g) The relation between elements of $\mathsf{Lists}_S$ which relates two lists if and only if they have the same number of elements is another example.

(h) The relation which relates two objects of class List if the method equal returns true (see Example 6.14) is an equivalence relation.

**CExercise 160.** Which of the following relations are equivalence relations? Justify your answers.

(a) The relation where two first year students in the Department are related if their last name starts with the same letter.

(b) Two first year students in the Department being related if there is a university society they both belong to.

(c) The $\equiv$ relation on propositional formulae from Chapter 4.

(d) The relation on Java objects of the class java.lang.Object defined by object A being related to object B if and only if the default instance method A.equals(B) returns true.

(e) The relation on objects of the class BTree, where t1 is related to t2 if and only if the following method (compare Code Example 6.6, when called as equal(t1,t2), returns true:

```
public static boolean equal (BTree t1, BTree t2)
{
 if (t1 == null)
     return (t2 == null);
 else {
     if (t2 == null)
         return false;
     else
         return ((t1.value == t2.value)
         && equal (t1.left, t2.left)
         && equal (t1.right, t2.right));
```

[14]This ignores the possibility of dual citizenship.
[15]Formally defining what this means is tricky, but you should get the general idea.
[16]Formally one has to be a bit careful since there is no such thing as the 'set of all sets', but the idea remains.
[17]A formal definition of what that means is given in COMP11212.

}

}

(f) The following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, a), (b, b), (a, b), (b, a), (a, c), (c, a), (b, c), (c, b), (d, d), (e, e)\}$$

(g) The reflexive closure of the following relation on the set consisting of the elements $a$, $b$, $c$, $d$ and $e$:

$$\{(a, b), (b, a), (b, c), (c, b)\}$$

(h) The reflexive closure of the following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, b), (b, a)\}$$

(i) The relation on $\mathbb{N}$ where $m$ is related to $n$ if and only if $m + n$ is even.

(j) The relation on $\mathbb{N}$ where $m$ is related to $n$ if and only if $m$ and $n$ have a common divisor greater than 1 or if $m = n = 1$.

(k) The relation on $\mathbb{N} \setminus \{0\}$ where $m$ is related to $n$ if and only if

$$m \bmod n = 0.$$

**Exercise 161.** We look at the idea that functions generate equivalence relations. Assume that $f\colon S \longrightarrow T$ is a function.

(a) Show that the following defines an equivalence relation on $S$:

$$s \sim_f s' \qquad \text{if and only if} \qquad fs = fs'.$$

(b) Show that $f$ is injective if and only if the corresponding equivalence relation $\sim_f$ from (a) is given by the identity relation on $S$,

$$I_S = \{(s, s) \mid s \in S\}.$$

(c) Show that if $g\colon T \longrightarrow U$ is a function then

$$s \sim_{g \circ f} s' \qquad \text{if and only if} \qquad s \sim_f s' \text{ or } fs \sim_g fs'.$$

Given a binary relation $R$ on a set the **equivalence relation generated by a binary relation $R$** is the relation obtained by forming the transitive closure of the symmetric closure of the reflexive closure of $R$. The resulting relation is reflexive, symmetric and transitive by EExercise 159. On page 379 there is a description of how to calculate that closure step-by-step. See Example 7.49 for a concrete example of carrying out the procedure.

We use an equivalence relation wherever we wish not to distinguish between certain elements of a set.

**Example 7.36.** When we use fractions to refer to rational numbers we do this with respect

to the following equivalence relation: For $m, n, m', n'$ in $\mathbb{Z}$ we set

$$\frac{m}{n} \sim \frac{m'}{n'} \qquad \text{if and only if} \qquad mn' = m'n.$$

One typically writes

$$\frac{m}{n} = \frac{m'}{n'}$$

in that situation. How to define the rational numbers formally, and how that connects with this idea of fractions, is explained in Section 7.4.7.

### 7.4.5 Equivalence classes—modular arithmetic

When we have an equivalence relation we often do not wish to distinguish between elements which are equivalent. We look at one important example before we consider the general case.

When we calculate 'modulo' a given number we are not really interested in the numbers involved, just in the remainder they leave when dividing by the given number.

The formal definition of equivalence class is given on 396; for the current section on modular arithmetic we mean all elements that are equivalent to the given element when calculating modulo some number.

**Example 7.37.** For the simplest example assume that we are concerned only whether a number is odd or even. We know we have rules that allow us to calculate with 'even' and 'odd':

| $+$ | even | odd |
|---|---|---|
| even | even | odd |
| odd | odd | even |

| $\cdot$ | even | odd |
|---|---|---|
| even | even | even |
| odd | even | odd |

We may make this idea formal by defining an equivalence relation on $\mathbb{N}$, or on $\mathbb{Z}$, and then calculating with the equivalence classes.

For $i$ and $j$ in $\mathbb{N}$ (or in $\mathbb{Z}$) we set

$$i \sim j \qquad \text{if and only if} \qquad i \bmod 2 = j \bmod 2.$$

It is easy to check that this is an equivalence relation. We have two equivalence classes for this relation,

- the even numbers, all of which leave remainder 0 when divided by 2 and which all are in $[0]$
- the odd numbers, all of which leave remainder 1 when divided by 2 and all of which are in $[1]$.

So we have a new set where we distinguish between elements of $\mathbb{N}$ only up to odd- and evenness. This is known as the *quotient set*, and the formal notation is

$$\mathbb{N}/_{\sim} = \{[0], [1]\}.$$

This idea is formally introduced in Definition 57 below.

We can calculate with these equivalence classes by adding or multiplying them, defining[18]

| $+$ | $[0]$ | $[1]$ |
|---|---|---|
| $[0]$ | $[0]$ | $[1]$ |
| $[1]$ | $[1]$ | $[0]$ |

| $\cdot$ | $[0]$ | $[1]$ |
|---|---|---|
| $[0]$ | $[0]$ | $[0]$ |
| $[1]$ | $[0]$ | $[1]$ |

These tables fit those given for even and odd numbers above.

But, in fact, these operations may be derived from the addition and multiplication operations that exist on the set we started with, $\mathbb{N}$.

Note that if we have two pairs of numbers, say $i$, $j$, $i'$ and $j'$ in $\mathbb{N}$ with the property that

$$i \sim i' \qquad \text{and} \qquad j \sim j'$$

then both,

$$i + j \sim i' + j' \qquad \text{and} \qquad ij \sim i'j'.$$

To show this we have to go through all the possible cases:

- If $i$ and $j$ are both even then so are $i'$ and $j'$, and the sum of $i$ and $j$ is even, as is that of $i'$ and $j'$. In that case the product of $i$ and $j$ is also even, as is that of $i'$ and $j'$.
- If $i$ is even and $j$ is odd, then $i'$ is also even, and $j'$ is also odd. The sum of $i$ and $j$ is odd, as is that of $i'$ and $j'$, and the product of $i$ and $j$ is even, as is that of $i'$ and $j'$.
- If $i$ is odd and $j$ is even then by commutativity of addition and multiplication on $\mathbb{N}$ the argument from the previous case applies.
- If $i$ and $j$ are both odd then so are $i'$ and $j'$, and the sum of $i$ and $j$ is even, as is that of $i'$ and $j'$. The product of $i$ and $j$ is odd as is that of $i'$ and $j'$.

What this means is that in order to find the result of

$$[i] + [j]$$

all we have to do is pick any element of $[i]$, say $i'$, and any element of $[j]$, say $j'$, and calculate $i' + j'$—the result $[i] + [j]$ we are looking for is $[i' + j']$.

In other words, if we define

$$[i] + [j] = [i + j],$$

then this definition works. This is a non-trivial observation for the following reason:

What we have proved above is that, for example, we can pick any number in $[0]$, say 6, and any number in $[1]$, say 17, we can add them to each other and the result will tell us the result of $[0] + [1]$:

$$[0] + [1] = [6] + [17] = [6 + 17] = [23] = [1].$$

We might say that our equivalence relation is well-behaved for our given operations of addition and multiplication, and mathematicians might say that it is a *congruence relation with respect to both* $+$ *and* $\cdot$.

We refer to calculating 'modulo 2' when we think in this way. There is nothing special about 2, and below we consider calculations modulo other numbers.

[18] Note that we are using the operations $+$ and $\cdot$ in two different senses, once for numbers and once for equivalence classes.

**Exercise 162.** Determine the properties of both, multiplication and addition, for $\mathbb{N}/_\sim$. Are they associative or commutative? Do they have a unit? Do inverses exist for them? How does this compare with the properties of the corresponding operations for $\mathbb{N}$?

Mathematicians have a name for the set of equivalence classes from Example 7.37 with these operations, they call it the two-element field $\mathbb{F}_2$.

Calculating modulo 2, that is, with odd and even numbers, is just one of infinitely many such cases.

**Example 7.38.** We may look at calculating modulo 3, for $i$ and $j$ in $\mathbb{N}$ setting

$$i \sim j \qquad \text{if and only if} \qquad i \bmod 3 = j \bmod 3.$$

Like many relations based on an equality this is an equivalence relation as well. There are now three equivalence classes, namely

- the numbers divisible by 3, all of which are in $[0]$,
- the numbers which leave a remainder of 1 when divided by 3, all of which are in $[1]$ and
- the numbers which leave a remainder of 2 when divided by 3, all of which are in $[2]$.

Again we would like to add and multiply these equivalence classes, but it is a bit tedious to show separately for each $n$ that calculating modulo $n$ is safe, so we do it once and for all.

**Proposition 7.5**

Let $n$, $k$, $l$, $k'$ and $l'$ be natural numbers with the property that

$$k \bmod n = k' \bmod n \qquad \text{and} \qquad l \bmod n = l' \bmod n.$$

Then

$$\begin{aligned} & (k+l) \bmod n && = (k'+l') \bmod n \\ \text{and} \quad & kl \bmod n && = k'l' \bmod n. \end{aligned}$$

**Proof.** We know from Fact 2 that there exist unique $m$, $m'$, $i$, $i'$, $j$ and $j'$ in $\mathbb{N}$ with

$$0 \leq m, m' \leq n-1$$

and

$$k = in + m \qquad k' = i'n + m \qquad l = jn + m' \qquad l' = j'n + m'.$$

We calculate

$$k + l = in + m + jn + m' = (i+j)n + m + m'$$

and conclude[19] that

$$(k+l) \bmod n = (m+m') \bmod n.$$

We further calculate

$$k' + l' = i'n + m + j'n + m' = (i'+j')n + m + m'$$

and conclude that

$$(k' + l') \bmod n = (m + m') \bmod n = (k + l) \bmod n.$$

We next calculate

$$\begin{aligned} kl &= (in + m)(jn + m') \\ &= ijn^2 + im'n + jmn + mm' \\ &= (ijn + im' + jm)n + mm', \end{aligned}$$

and conclude that

$$kl \bmod n = mm' \bmod n.$$

Similarly we calculate

$$\begin{aligned} k'l' &= (i'n + m)(j'n + m') \\ &= i'j'n^2 + i'm'n + j'mn + mm' \\ &= (i'j'n + i'm' + j'm)n + mm', \end{aligned}$$

and so

$$k'l' \bmod n = mm' \bmod n = kl \bmod n$$

as required.

Hence for every number $n$ we may calculate with equivalence classes modulo $n$. Moreover, the resulting operations are commutative and associative, and have a unit. To see commutativity, note that for all $i$ and $j$ in $\mathbb{N}$ we have

$$[i] + [j] = [i + j] = [j + i] = [j] + [i].$$

The argument for associativity is similar. The unit for addition is $[0]$ since for all $i$ in $\mathbb{N}$ we have

$$[i] + [0] = [i + 0] = [i] = [0 + i] = [0] + [i].$$

The unit for multiplication is $[1]$, and the proof is very similar.

**Example 7.39.** We continue Example 7.38 by giving the tables for calculating modulo 3:

| $+$ | $[0]$ | $[1]$ | $[2]$ |
|---|---|---|---|
| $[0]$ | $[0]$ | $[1]$ | $[2]$ |
| $[1]$ | $[1]$ | $[2]$ | $[0]$ |
| $[2]$ | $[2]$ | $[0]$ | $[1]$ |

| $\cdot$ | $[0]$ | $[1]$ | $[2]$ |
|---|---|---|---|
| $[0]$ | $[0]$ | $[0]$ | $[0]$ |
| $[1]$ | $[0]$ | $[1]$ | $[2]$ |
| $[2]$ | $[0]$ | $[2]$ | $[1]$ |

**CExercise 163.** Carry out the following studies.

(a) Give tables for the addition and multiplication of equivalence classes when calculating modulo 4. Identify units for addition and multiplication if they exist. Determine whether you have inverses for addition and multiplication.

(b) Give tables for the addition and multiplication of equivalence classes when calculating

[19] This uses that for all $k$, $i$ and $n$ in $\mathbb{N}$ we have $k \bmod n = (in + k) \bmod n$, see CExercise 25.

modulo 5. Identify units for addition and multiplication if they exist. Determine whether you have inverses for addition and multiplication.

(c) Give tables for the addition and multiplication of equivalence classes when calculating modulo 6. Identify units for addition and multiplication if they exist. Determine whether you have inverses for addition and multiplication.

*Hint: You may want to go back to Section 2.5 to remind yourself of the notion of unit and inverse element for a binary operation.*

Modular arithmetic is important in cryptography. It also appears as an important example in COMP26120. For this reason we look at more of the properties it has.

From now on we use

$$k \sim_n l \qquad \text{to mean} \qquad k \bmod n = l \bmod n.$$

**Proposition 7.6**

The following hold for calculating modulo $n$.

(i) For all $n \in \mathbb{N} \setminus \{0, 1\}$, addition is commutative and associative for $\mathbb{N}/_{\sim_n}$; the additive unit is given by $[0]$ and every element of the set $\mathbb{N}/_{\sim_n}$ has an additive inverse.

(ii) For all $n \in \mathbb{N} \setminus \{0, 1\}$, multiplication is commutative and associative for $\mathbb{N}/_{\sim_n}$; the multiplicative unit is given by $[1]$, and the element $m$ of $\mathbb{N}/_{\sim_n}$ has a multiplicative inverse if and only if the greatest common divisor of $m$ and $n$ is 1.

In particular, if $n$ is a prime number then every number has both, an additive and a multiplicative inverse when calculating modulo $n$.

**Proof**. We show the two parts.

(i) Associativity and commutativity of addition follow immediately from Proposition 7.5 since, for example,

$$\begin{aligned} [k] + [l] &= [k + l] && \text{def } + \text{ on } \mathbb{N}/_{\sim_n} \\ &= [l + k] && \text{addition commutative on } \mathbb{N} \\ &= [l] + [k] && \text{def of } + \text{ on } \mathbb{N}/_{\sim_n}. \end{aligned}$$

The fact that $[0]$ is the additive unit also follows immediately from Proposition 7.5. Let $k$ be an arbitrary element of $\mathbb{N}$. By Fact 2 we know that $k \bmod n < n$. Hence we can set $l = n - (k \bmod n) \in \mathbb{N}$. Using the same proposition we calculate that

$$\begin{aligned} [k] + [l] &= [k \bmod n] + [l] \\ &= [(k \bmod n) + l] = [(k \bmod n) + n - (k \bmod n)] \\ &= [n] = [0] \end{aligned}$$

and in for much the same reason

$$[l + k] = [n] = [0].$$

(ii) Commutativity and associativity of multiplication follows in the same way as for addition. The fact that $[1]$ is the multiplicative unit also follows from the Proposition 7.5. Regarding multiplicative inverses below we give a method for calculating the multiplicative inverse by giving employing Euclid's algorithm (see Example 6.44) and from the definition of the method it is clear that such an inverse is produced whenever the two numbers we start with have 1 as their largest common divisor.

This completes the proof.

For public key cryptography the concept of *modular exponentiation* is particularly important. If we take the powers of a natural number then we eventually reach numbers that are too large for whichever format we are using. This cannot happen in modular arithmetic, because when calculating modulo $n$ we always get a number which is below $n$.

When performing these kinds of calculation then using the square brackets for equivalence classes becomes a bit tedious. What is typically done is to merely use the numbers

$$0, 1, 2, \ldots, n-2, n-1,$$

which represent their respective equivalence classes. Typical notation is then

$$2 + 3 = 5 == 0 \pmod 5 \qquad \text{or} \qquad 2 \cdot 4 = 8 = 3 \pmod 5,$$

where previously we would have written

$$[2] + [3] = [0] \qquad \text{or} \qquad [2] \cdot [4] = [3],$$

where we didn't have a good way of specifying modulo which number we were doing our calculations.

Note that when we are carrying out calculations in modular arithmetic we may, at any point, move to a different representative of the equivalence class.

**Example 7.40.** By moving to different elements of some equivalence class we can make sure that we always calculate with the smallest numbers possible.

$$\begin{aligned} 4 \cdot 16 + 2 \cdot 17 &= 64 + 34 \pmod 3 \\ &= 1 + 1 \pmod 3 \\ &= 2 \pmod 3. \end{aligned}$$

Note that it is customary to ensure that the final result is the canonical representative of its equivalence class, that is, when calculating modulo $n$, a number from 0 to $n-1$.

**Multiplicative inverses.** Finding multiplicative inverses modulo some given number is possible (where they exist), and we describe how this works.

Assume we have a number $n \in \mathbb{N} \setminus \{0, 1\}$, and we are looking for the multiplicative inverse of $m$, modulo $n$, that is we want to determine

$$m^{-1} \pmod n.$$

This means we are asking for a number $l$, the multiplicative inverse, with the property that

$$m \cdot l = 1 \pmod n,$$

which is equivalent to there being a number $i$ such that

$$m \cdot l = i \cdot n + 1.$$

Assume we know that the greatest common divisor of $m$ and $n$ is 1. If we apply Euclid's algorithm to $n$ and $m$, see Example 6.44, we can see that, for

$$r_0 = n \qquad \text{and} \qquad r_1 = m$$

we get equalities

$$\begin{aligned} r_0 &= k_1 \cdot r_1 + r_2 \\ r_1 &= k_2 \cdot r_2 + r_3 \\ &\dots \\ r_{n-2} &= k_{n-1} r_{n-1} + 1 \\ r_{n-1} &= k_n \cdot 1 + 0 \end{aligned}$$

that is at some point we have

$$r_n = 1 \qquad\qquad r_{n+1} = 0,$$

since the $r_i$ that appears before one obtains 0 is the greatest common divisor of $r_0 = n$ and $r_1 = m$.

We take the equalities and isolate the $r_i$ with the highest index appearing in each we obtain the following.

$$\begin{aligned} r_2 &= r_0 - k_1 \cdot r_1 \\ r_3 &= r_1 - k_2 \cdot r_2 \\ &\dots \\ r_n &= 1 = r_{n-2} - k_{n-1} \cdot r_{n-1} \end{aligned}$$

We can now recursively substitute each $r_i$ in the final equality according to the previous ones, which gives us an equality that expresses

$$1 \qquad \text{in terms of} \qquad r_0 = n \text{ and } r_1 = m.$$

This gives the desired solution for the original problem.

**Example 7.41.** We would like to calculate the multiplicative inverse for 5 modulo 7. Following Euclid's algorithm we calculate as follows

$$\begin{aligned} 7 &= 1 \cdot 5 + 2 \\ 5 &= 2 \cdot 2 + 1 \end{aligned}$$

We reorganize the two equalities as instructed above to obtain

$$\begin{aligned} 2 &= 7 - 1 \cdot 5 \\ 1 &= 5 - 2 \cdot 2. \end{aligned}$$

We insert the right hand side of the first equality for the right hand 2 in the second equality and get

$$1 = 5 - 2 \cdot (7 - 1 \cdot 5) = 3 \cdot 5 - 2 \cdot 7,$$

and so

$$3 \cdot 5 = 2 \cdot 7 + 1.$$

Hence the multiplicative inverse of 5 modulo 7 is 3. And indeed,

$$3 \cdot 5 = 15 = 2 \cdot 7 + 1 = 1 \pmod 7.$$

**Example 7.42.** We carry out another example calculation. The multiplicative inverse of 4 modulo 11 can be found as follows.

$$\begin{aligned} 11 &= 2 \cdot 4 + 3 \\ 4 &= 1 \cdot 3 + 1 \end{aligned}$$

Rearranging gives

$$\begin{aligned} 3 &= 11 - 2 \cdot 4 \\ 1 &= 4 - 1 \cdot 3 \end{aligned}$$

Inserting the first into the second shows that

$$1 = 4 - (11 - 2 \cdot 4) = 3 \cdot 4 - 11.$$

Hence the multiplicative inverse for 4 is 3, and indeed,

$$4 \cdot 3 = 12 = 1 \pmod{11}.$$

**Example 7.43.** In cases where the multiplicative inverse does not exist the algorithm proceeds as follows. Note that Euclid's algorithm calculates the greatest common divisor of the two given numbers, Compare Example 6.44. If the greatest common divisor of the two given numbers is a number other than 1 we cannot use this idea, and indeed in that case there is no multiplicative inverse for the chosen modular arithmetic. We wish to find a multiplicative inverse for 4 modulo 6.

$$\begin{aligned} 6 &= 1 \cdot 4 + 2 \\ 4 &= 2 \cdot 2 + 0. \end{aligned}$$

In other words we reach 0 as the remainder without having reached 1 first.

**Example 7.44.** We give one final example. We wish to calculate the multiplicative inverse for 11 modulo 17.

$$\begin{aligned} 17 &= 1 \cdot 11 + 6 \\ 11 &= 1 \cdot 6 + 5 \\ 6 &= 1 \cdot 5 + 1 \end{aligned}$$

Rearranging give

$$\begin{aligned} 6 &= 17 - 1 \cdot 11 \\ 5 &= 11 - 1 \cdot 6 \end{aligned}$$

$$1 = 6 - 1 \cdot 5.$$

Inserting the first two into the final equality tells us that

$$\begin{aligned} 1 &= 6 - 1 \cdot 5 && \text{given} \\ &= 6 - (11 - 1 \cdot 6) && \text{snd equality into third} \\ &= 2 \cdot 6 - 11 && \text{simplification} \\ &= 2 \cdot (17 - 1 \cdot 11) - 11 && \text{fst equality into result so far} \\ &= 2 \cdot 17 - 3 \cdot 11 && \text{simplification.} \end{aligned}$$

So the multiplicative inverse of 11 modulo 17

$$-3 = 14 \pmod{17}.$$

Again we can verify that

$$14 \cdot 11 = 154 = 9 \cdot 17 + 1 = 1 \pmod{17}.$$

Make sure that the answers you give are indeed numbers from 0 to $n - 1$ when calculating modulo $n$ in this notation. If in the previous example you give $-3$ as an answer you will lose marks since this number is not an element of the set we are considering.

Note that 0 cannot have a multiplicative inverse as is shown Proposition 2.1.

**Exponentiation.** The modular operation of most interest in cryptography is that of exponentiation, and we look at the basics here.

If we look at the powers of 2 $\pmod{5}$ we find that they are

| $n$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $2^n \pmod{5}$ | 1 | 2 | 4 | 3 | 1 | 2 | 4 | 3 | 1 | 2 | ... |

Of course the numbers used in cryptography are considerably larger than 5. For RSA, for example, two large primes are selected, and then exponentiation modulo their product is performed. For this and other applications it becomes important to be able to carry out exponentiation efficiently.

We look at the question of how to calculate

$$a^b \pmod{n}.$$

**Code Example 7.1.** The really naive method is to calculate $a$ to the power of $b$ as integers, and then calculate the remainder when dividing by $n$, that is[20]

```
public static int modpower (int a, int b, int n)
{
    if (b==0)
        return 1;
    else {
        int power = 1;
```

```
        for (int count=1, count <= b, count++)
            power = (power * a);
        return power % n;
    }
}
```

Using this code quickly runs out of memory[21] despite the fact that we know that the result is a number from 0 to $n-1$. We look at a sample calculation of

$$13^5 \pmod{197}.$$

The table gives the value of power as the loop progresses.

| count | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| power | 13 | 169 | 2197 | 28561 | 371293 |

We then take the final result and calculate

$$371293 \bmod 197 = 145.$$

**Code Example 7.2.** The slightly less naive algorithm is to just keep multiplying by the base and calculating the remainder when dividing by $n$ at each step.

```
public static int modpower (int a, int b, int n)
{
    if (b==0)
        return 1;
    else {
        int power = 1;
        for (int count=1, count <= b, count++)
            power = (power * a) % n;
        return power;
    }
}
```

This works better but still is not very efficient. It performs $b$ many multiplications and remainder operations. Again, here is a sample calculation of the same number as before.

| count | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| power | 13 | 169 | 30 | 193 | 145 |

This program does not run out of memory as the first one inevitably does.

Note that this works because of Proposition 7.5: We want to calculate

$$a^b \pmod{n},$$

[20]Note that one could improve efficiency by checking whether $a$ is equal to 0 or 1, but this would not improve matters much and we are looking for a short program.

[21]even if you pick one of the large integer classes in Java instead of **int**.

but we know that

$$i \cdot j \pmod{n} \qquad \text{has the same result as} \qquad (i \bmod n) * (j \bmod n),$$

and so taking the remainder when dividing by $n$ at each step of the exponentiation process is the same as taking it at the end:

$$\begin{aligned} a^b \bmod n &= (a \cdot a^{b-1}) \bmod n \\ &= (a \cdot (a^{b-1} \bmod n)) \bmod n \qquad \text{Prop 7.5.} \end{aligned}$$

Indeed, one way of paraphrasing that proposition is to say that when adding or multiplying $\pmod{n}$ one may form the remainder when dividing by $n$ at any time![22]

**Code Example 7.3.** In COMP26120 you will look at a third method for performing this calculation.

```
public static int modpower (int a, int b, int n)
{
    if (b==0)
        return 1;
    else {
        int power = 1;
        while (b != 0) {
            if (b % 2 == 1)
                power = (power * a) % n;
            a = (a*a) % n;
            b = b/2;
        }
        return power;
    }
}
```

Again we perform a sample calculation, but this time we have to track not just of the value held in the variable power, but also of what is held in the variables a and b, the latter of which controls the **while** loop. If that variable is even then nothing happens in the **while** loop, which we denote in the table by an empty cell for power. We repeat the above calculation, so at the start we have $a = 13$, $b = 5$ and $n = 197$. The method sets the variable power to 1, so we use that as our first value for that variable.

| b | | 5 | 2 | 1 | 0 |
|---|---|---|---|---|---|
| power | 1 | 13 | | 145 | |
| a | 13 | 169 | 193 | 16 | |

Because this is a short calculation it looks as if we have just as many steps to perform as before, since we now have to calculate changing values for a as well. But you can see that when $b$ is large the fact that the **while** loop is carried out only $\log b$ many times becomes more important than the fact that each time we have between one and two multiplications to carry out.

[22] In COMP11212 you will learn about *loop invariants* and then you can make a rigorous step-by-step argument that the two programs compute the same number.

The idea behind this algorithm is quite simple: Instead of multiplying with $a$ the required number of times the number of multiplications is brought down by the squaring operation. To do this one may think of $b$ in base 2.

**Example 7.45.** We illustrate how this works in an example: For $b = 5$, which is 101 in binary, we have

$$a^5 = a^4 \cdot a^1 = (a^2)^2 \cdot a^1 = a^{1\cdot 2^2} \cdot a^{0\cdot 2^1} \cdot a^{1\cdot 2^0},$$

that is, we have one count of an iterated square of $a$ for each position where $b$ in binary has 1. A larger example is $b = 21$, where we have

| | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|
| $b$ | 1 | 0 | 1 | 0 | 1 |
| powers | $(((a^2)^2)^2)^2$ | | $(a^2)^2$ | | $a$ |

and so

$$a^{21} = a^{1\cdot 2^4} \cdot a^{0\cdot 2^3} \cdot a^{1\cdot 2^2} \cdot a^{0\cdot 2^1} \cdot a^{1\cdot 2^0}.$$

The algorithm ensures that what is multiplied is the appropriate selection of iterated squares.

**Example 7.46.** We carry out one more example that illustrates how the two algorithms work with numbers where it's a bit easier to follow the calculation. We calculate

$$7^7 \pmod{11}.$$

We follow the second algorithm.

| $n$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $7^n \pmod{11}$ | 7 | 5 | 2 | 3 | 10 | 4 | 6 |

For the third algorithm at the start we have $a = 7$, $b = 7$, $n = 11$.

| b | 7 | 7 | 3 | 1 | 0 |
|---|---|---|---|---|---|
| power | 1 | 7 | 2 | 6 | |
| a | 7 | 5 | 3 | 9 | |

**EExercise 164.** Carry out the following tasks.

(a) Find the multiplicative inverses, where they exist, for

$$7 \pmod{11}, \qquad 15 \pmod{17}, \qquad 8 \pmod{12}, \qquad 5 \pmod{9}.$$

*Hint: See Examples 7.41, 7.42, 7.43 and 7.44.*

(b) Calculate the following using the two algorithms from Code Examples 7.2 and 7.3.

$$4^5 \pmod{13}, \qquad 7^4 \pmod{9}, \qquad 6^5 \pmod{11}, \qquad 8^6 \pmod{16}.$$

(c) Depending on $b$, how many times is the loop in the algorithm from Code Example 7.2 carried out? What about the other algorithm?

(d) For which of the following can you find an exponent different from 0 so that the given number is equal to 1 in the relevant modular arithmetic? Give the smallest such exponent if possible, argue why there is none otherwise.

$$7 \pmod{11}, \qquad 15 \pmod{17}, \qquad 8 \pmod{12}, \qquad 5 \pmod{9}.$$

*Hint: There is no algorithm that determines the required number for all cases.*

### 7.4.6 General equivalence classes

In the previous section we looked at a particular situation where we want to treat equivalent elements at the same. In general we may want to do this when we have an equivalence relation. In this situation we construct another set where there is only one representative for each class of equivalent elements, just as in $\mathbb{N}/_{\sim_2}$ there are only two elements, $[0]$ and $[1]$.

Given a set $S$ with an equivalence relation $R$ and an element $s$ of $S$ the **equivalence class with respect to $R$ generated by $s$**, $[s]$, is the subset of $S$ consisting of all elements of $S$ which are related to $s$ by $R$, that is

$$[s] = \{s' \in S \mid (s, s') \in R\}.$$

Note that since an equivalence relation is symmetric this is the same as

$$\{s' \in S \mid (s', s) \in R\}.$$

If the equivalence relation in question is not clear from the context, the equivalence class generated by $s$ for a relation $R$ is written as $[s]_R$. Usually equivalence relations are written in infix notation and for the remainder of this section that is the notation we use.

**Example 7.47.** We go through the examples given in Example 7.35.

(a) For the relations given by functions we have the following equivalence classes:

(i) building blocks of the same shape,

(ii) groups of people of the same height (up to the nearest centimetre),

(iii) tutorial groups,

(iv) all people of the same nationality,

(v) implementations of the same algorithm.

(b) This example is too general to give the equivalence classes.

(c) All sets which have the same size as a given form an equivalence class (for example, all sets with 5 elements, or all countably infinite sets).

(d) Algorithms which belong to the same complexity class form an equivalence class.

(e) All propositions with the same boolean interpretation (with respect to every valuation) form an equivalence class.

(f) Functions are in the same equivalence class if they eventually dominate each other, up to a factor.

(g) Objects of class List are in the same equivalence class for equal if and only if the underlying elements of $\mathsf{Lists}_{\mathbb{Z}}$ have the same elements in the same order.

Alternatively we can describe equivalence classes for this relation as follows. Two objects l1 and l2 of the class List are in the same equivalence class if and only if

- their instance variables l1.value and l2.value give the same integer value and
- their instance variables l1.next and l2.next are references to lists which are in the relation.

Example 6.14 looks at a Java method that implements this idea by taking two lists as input and returning true if and only if the two lists are equivalent in this sense.

**Example 7.48.** We may turn the last example from above into a recursively defined set, see Section 6.4.3. We want to create a set that contains all those pairs

$$(\mathrm{l1}, \mathrm{l2})$$

where l1 and l2 are List objects, such that l1 andl2 are equivalent. We may do this using the following recursive definition.

**Base case** $E_{\text{List}}$. $(\textbf{null}, \textbf{null}) \in E_{\text{List}}$.

**Step case** $E_{\text{List}}$. $(\mathrm{l1}, \mathrm{l2}) \in E_{\text{List}}$ and m==n for m:**int** and n:**int** implies

$$(\textbf{new}\ \text{List (m,l1)}, \textbf{new}\ \text{List (n,l2)}) \in E_{\text{List}}.$$

The base case tells us that two List objects that are a **null** reference are considered equivalent, and using the step case repeatedly we can build up to longer and longer lists being considered equal, where we should think of m and n as expressions of type **int** which evaluate to the same number.

Equivalence classes split the given set into disjoint blocks of equivalent elements and we say that they *partition* the set. In other words, they give us a new set where we no longer distinguish between equivalent elements.

**Definition 57: quotient set**

Given a set $S$ with an equivalence relation $\sim$, the **quotient set of $S$ with respect to $\sim$** consists of the equivalence classes of $S$ with respect to $\sim$. This is written as $S/_{\sim}$.

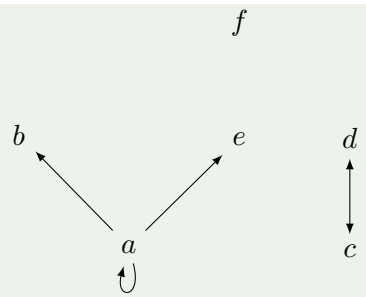
**Takeaway**

It's possible to think of all equivalent items as just one element.

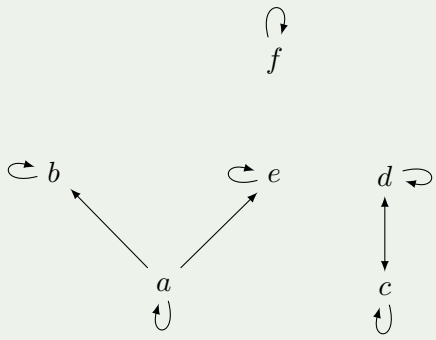
**Example 7.49.** We begin our study of formal examples with a relation on a small finite set $\{a, b, c, d, e, f\}$, namely

$$\{(a, a), (a, b), (a, e), (c, d), (d, c)\}.$$

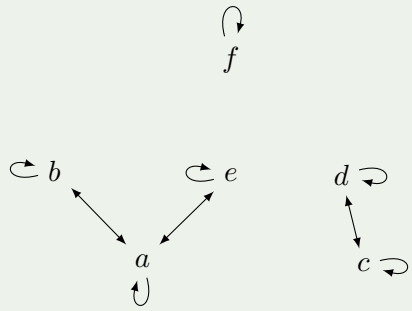
As before we picture the relation using a directed graph.



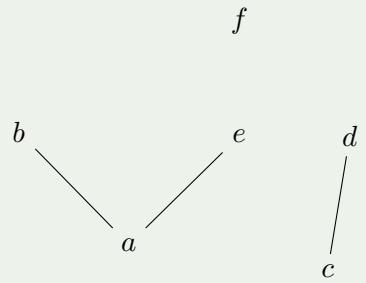

The reflexive closure of this relation adds connections to itself for each element:



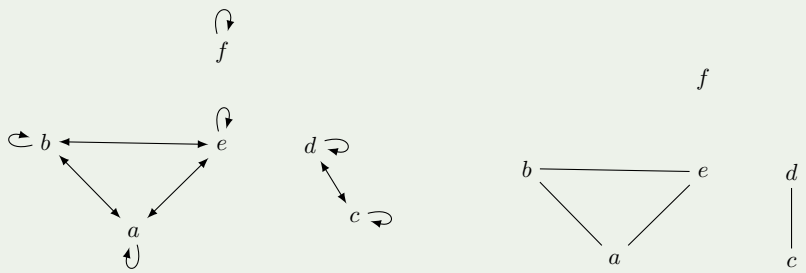

The symmetric closure of the relation turns all edges between different elements into double-tipped ones:



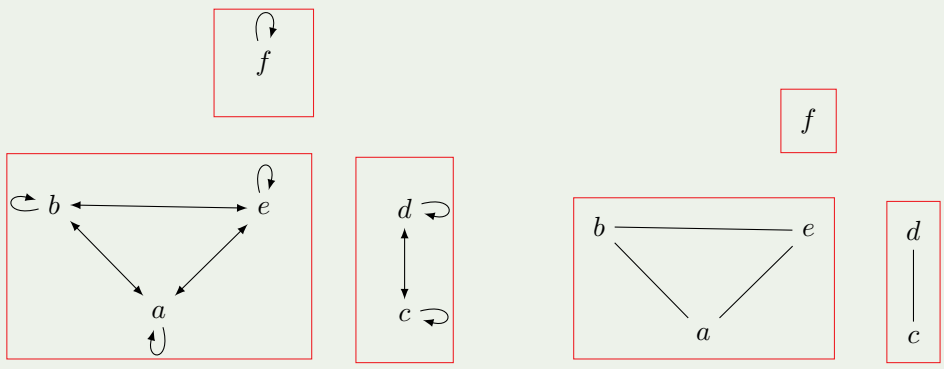

If we draw the result as an undirected graph we lose some of the now redundant information:



The transitive closure of this relation, both as a directed and as an undirected graph:




We draw the equivalence classes in both graphs:




We have three equivalence classes:

$$[a] = [b] = [e] = \{a, b, e\}, \qquad [c] = [d] = \{c, d\}, \qquad [f] = \{f\}.$$

Note the following: If we pick any pair of nodes in an equivalence class then there is a connection between them (and this is a two-sided connection in the case of a directed graph). This is true for all equivalence classes in all equivalence relations.

**Example 7.50.** We look at another example. In Chapter 3, an algorithm is described which, given a propositional formula, arrives at a conjunctive normal form for that formula. We may think of this as defining a relation on propositional formulae.

If we also allow the rules for simplifying formulae, and take the reflexive symmetric transitive closure of the resulting relation, we obtain an equivalence relation for propositional formulae.

Two formulae are equivalent for that relation if and only if, starting with both given formulae, we can apply the rules in such a way that we arrive at the same CNF.

This is the same equivalence relation as that which considers two formulae equivalent if and only if for every valuation they give the same truth table.

The equivalence class of a formula, say $P$, is then the set of all formulae that have the simplified CNF $P$, such as $P \wedge P$ or $P \vee P \vee \bot$.

**Example 7.51.** If you are writing a programme where queues are implemented, and there is one central resource that all existing queues need to access (for example processing time) then you might want to implement a procedure which allocates the resource to the longest

queue. From the point of view of that program, it is only important how long the queues are, and not what elements they have. We use this idea, but for our previously defined type of list.

We define a binary relation on $\mathsf{Lists}_S$ where

$$l \sim l' \qquad \text{if and only if} \qquad \operatorname{len} l = \operatorname{len} l',$$

using the $\operatorname{len}$ function from CExercise 117.

Two elements of $\mathsf{Lists}_S$ are in the relation $\sim$ if and only if they have the same number of elements. This means that there are infinitely many equivalence classes, one for each natural number. The empty list is the only element of its equivalence class, but as long as $S$ has more than one element there is more than one element in all the other equivalence classes. For example, for the list $[s]$ the equivalence class consists of all the $[s']$ for which $s' \in S$.

One can define a corresponding relation on objects of class List where

l1 is related to l2

if and only if the following method, called as

length(l1)==length(l2),

where length is the method from Section 6.1.3 defined in detail in CExercise 117.

Note that whenever a set is *partitioned*, that is, split into disjoint subsets, there is an equivalence relation at the heart: Given a set $S$, all we have to do is to define

$$s \sim s' \qquad \text{if and only if} \qquad s \text{ and } s' \text{ are in the same partition.}$$

In this case the equivalence classes are exactly the partitions. This means that partitioning a set is exactly the same thing as forming the equivalence classes for an equivalence relation.

**CExercise 165.** For the following relations, calculate the equivalence relation they generate, try to[23] describe the resulting equivalence classes, and count their number.

(a) The reflexive symmetric transitive closure of the following relation on the set consisting of the elements $a$, $b$, $c$, $d$, $e$ and $f$:

$$\{(a,b),(b,a),(b,c),(d,e),(e,f)\}.$$

(b) The reflexive symmetric transitive closure of the following relation on the same set as in the previous part:

$$\{(a,a),(b,c)\}.$$

(c) The reflexive symmetric transitive closure of the following relation on the same set as in the previous part:

$$\{(a,a),(b,c),(c,d),(d,e)\}.$$

(d) On the set $\mathbb{N}$ the relation $m \sim n$ if and only if $m \bmod 4 = n \bmod 4$. What is $[1]$ in this example? Can you describe $\mathbb{N}/_\sim$?

(e) Consider the set $D$ of decimal numbers. We assume here that such a number consists of a finite number of digits from 0-9, followed by a decimal point, followed by an infinite

number of digits 0–9. (Note that this is not how we usually write decimal numbers—we drop all (or most) of the infinitely many 0s that appear.) Take the reflexive symmetric closure $\sim$ of the relation where two numbers are related if and only if:

- the first number ends with infinitely many 0s,
- the second number ends with infinitely many 9s and
- the (finitely many) digits to the left of these are equal, with the exception of the right-most such digit, which is one less for the second number.

What is $[1]$ in this example? Can you describe $D/_\sim$?

(f) On the natural numbers $\mathbb{N}$ the relation where $m \sim n$ if $m + n$ is even (compare CExercise 160). Can you describe $\mathbb{N}/_\sim$?

(g) On the complex numbers the relation where $a + bi$ is related to $a' + b'i$ if and only if $a = a'$.

(h) On the complex numbers the relation where $z$ is related to $z'$ if and only if $z\overline{z} = z'\overline{z'}$.

**Example 7.52.** We recursively define a binary relation on the set $\mathsf{Lists}_S$ of lists over the set $S$ as follows.

**Base case** $\sim$. $\quad [\,] \sim [\,]$

**Step case** $\sim$. For $l \sim l'$ and $s, s' \in S$ we have

$$s : l \sim s' : l'.$$

The remainder of this example is concerned with understanding what this relation does. We show first of all that it is a reflexive relation, that is, that each list is related to itself. This is a proof by induction.

**Base case** $\sim$. We note that

$$[\,] \sim [\,] \qquad\qquad \text{base case } [\,].$$

**Ind hyp**. We assume the statement holds for the list $l$, that is $l \sim l$.

**Step case** $\sim$. We see that given $s \in S$ we have that

$$s : l \sim s : l \qquad\qquad \text{by step case } \sim .$$

We can also show by induction that this relation is also symmetric and transitive. We add a proof for the former.

[23] For the finite examples you should list all equivalence classes.

**Base case $\sim$.** We note that

$$[\,] \sim [\,] \qquad\qquad \text{base case } [\,],$$

and so the only instance of using the base case of the definition of $\sim$ results in a symmetric relation.

**Ind hyp.** We assume the statement holds for the lists $l$ and $l'$, that is $l \sim l'$ implies that $l' \sim l$.

**Step case $\sim$.** We see that the only way of building further instances of the relation $\sim$ is to take note that, for $l \sim l'$, as well as $s$ and $s'$ in $S$ we get

$$s : l \sim s' : l' \qquad\qquad \text{by step case } \sim,$$

but we also get, from the induction hypothesis, that $l' \sim l$, and so

$$s' : l' \sim s : l \qquad\qquad \text{by step case } \sim,$$

and so the relation remains symmetric as we add additional instances.

So what does this relation do? One possibility is to look at some examples, and let's assume that the underlying set $S$ is $\mathbb{N}$. We know that we start with

$$[\,] \sim [\,].$$

The step case tells us that we can now add any one element to $\emptyset$ and all the resulting lists are related, so

$$[0] \sim [1] \sim [2], \ldots$$

and all lists of length one are related. We can take any two of these, and add an element to each, and get two more related lists, which means that all lists of length 2 are related.

This gives rise to the conjecture that for all lists $l$ and $l'$ we have

$$l \sim l' \qquad \text{if and only if} \qquad \operatorname{len} l = \operatorname{len} l'.$$

We give a formal proof of this. This is an 'if and only if' statement and we show it by proving each part separately.

We first show that if $l \sim l'$ then $l$ and $l'$ have the same length.

**Base case $\sim$.** We have the base case of the relation,

$$[\,] \sim [\,],$$

and we can see that both sides are equal, so applying the length function gives the same result.

**Ind hyp.** For the lists $l$ and $l'$ we have that $l \sim l'$ implies $\operatorname{len} l = \operatorname{len} l'$.

**Step case** $\sim$. If we have lists $l$ and $l'$ with $l \sim l'$ and for $s$ and $s'$ in $S$ we use the step case of $\sim$ to derive that

$$s : l \sim s' : l'$$

we may conclude that

$$\begin{aligned} \operatorname{len}(s : l) &= 1 + \operatorname{len} l && \text{step case len} \\ &= 1 + \operatorname{len} l' && \text{ind hyp} \\ &= \operatorname{len}(s' : l') && \text{step case len} \end{aligned}$$

In the other direction we want to show that if for two lists $l$ and $l'$ we know that $\operatorname{len} l = \operatorname{len} l'$ then we have $l \sim l'$. How do we know that the two lengths are equal? This can only happen by another inductive process: The length of a list can only be 0 if the list is empty, which gives the base case. If we have two lists $l$ and $l'$ of equal lengths, then for $s$ and $s'$ in $S$ we have

$$\begin{aligned} \operatorname{len}(s : l) &= 1 + \operatorname{len} l && \text{step case len} \\ &= 1 + \operatorname{len} l' && \text{assumption} \\ &= \operatorname{len}(s' : l'). \end{aligned}$$

**Base case** $\sim$. We have the base case of equal length

$$\operatorname{len}[\,] = \operatorname{len}[\,],$$

and we can see that

$$[\,] \sim [\,]$$

by the base case of $\sim$.

**Step case**. If we have lists $l$ and $l'$ of equal length, which allows us, for $s$ and $s'$ in $S$, to conclude that

$$\operatorname{len}(s : l) = \operatorname{len}(s' : l'),$$

then we get that $l \sim l'$ from the induction hypothesis and we may conclude that

$$s : l \sim s' : l'$$

by the step case of $\sim$.

Hence the equivalence relation defined in this example is the same as that from Example 7.51.

Understanding recursive definitions is not easy, and typically one wants to find an alternative description that is easier to grasp.

**CExercise 166.** Consider the following relation on $\mathsf{FBTrees}_S$.

**Base case** $\sim$. For all $s, s' \in S$ we have

$$\mathtt{tree}\, s \sim \mathtt{tree}\, s'.$$

**Step case** $\sim$. For $t \sim t'$, $t'' \sim t'''$ and $s, s' \in S$ we have

$$\mathtt{tree}_s(t, t'') \sim \mathtt{tree}_{s'}(t', t''').$$

(a) Which of the following trees are $\sim$-related?

(i) $\texttt{tree}_2(\texttt{tree}\,1, \texttt{tree}\,3)$

(ii) $\texttt{tree}_3(\texttt{tree}\,2, \texttt{tree}_3(\texttt{tree}\,2, \texttt{tree}\,1))$

(iii) $\texttt{tree}_3(\texttt{tree}_2(\texttt{tree}\,1, \texttt{tree}\,1), \texttt{tree}_2(\texttt{tree}\,3, \texttt{tree}\,2))$.

(iv) $\texttt{tree}_3(\texttt{tree}_2(\texttt{tree}\,1, \texttt{tree}\,3), \texttt{tree}_2(\texttt{tree}\,3, \texttt{tree}\,1))$.

(v) $\texttt{tree}_3(\texttt{tree}_2(\texttt{tree}\,1, \texttt{tree}\,3), \texttt{tree}\,1))$.

(vi) $\texttt{tree}_3(\texttt{tree}_2(\texttt{tree}\,1, \texttt{tree}\,1), \texttt{tree}_2(\texttt{tree}\,3, \texttt{tree}\,2))$.

(vii) $\texttt{tree}_3(\texttt{tree}\,2, \texttt{tree}\,3)$

*Hint: You may want to draw them and then think about which trees are related. Begin with trees related by the base case, and then apply the step case once or twice to see how it all works*

(b) Prove by induction that the relation is reflexive. In fact, it is an equivalence relation and you may use this to answer the following part.

(c) Informally describe the equivalence classes of this relation. How would you describe all the trees that are equivalent to a given one?

(d) How would you implement a corresponding relation for objects of class BTree? You are trying to write a method with the following first line:

**public static boolean** similar (BTree t1, BTree t2)

The method should return true exactly when the two input trees are related by $\sim$.

**Exercise 167.** Show the following for an arbitrary equivalence relation $\sim$ on a set $S$.

(a) For all $s \in S$ we have $s \in [s]$.

(b) For all $s$, $s'$, $s''$ in $S$ we have

$$s', s'' \in [s] \qquad \text{implies} \qquad s' \sim s''.$$

(c) For all $s$ and $s'$ in $S$ we have that

$$s \sim s' \qquad \text{implies} \qquad [s] = [s'].$$

(d) For all $s$ and $s'$ in $S$ we have

$$[s] \cap [s'] \text{ is either empty or equal to } [s].$$

(e) For every element of $S$ there is an equivalence class it belongs to.

(f) The equivalence classes for the relation split $S$ into a pairwise disjoint collection of sets.

Hence the equivalence classes of $S$ partition the whole set into disjoint subsets.

While it may be useful to determine a new set which has only one representative for each equivalence class, this construction becomes significantly more useful when we have sets with operations on them. The following sub-section cover relevant examples.

### 7.4.7 The integers and the rationals

The point of this and the following account of the rationals is to show you how to formally define these numbers with their operations. Above we give an account of how to formally define the natural numbers and the usual operations for those, and that allows us to prove properties given as facts in Chapter 0. In this section we look at how to do something similar for the integers and the rational numbers. We don't give as many examples of operations, but we do show how to define addition and multiplication (based on those for the natural numbers).

The material is not examinable as such, but does provide more examples for using equivalence relations and understanding it may be helpful for answering other exam questions.

#### The integers

We look at a formal definition of the integers.

Consider the following set:

$$\mathbb{N} \times \mathbb{N} = \{(m, n) \mid m, n \in \mathbb{N}\}.$$

On this set we define a relation, namely

$$(m, n) \sim (m', n') \qquad \text{if and only if} \qquad m + n' = m' + n.$$

It is easy to see that this is an equivalence relation: Since for all $m$ and $n$ in $\mathbb{N}$ we have that $m + n = m + n$ it is reflexive, and symmetry is obvious from the definition. Transitivity is obtained as follows:

$$(m, n) \sim (m', n') \qquad \text{and} \qquad (m', n') \sim (m'', n''),$$

imply

$$m + n' = m' + n \qquad \text{and} \qquad m' + n'' = m'' + n'$$

respectively, and together these imply that

$$m + n' + m' + n'' = m' + n + m'' + n'$$

which by commutativity and assoiativity of addition is equivalent to

$$(m + n'') + (n' + m') = (m'' + n) + (m' + n')$$

which by the final statement of Fact 1 implies that

$$m + n'' = m'' + n.$$

which means that we have

$$(m, n) \sim (m'', n'').$$

We define an operation we call 'addition' on this set, by setting

$$(m, n) \oplus (m', n') = (m + m', n + n').$$

We next show that

$$\begin{aligned} &(k, l) \sim (m, n) \text{ and } (k', l') \sim (m', n') \\ &\text{implies} \qquad (k, l) \oplus (k', l') \sim (m, n) \oplus (m', n'), \end{aligned}$$

by calculating

$$(k,l) \oplus (k',l') = (k+k', l+l') \qquad \text{and} \qquad (m,n) \oplus (m',n') = (m+m', n+n')$$

and

$$\begin{aligned} k+k'+n+n' &= (k+n) \oplus (k'+n') && \text{comm, ass } + \\ &= (m+l) \oplus (m'+l') && (k,l) \sim (m,n),\ (k',l') \sim (m',n') \\ &= m+m'+l+l' && \text{comm, ass } + . \end{aligned}$$

This allows us to define an addition operation *on the set of equivalence classes with respect to* $\sim$, $(\mathbb{N} \times \mathbb{N})/_\sim$.

Given two arbitrary equivalence classes for our set, if we pick two respective elements, say $(k,l)$ and $(m,n)$ we may set

$$[(k,l)] + [(m,n)] = [(k,l) \oplus (m,n)]$$

since our previous calculation assures us that no matter which elements of each equivalence class we pick, the result of adding the two always determines the same equivalence class.

It is fairly easy to show that addition is commutative and associative for our original set, $\mathbb{N} \times \mathbb{N}$, and the same is then true for the derived operation on $(\mathbb{N} \times \mathbb{N})/_\sim$. We can also check that there is a unit, namely the equivalence class of $(0,0)$: Given $(m,n) \in \mathbb{N} \times \mathbb{N}$ we have

$$[(m,n)] \oplus [(0,0)] = [(m+0, n+0)] = [(m,n)] = [(0,0)] \oplus [(m,n)].$$

This raises the question of whether we have inverses with respect to this operation. In other words, given $(m,n) \in \mathbb{N} \times \mathbb{N}$, can we find an element whose equivalence class we can add to $[(m,n)]$ such that the result is $[(0,0)]$? To find the answer to this question it is a good idea to first think about what the elements of $[(0,0)]$ are. An element $(m,n)$ of our set is equivalent to $(0,0)$ if and only if

$$m+0 = 0+n,$$

that is, if $m = n$.

Given $(m,n) \in \mathbb{N} \times \mathbb{N}$ we note that

$$[(m,n) \oplus [(n,m)] = [(m+n, n+m)] = [(0,0)],$$

and so we do indeed have an inverse element, namely $[(n,m)]$ for $[(m,n)]$. In other words, $(\mathbb{N} + \mathbb{N})/_\sim$ is a commutative group with respect to addition. Indeed, this is nothing but an alternative description for a group you are all familiar with.

As a preliminary calculation we note that for $(m,n) \in \mathbb{N} \times \mathbb{N}$ with $m \geq n$ we have

$$[(m,n)] = [(m-n, 0)],$$

since

$$m+0 = m = m-n+n$$

and so

$$(m,n) \sim (m-n, 0).$$

Consider the following function from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{Z}$:

$$(m,n) \longmapsto m-n.$$

This function is constant on equivalence classes, since if we have

$$(m, n) \sim (m', n')$$

then we have

$$m - n = m + n' - n' - n = m' + n - n' - n = m' - n'.$$

Hence we may define a function $f$ from $(\mathbb{N} \times \mathbb{N})/_\sim$ to $\mathbb{Z}$ by setting

$$f\colon [(m, n)] \longmapsto m - n,$$

without worrying which element of $[(m, n)]$ we may be referring to. We further define a function $g$ from $\mathbb{Z}$ to $(\mathbb{N} \times \mathbb{N})/_\sim$ by setting

$$g\colon i \longmapsto \begin{cases} [(i, 0)] & i \geq 0 \\ [(0, -i)] & \text{else.} \end{cases}$$

We claim that this defines a bijection between the two sets: Given $i \in \mathbb{Z}$ we calculate

- if $i \geq 0$ we have $f(gi) = f[(i, 0)] = i - 0 = i$, and
- if $i < 0$ we have $f(gi) = f[(0, -i)] = 0 - (-i) = i$.

Given $(m, n) \in \mathbb{N} \times \mathbb{N}$ we have that $f[(m, n)] = m - n$, and

- if $m - n \geq 0$ we have that

$$g(f[(m, n)]) = g(m - n) = [(m - n, 0)] = [(m, n)],$$

- if $m - n < 0$ we have that

$$g(f[(m, n)]) = g(m - n) = [(0, -(m - n))] = [(0, -m + n))] = [(m, n)].$$

This shows that the two functions are mutual inverses. But much more is true: These two functions *preserve the addition operation*, that is, we have, for $(k, l)$ and $(m, n)$ in $\mathbb{N} \times \mathbb{N}$ that

$$\begin{aligned} f[(k, l)] + f[(m, n)] &= (k - l) + (m - n) && \text{def } f \\ &= (k + m) - (l + n) && \text{ass, comm } +, \text{ EEx 34} \\ &= f[(k + m, l + n)] && \text{def } f \\ &= f([(k, l)] + [(m, n)]) && \text{def addition,} \end{aligned}$$

and for $i$ and $j$ in $\mathbb{Z}$ we have

$$gi + gj = [(i, 0)] + [(j, 0)] = [(i + j, 0)] = g(i + j).$$

This tells us that our set $(\mathbb{N} + \mathbb{N})/_\sim$ really is describing the set $\mathbb{Z}$ with its addition operation by simply giving a different name to all the elements.

Indeed, the *formal definition of the set of integers*, $\mathbb{Z}$, is $(\mathbb{N} \times \mathbb{N})/_\sim$, where we use the integer $m - n$ as a name for the equivalence class of $(m, n)$.

We can also define multiplication on our 'formal integers', $(\mathbb{N} \times \mathbb{N})/_\sim$ by setting

$$[(m, n)] \cdot [(m', n')] = [(mm' + nn', mn' + m'n)].$$

**Exercise 168.** This exercise is concerned with checking that this definition of multiplication makes sense and fits with the usual one on $\mathbb{Z}$.

(a) Show that for $(k, l)$, $(k', l')$, $(m, n)$ and $(m', n')$ in $\mathbb{N} \times \mathbb{N}$ we have that $(k, l) \sim (k', l')$ and $(m, n) \sim (m', n')$ implies

$$(k, l) \cdot (m, n) \sim (k', l') \cdot (m', n').$$

This means that our definition of multiplication works properly on equivalence classes with respect to $\sim$.

(b) Show that $[(1, 0)]$ is the unit for this multiplication.

(c) Show that for $i, j \in \mathbb{Z}$ we have $(gi) \cdot (gj) = g(ij)$.

(d) Show that for $(k, l), (m, n) \in \mathbb{N} \times \mathbb{N}$ we have

$$f[(k, l)]f[(m, n)] = f([(k, l)] \cdot [(m, n)]).$$

Hence our formal integers can play the role of $\mathbb{Z}$ for all the usually considered operations, namely addition (which gives subtraction thanks to the existence of inverses) and multiplication. Mathematicians say that the two are *isomorphic as rings*.

**Optional Exercise 32.** In the formal definition of $\mathbb{Z}$ we use a number of properties that hold on a very general level. This exercise is about understanding those properties better. Assume you have an equivalence relation $\sim$ on a set $S$.

(a) Define a function $q$ from $S$ to $S/_\sim$ which maps each element to its equivalence relation. This is known as the *quotient map*.

(b) Assume you have a function $f$ from $S$ to a set $T$ with the property that, for $s, s' \in S$ we have $s \sim s'$ implies $fs = fs'$. Define a function $f_\sim$ from $S/_\sim$ to $T$ with the property that $f_\sim \circ q = f$. Can you argue that $f_\sim$ is uniquely determined by these properties?

(c) Assume you have a binary operation $\circledast$ on $S$ such that for $s, s', t, t' \in S$ we have $s \sim t$ and $s' \sim t'$ implies $s \circledast s' \sim t \circledast t'$ (we may think of this as saying that the operation is well-behaved with respect to the equivalence relation)[24]. Define a binary operation $\circledast$ on $S$ with the property that for all $s, s' \in S$ you have

$$qs \circledast qs' = q(s \circledast s').$$

Can you show that this operation is uniquely determined by these properties?

(d) Assume that we have the same situation as in the previous part. Show that if the given operation is commutative or associative, then the newly defined operation has the same property. Further show that if $e$ is a unit for the operation $\circledast$ then $[e]$ is a unit for the operation $\circledast$. Lastly prove that if $s'$ is an inverse for $s$ with respect to $\circledast$ then $[s']$ is an inverse for $[s]$ with respect to $\circledast$.

(e) Check that the addition and multiplication operations for $(\mathbb{N} \times \mathbb{N})/_\sim$ given in the text above are special cases of such binary operations.

[24] If we think of the operation as the more fundamental item we say that the relation is a *congruence relation for the given operation*.

**Definition 58: integers**

The commutative ring $\mathbb{Z}$ of integers is (formally) defined as the quotient of $\mathbb{N}\times\mathbb{N}$ with respect to the equivalence relation given above, and with addition and multiplication operations also given above.

**Takeaway**

We now have the tools to formally define the integers, and their operations, based on the natural numbers.

### The rationals

We can use a similar construction to obtain a formal definition of the rational numbers. As a basis we use the set

$$\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}) = \{(i, m) \mid i,\, m \in \mathbb{Z},\, m \neq 0\}.$$

We use the following equivalence relation on this set:

$$(i, m) \sim (j, n) \qquad \text{if and only if} \qquad in = jm.$$

**Exercise 169.** Show that the relation defined above is an equivalence relation.

We may then form the set $(\mathbb{Z} \times (\mathbb{Z} - \setminus\{0\}))/_\sim$ of $\sim$-equivalence classes. As before we are concerned with defining operations on this new set. This time we begin with multiplication. For $i$, $j$ in $\mathbb{Z}$ and $m$, $n$ in $\mathbb{Z} \setminus \{0\}$ we set[25]

$$[(i, m)] \cdot [(j, n)] = [(ij, mn)].$$

This defines a commutative and associative operation with unit $[(1, 1)]$. We have inverses for this operation: Assume we have $i, m \in \mathbb{Z}$ with $m \neq 0$. We claim that the inverse for $[(i, m)]$ is $[(m, i)]$ and verify this by calculating

$$[(i, m)] \cdot [(m, i)] = [(im, mi)] = [(1, 1)],$$

and

$$[(m, i)] \cdot [(i, m)] = [(mi, im)] = [(1, 1)].$$

We still owe the connection between our set of equivalence classes and the set of rational numbers $\mathbb{Q}$. This is a bit trickier to describe cleanly than the case of the integers above because effectively the rational numbers have been treated as a quotient all along: Given a rational number we know that we may find integers $m$ and $n$, with $n \neq 0$ such that our number is equal to the fraction $m/n$. But $m$ and $n$ are not defined uniquely, and so we effectively use an equivalence relation on fractions, where

$$\frac{m}{n} \approx \frac{m'}{n'} \qquad \text{if and only if} \qquad mn' = m'n.$$

This suggests that the proper translation between our set

$$(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))/_\sim \qquad \text{and} \qquad \mathbb{Q}$$

[25] Again one has to check that this makes sense—see the exercise below.

is as follows. We define a function $f$ from $(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))/_\sim$ to $\mathbb{Q}$ by setting

$$[(m,n)] \longmapsto \frac{m}{n},$$

and a function $g$ in the opposite direction by setting

$$\frac{m}{n} \longmapsto [(m,n)].$$

**Exercise 170.** This exercise is concerned with checking that the claims made above are true.

(a) Show that for $(i,m)$, $(i',m')$, $(j,n)$ and $(j',n')$ in $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ we have that $(i,m) \sim (i',m')$ and $(j,n) \sim (j',n')$ implies $(ij,mn) \sim (i'j',m'n')$. This means that our definition of multiplication works properly on equivalence classes with respect to $\sim$.

(b) Show that $[(1,1)]$ is the unit for this multiplication.

(c) Observe that for $i$, $j$, $m$, $n$ from $\mathbb{Z}$ with $m,n \neq 0$ we have

$$(i,m) \sim (j,n) \qquad \text{if and only if} \qquad \frac{i}{m} \approx \frac{j}{n}$$

and conclude that $\approx$ is an equivalence relation. Further note that this implies that the definition of $g$ makes sense by mapping equivalent fractions to the same equivalence class for $\sim$ in $(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))$.

(d) Show that for $i,j,m,n \in \mathbb{Z}$ with $m,n \neq 0$ we have $g(\frac{i}{m}) \cdot g(\frac{j}{n}) = g(\frac{i}{m}\frac{j}{n})$.

(e) Show that for $(i,m),(j,n) \in \mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ we have

$$f[(i,m)]f[(j,n)] = f([(i,m)] \cdot [(j,n)]).$$

This means that our $(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))/_\sim$ really is a way of talking about the rationals cleanly, that is, we know exactly what this set looks like, where before we did not have a clean way of referring to its elements. But so far we have only considered multiplication and still owe the addition operation.

We set, for $i$, $j$, $m$ and $n$ in $\mathbb{Z}$, with $m,n \neq 0$,

$$[(i,m)] + [(j,n)] = [(in + jm, mn)].$$

This defines a commutative associative operation with unit $[(0,1)]$, and with inverses where the additive inverse of $[(i,m)]$ is given by $[(-i,m)]$.

**Exercise 171.** This exercise is concerned with checking that this definition of addition makes sense and fits with the usual one from $\mathbb{Q}$.

(a) Show that for $(i,m)$, $(i',m')$, $(j,n)$ and $(j',n')$ in $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ we have that $(i,m) \sim (i',m')$ and $(j,n) \sim (j',n')$ implies

$$(i,m) + (j,n) \sim (i',m') + (j',n').$$

This means that our definition of addition works properly on equivalence classes with respect to $\sim$.

(b) Show that $[(0,1)]$ is the unit for this addition.

(c) Show that for $i, j, m, n \in \mathbb{Z}$ with $m, n \neq 0$ we have

$$g(\frac{i}{m}) + g(\frac{j}{n}) = g(\frac{i}{m} + \frac{j}{n}).$$

(d) Show that for $(i, m), (j, n) \in \mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ we have

$$f[(i, m)] + f[(j, n)] = f([(i, m)] + [(j, n)]).$$

Hence we may use $(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))/_\sim$ as a formal definition of the rational numbers (with addition and multiplication)[26], and that is what mathematicians do:. Instead of assuming that $m/n$ has a predefined meaning, they use $(m, n)$ to express the same number, being aware of the fact that an equivalence relation is required since there is more than one way of representing a given number. A mathematician would say that what we have defined above is a *field*.

**Optional Exercise 33.** Carry out the last part of Optional OExercise 32 for multiplication and addition on $(\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}))/_\sim$ as defined in this section.

**Definition 59: rationals**

The field $\mathbb{Q}$ of rational numbers is (formally) defined as the quotient of $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\}$ with respect to the equivalence relation given above, and with addition and multiplication operations also given above.

**Takeaway**

We now have the tools to formally define the rational numbers, and their operations, based on the natural numbers via the integers.

### 7.4.8 Further examples

**Polynomial functions** (see also Section 0.3.5) often come up as describing the complexity of various algorithms or programs. These functions exist over all our sets of numbers. Assume we have fixed a set of numbers $N$.

The formal definition of a polynomial is that of a function[27] from $N$ to $N$ for which we can find $n \in \mathbb{N}$ and $a_0, a_1, \ldots, a_n$ in $N$ such that the function is given by

$$\sum_{i=0}^{n} a_i x^i = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0.$$

For our purposes we *exclude* the polynomial all of whose coefficients are 0. It is possible to extend the ideas that follow below to this polynomial, but it creates quite a lot of exceptions which make this account more complicated, and which distract from the ideas presented below.

The $a_i$ are the **coefficients**. We define the **degree** of the polynomial to be the largest $n$ for which $a_n \neq 0$, and write

$$\deg\left(\sum_{i=0}^{n} a_i x^i\right)$$

for this number. The corresponding coefficient $a_n$ is called the *leading coefficient of the polynomial* in question.

[26] Since subtraction and division are defined via inverses for addition and multiplication, respectively, these are also included.

[27] A more formal mathematical definition of polynomials works without considering them as functions, but merely as formal expressions involving coefficients and powers of some variable, say $x$. The definitions of the addition and multiplication operations for polynomials given below still work in that case.

**Example 7.53.** We calculate some degrees.

$$\begin{aligned}\deg\left(3x^4+2x^2-5\right)&=4\\ \deg\left(0x^5+3x^3-2x^1\right)&=3\\ \deg\left(0x^5+0x^2-4\right)&=0.\end{aligned}$$

The leading coefficients of these polynomials are 3, 3, and $-4$ respectively.

When a coefficient is 0 it is customary not to write the corresponding part of the term, that is, we omit expressions of the form $0x^i$.

We can add such polynomial functions, and we can multiply them, in an obvious way. We first look at an example for each, and then give a general definition.

**Example 7.54.** We look at how to add polynomial to another. In the example below we have a polynomial of degree 3 which we want to add to one of degree 2, for example,

$$\begin{array}{cccccc} & x^3+ & & 2x+ & 1 \\ + & & 3x^2+ & 5x+ & 10 \\ = & x^3+ & 3x^2+ & 7x+ & 11. \end{array}$$

In general we would like to write

$$\sum_{i=0}^{m} a_i x^i + \sum_{i=0}^{n} b_i x^i = \sum_{i=0}^{\max\{m,n\}} (a_i+b_i)x^i,$$

but it may be the case that some of the coefficients are not defined. If we go back to the concrete example above we have

$$\begin{array}{lllll} a_3=1 & a_2=0 & a_1=2 & a_0=1 & \text{and} \\ & b_2=3 & b_1=5 & b_0=10. & \end{array}$$

Here the coefficient $b_3$ is not defined, so the convention is to assume that such coefficients are equal to 0 to make the formal definition work.

Multiplication is slightly more complicated.

**Example 7.55.** Again we start with an example.

$$\begin{aligned}&(x^3+2x+1)(3x^2+5x+10)\\ &=(1\cdot 3)x^5+(1\cdot 5)x^4+(1\cdot 10+2\cdot 3)x^3+(2\cdot 5+1\cdot 3)x^2\\ &\qquad+(2\cdot 10+1\cdot 5)x+1\cdot 10\\ &=3x^5+5x^4+16x^3+13x^2+25x+10.\end{aligned}$$

In general, the definition is

$$\left(\sum_{i=0}^{m} a_i x^i\right)\left(\sum_{i=0}^{n} b_i x^i\right)=\sum_{i=0}^{m+n}\left(\sum_{j=0}^{i} a_{i-j}b_j\right)x^i,$$

which multiplied out looks something like

$$a_0b_0+(a_0b_1+a_1b_0)x+(a_2b_0+a_1b_1+a_0b_2)x^2$$

$$+ \cdots + (a_{m-1}b_n + a_m b_{n-1})x^{m+n-1} + a_m b_n x^{m+n}.$$

These operations on polynomials are associative and commutative. The unit for addition is the constant 0 polynomial, all of whose coefficients are 0, while the unit for multiplication is the constant 1 polynomial, where $a_0 = 1$ and all other coefficients are 0.

When we consider the complexity of an algorithm we often are only interested in the degree of the polynomial, that is, the highest power of $x$ that occurs in the term.

We define an equivalence relation on polynomial functions, whereby

$$\sum_{i=0}^{m} a_i x^i \sim \sum_{i=0}^{n} b_i x^i \qquad \text{if and only if} \qquad \deg\left(\sum_{i=0}^{m} a_i x^i\right) = \deg\left(\sum_{i=0}^{n} b_i x^i\right).$$

In other words we consider two polynomial functions equivalent if and only if they have the same degree, that is if the largest $n$ for which $x^n$ has a non-zero coefficient, is the same for both. This means that two polynomials are in the same equivalence class if and only if they define the same 'big O' class of functions, compare Example 7.35. You are asked to think about some aspects of this idea in EExercise 174.

Now when we want to determine the complexity of an algorithm or program, and we know this is a polynomial function, we can concentrate on determining the degree. For this we need to be able to calculate with this equivalence relation, and the following exercise invites you to work out how this works. You will study these ideas in more generality in COMP11212 and COMP26120.

**Exercise 172.** For the following polynomials, give the degrees and leading coefficients, and calculate their sum and product.

- $2x^5 + x^3 - 1$ and
- $x + 1$.

What are the degrees and coefficients of the two resulting polynomials? How do they arise from the degrees and leading coefficients of the two polynomials you started with?

**EExercise 173.** Assume you are trying to implement polynomials with coefficients from $\mathbb{N}$ in Java by using the List class from Section 6.1, see Code Example 6.1. You want to think of the list as giving you the coefficient of the polynomial, so that

$$[3, 5, 2]$$

encodes the polynomial

$$3 + 5x + 2x^2.$$

The empty list (or **null** reference) does not correspond to a polynomial via this encoding. We are going to use it as another name for the polynomial of degree 0 whose (only) coefficient is 0, that is, it encodes the same polynomial as the list $[0]$.

Give definitions for the following operations, either by recursive definition for elements of $\mathsf{Lists}_{\mathbb{Z}}$ or by giving a recursive Java method for the class List. You have to stick to one of these for all the parts.

(a) Addition of polynomials.

(b) Multiplication of polynomials.

This is quite complicated. You may either develop your own way of doing this, or you may work according to the following idea.

When we multiply a given polynomial, say

$$2 + x + 3x^2$$

with another polynomial, say $p$, we have

$$\begin{aligned} (2 + x + 3x^2) \cdot p &= 2 \cdot p && (7.1) \\ &+ x \cdot p && (7.2) \\ &+ 3x^2 \cdot p. \end{aligned}$$

Hence in order to define multiplication of polynomials it is sufficient to use addition provided we also define a method that takes a polynomial and multiplies it by a polynomial of the form

$$mx^n,$$

where $m$ and $n$ are elements of $\mathbb{N}$.

- For multiplying a given polynomial by a polynomial of the form $x^n$ you may find it convenient to have an operation that takes as input a natural number $n$ and returns a list which consists of $n$ many 0s.
- It is useful to have an operation that takes a polynomial and multiplies is by a natural number (by multiplying all the coefficients by that natural number).
- Using the previous two parts one may define an operation that multiplies a given polynomial by a polynomial of the form

  $$mx^n,$$

  where $m, n \in \mathbb{N}$.
- Using the previous part and the addition operation for polynomials you should now be able to define multiplication of polynomials.

You may use previously defined operations for lists at any point.

**EExercise 174.** Let $P_{\mathbb{N}}$ be the set of polynomials with coefficients in $\mathbb{N}$.

(a) Above there is the definition of an equivalence relation $\sim$ on the set of polynomials which relates two polynomials if they have the same degree. Find a way of representing these so that you can describe the set $P_{\mathbb{N}}/_{\sim}$. Which polynomials are in $[x^2]$?

(b) Define an addition operation $\oplus$ on $P_{\mathbb{N}}/_{\sim}$. Ensure that for polynomials $p$ and $q$ you have $[p] \oplus [q] = [p + q]$—in other words, the equivalence class we obtain when adding the class of $p$ to the class of $q$ is the class of $p + q$.

(c) Define a multiplication $\circledast$ operation on $P_{\mathbb{N}}/_{\sim}$. Ensure that for polynomial functions $p$ and $q$ you have $[p] \circledast [q] = [pq]$.

*Hint: In CExercise 163 you are asked to define addition and multiplication operations on equivalence classes when calculating modulo some number. You can think of this exercise*

*as asking you to define addition and multiplication operations on equivalence classes of polynomials given by their degrees.*

*If you are struggling with this then start by carrying out Exercise 172. To get the marks for preparing the exercise that's a requirement if you cannot do anything else.*

**Finite state automata** have an interesting equivalence relation on them, given by bisimulations.

**Exercise 175.** Argue that the following relation is an equivalence relation: The relation on finite state automata, where

$$A \sim A'$$

if and only if there is a bisimulation between them. There's no need to give a formal proof, just explain in English why you think the given relation has the required properties.

**When programming** we sometimes want to have a notion that two objects are equivalent for some equivalence relation. For example in Java the standard class `java.lang.Object` contains an instance method `equals()` which is designed to model the notion that two objects may be equivalent. See *Java: Just in Time* for more details in Section 20.4. This can only work properly if the relation in question is an equivalence relation.

## 7.5 Partial orders

Many structures we consider carry an *order* on them which allows us to *compare their elements*. There is a standard way of axiomatizing those properties that make a relation into an order in a way that agrees with our intuitions.

### 7.5.1 Posets

When comparing two elements $s$ and $s'$ we write

$$s \leq s'$$

to state that $s$ is less than, or equal to, $s'$, and

$$s' \geq s$$

to state that $s'$ is greater than or equal to $s$. We consider the two equivalent. We further write

$$s < s'$$

if and only if

$$s \leq s' \qquad \text{and} \qquad s \neq s'.$$

We are using $\leq$ to give the infix notation for a *binary relation on* a set $S$. In this section we study the properties a binary relation needs to have in order to be something that behaves as we expect a comparison to behave.

First of all we expect each element $s$ of $S$ to be less than or equal to itself, so we expect

$$s \leq s$$

to hold for all $s \in S$. This means that we expect $\leq$ to be a reflexive relation.

Secondly we expect to be able to combine comparisons, that is if

$$s \leq s' \qquad \text{and} \qquad s' \leq s''$$

for elements $s$, $s'$, $s''$ from $S$ then we expect to be able to conclude that

$$s \leq s''.$$

In other words, we expect $\leq$ to be transitive.

These two properties together are sufficient to ensure that $\leq$ has most of the properties we expect from a comparison (see Optional OExercise 34), but it is customary to demand more.

When we know that

$$s \leq s' \qquad \text{and} \qquad s' \leq s$$

hold for elements $s$ and $s'$ of $S$ we want to be able to conclude that

$$s = s'.$$

**Definition 60: antisymmetric**

A binary relation $R$ on a set $S$ is **anti-symmetric** if for all elements $s$ and $s'$ from $S$ we have that

$$(s, s') \in R \text{ and } (s', s) \in R \qquad \text{implies} \qquad s = s'.$$

**Takeaway**

It's not possible for two elements to be related both ways unless they're equal.

We cannot describe this property without having a binary equality predicate, say $E$. If we use that then the first order formula expressing anti-symmetry is

$$\forall x. \forall y. ((R(x, y) \wedge R(y, x)) \to E(x, y)).$$

We may describe this property by noting that it says that

$$R \cap R^{\mathsf{op}} \subseteq I_S.$$

Anti-symmetry means that two elements cannot be less than or equal to each other unless they are equal. This property is the opposite of demanding symmetry of a relation, see Exercise 176, which explains the name.

In other words the only way we could have a two-sided arrow in the picture of an anti-symmetric relation is for that arrow to be a loop.[28]

**Exercise 176.** Show that a symmetric reflexive relation on a set $S$ is anti-symmetric if and only if it is the identity on $S$.

Note that unlike for the other properties of binary relations studied in these notes, there is no such thing as an 'anti-symmetric closure' of a relation. For the other properties it is possible to *add* elements to the relation to obtain the desired property. But to make a relation anti-symmetric we would have to remove one of $(s, s')$ or $(s', s)$ if both are elements of the relation, and there's no sensible way to decide which one to take away. One could remove both, but then one constructs something that is a long way removed from the starting relation.

[28] But we don't draw loops with arrows at both ends!

**Exercise 177.** Show that the transitive closure of the reflexive closure of an anti-symmetric relation is not necessarily anti-symmetric. Hence we cannot generate partial orders from relations in the way we can do with equivalence relations. *Hint: The smallest counterexample is a binary relation on a set with three elements.*

Now we have all the properties required to define a partial order.

**Definition 61: partial order**

A **partial order** on a set $S$ is a binary relation which is reflexive, anti-symmetric and transitive. A set together with a partial order is known as a **partially ordered set**, or in short, a **poset**.

**Takeaway**

When we want to compare the elements of a set, we need these properties.

We often write $(S, \leq)$ for the poset consisting of the set $S$ with the partial order $\leq$.

**Example 7.56.** The standard definition for $\leq$ on sets of numbers between $\mathbb{N}$ and $\mathbb{R}$ are all partial orders.[29]

All these orders on sets of numbers satisfy an extra condition, see Definition 62, and you may find that you are assuming properties because you are guided by the examples you already know. Nonetheless we use the symbol $\leq$ for all partial orders in this section, so be careful not to make assumptions about such relations you make.

In order to check that a given relation on a small finite set is a partial order we may draw it as a directed graph, and then check reflexivity and transitivity as before. To check anti-symmetry we just have to make sure that there's no connection with arrow tips on both ends. To put it differently, we make use of the transitivity of the relation when interpreting the diagram.

It is customary, however, to draw a partial order on a small finite set as an undirected graph that is *oriented* on the page. Since a partial order is always reflexive one usually does not draw this part of the relation. Since a partial order is typically not symmetric (indeed, it has a property which is in a way the opposite, see Exercise 176), you might think that one should draw a directed graph, but instead the convention is to use the orientation of the paper to provide direction: Whenever two elements are connected by a line then the one that sits lower on the page is considered to be less than or equal to the other.[30] Furthermore, in order to avoid drawing superfluous lines, the assumption is that if $a$ is connected to $b$ which is higher up on the page, and $b$ is connected to $c$, which is higher up again on the page, then the relation connects $a$ and $c$ even if no connection is drawn between the two.

The resulting graph is known as the *Hasse diagram* of the given partial order.

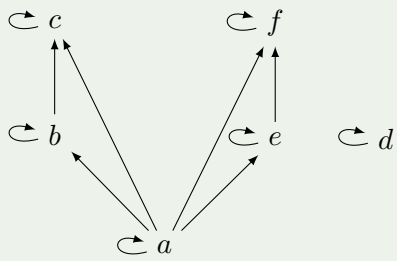
[29] Moreover, the operations on these sets of numbers, namely multiplication and addition, have specific properties which mean they are compatible with that order.

[30] It should go without saying that this means that we have to draw our Hasse diagrams in such a way that two elements on the same level are never connected.

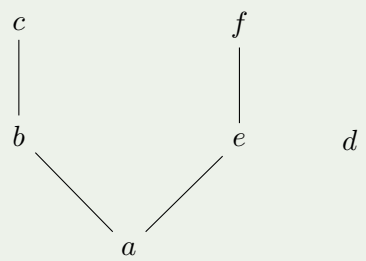
**Example 7.57.** Consider the partial order on $\{a, b, c, d, e, f\}$ which is the transitive reflexive closure of

$$\{(a, b), (b, c), (a, e), (e, f)\}$$

Previously we would have drawn this relation as follows.




But for relations known to be partial orders we draw the corresponding *Hasse diagram* as explained in the preceding paragraph, which for the given poset looks like this:

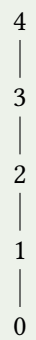



The picture tells us (because of the orientation on the page) that $a$ is less than or equal to $b$ which in turn is less than or equal to $c$, and that $a$ is also less than or equal to $e$, which in turn is less than or equal to $f$. The element $d$ is not comparable with any element other than itself.

**Example 7.58.** If we go back to the example from page 379, and form the transitive closure of the reflexive closure of

$$\{(0, 1), (1, 2), (2, 3), (3, 4)\}$$

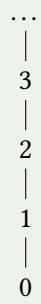
then the Hasse diagram of this partial order is simply




Note that this is a total order (see Definition 62). In the Hasse diagram of a totally ordered set all the elements are lined up.

**Example 7.59.** One particular partial order should be familiar.

On the natural numbers $\mathbb{N}$ we have the 'usual order' we consider most of the time. We picture this as on the right, but because we are considering an infinite set we can only draw the Hasse diagram of a small part of the whole. The expression $m \leq n$ is one that you will have seen before, with this meaning. Note that this is a total order.




**Example 7.60.** Consider the following partial order on the natural numbers without 0, $\mathbb{N}\backslash\{0\}$. We set

$$m \leq n \qquad \text{if and only if} \qquad m \text{ divides } n.$$

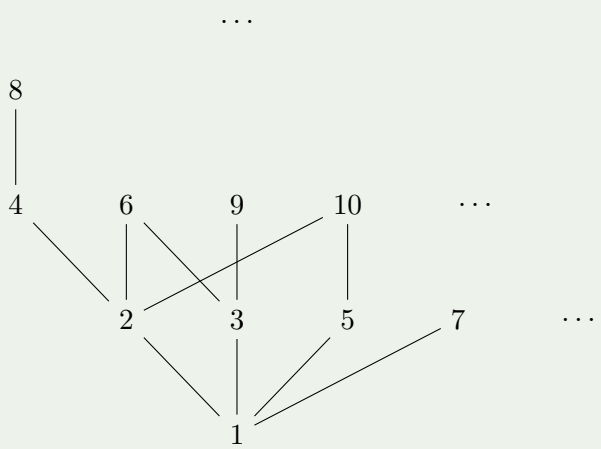
We show that this is a partial order by checking the properties required.[31]

- Clearly the relation defined above is reflexive since every natural number other than 0 divides itself.
- The relation is anti-symmetric since, given two natural non-zero numbers $m$ and $n$,

  $$m \text{ divides } n \qquad \text{and} \qquad n \text{ divides } m$$

  imply $m = n$.
- The relation is transitive—you can find a proof in Exercise 23 (c).

Drawing a picture of (part of) this partial order, for the numbers from 1 to 10, gives a bit of an idea of what it looks like.




[31]Note that these properties are established above in examples or exercises but we repeat some of the arguments here for completeness' sake.

Note that in the preceding Examples 7.59 and 7.60, we use the same symbol, $\leq$, for two *distinct* partial orders. Which partial order is intended in any given situation has to be clear from the context. If you find it confusing to use $\leq$ for an order other than the 'usual' one, you may instead use symbols such as $\sqsubseteq$ or $\preccurlyeq$ to make a visible distinction.

**Example 7.61.** Classes in Java are given a partial order by *inheritance*. You can define that one class is less than or equal to another if and only if it is a subclass, provided that you are happy to consider a class to be a subclass of itself, which makes sense if we want to consider all those classes which make the instance variables and methods of the given class available.

**Example 7.62.** Many rankings come close to being partial orders. But typically it is possible for two entities to be considered equal in ranking, and so they often are not anti-symmetric. For example, ranking tennis, golf or chess players by the points they have accumulated has this property, as does ranking students by their overall averages. In the case where such a relation is anti-symmetric it is a total order, compare Definition 62. See OExercise 34 for a look at relations that only lack anti-symmetry to be partial orders.

**Example 7.63.** We can define something that is almost a partial order on finite state machines by setting

$$A \leq A' \qquad \text{iff} \qquad \text{there is a simulation from } A \text{ to } A'.$$

This is reflexive since the identity relation is always a simulation. It is transitive since the relational composite of two simulations is a simulation (see OExercise 29. It is not anti-symmetric since it is possible for two automata to have simulations going each way without the automata being identical. OExercise 34 encourages you to think about relations that have these properties.

**Example 7.64.** Scheduling is typically performed based on information that Task $A$ has to be performed before Task $B$. This information has to be

- anti-symmetric, since otherwise it is impossible to schedule all tasks;
- transitive since if Task $A$ has to come before Task $B$, which in turn has to happen before Task $C$ can be performed, then Task $A$ has to happen before Task $C$.

Hence the reflexive closure of such a relation is a partial order. If we have to schedule tasks in a linear fashion, maybe because there is only one person (or robot or machine) to carry out all the tasks, then a valid schedule is one where

$$\text{Task } A$$

can only be scheduled when

$$\text{all tasks } B \qquad \text{with } B < A \qquad \text{have been scheduled.}$$

**Exercise 178.** A tutorial group of first year students has identified the following tasks for the first draft of their project plan, given here in a jumbled sequence:

- $A$: Test back end.
- $B$: Integrate front and back end.
- $C$: Implement front end according to requirements.
- $D$: Implement back end according to requirements.
- $E$: Test front end.
- $F$: Analyse requirements for front end.
- $G$: Carry out further testing on combined program.
- $H$: Analyse requirements for back end.

Carry out the following tasks:

(a) Identify which tasks have to be carried out before which other tasks and draw a Hasse diagram for the reflexive closure of that relation.

(b) Assuming that the whole group is involved in each task, give three possible ways of scheduling all the tasks so that they are carried out one after the other, and in such a way that every task is scheduled after all the tasks it relies on have been completed.

(c) By giving such a schedule you have given a total order for the set of tasks. Which property does such a total order have to have in order to satisfy the criterion given in the previous part?

We identify a special property shared by the partial orders on sets of numbers, see Example 7.56. Note that the following notion is defined in Section 20.3 of *Java: Just in Time*.

**Definition 62: total order**

A **total order** on a set $S$ is a partial order $\leq$ with the property that for all $s$ and $s'$ in $S$ we have

$$s \leq s' \qquad \text{or} \qquad s' \leq s.$$

In this situation we say that the poset $(S, \leq)$ is **totally ordered**.

**Takeaway**

Any two elements are comparable.

Again we give the first order proposition for this condition, where we assume a binary predicate symbol $R$ for the partial order.

$$\forall x. \forall y. \, (R(x, y) \vee R(y, x))$$

In a total order every two elements are comparable, so we may think of them all as being lined up, the way we usually think of $\mathbb{N}$ or $\mathbb{R}$. When we do not have this extra property then you may find that (partial) orders don't quite behave in the way you expect.

**Example 7.65.** Examples of totally ordered sets are $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ and $\mathbb{R}$ with the usual order.

**Example 7.66.** The partially ordered set from Example 7.58 shows a total order.

**CExercise 179.** Which of the following relations are partial orders? Try to sketch a Hasse diagram for each[32], and determine whether it is total. Justify your answers.

(a) The relation on complex numbers where $a + bi$ is less than or equal to $a' + bi'$ if and only if $a \leq a'$ and $b \leq b'$.

(b) The relation on complex numbers where $a + bi$ is less than or equal to $a' + b'i$ if and only if $a \leq a'$.

(c) The relation on $\mathbb{N} \times \mathbb{N}$ where $(m, n)$ is less than or equal to $(m', n')$ if and only if $m + n \leq m' + n'$.

(d) The relation on $\mathbb{N} \times \mathbb{N}$ where $(m, n)$ is less than or equal to $(m', n')$ if and only if

$$m < m' \qquad \text{or} \qquad m = m' \text{ and } n \leq n'.$$

(e) The relation on functions from $\mathbb{N}$ to $\mathbb{N}$ where $f$ is less than or equal to $g$ if and only if (compare Section 5.1)

$$g \text{ dominates } f,$$

that is

$$\text{for all } n \in \mathbb{N} \text{ we have } fn \leq gn.$$

This is known as the *pointwise order* on $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ since we look at each input point.

(f) The subset relation on the powerset of a set $X$.

(g) The relation on strings over a set $S$ where string $l$ is less then or equal to string $l'$ if and only if $l$ is a *prefix* of $l'$—in other words, if the beginning of $l'$ is $l$.

(h) Assume we have a poset $(S, \leq)$. Then we obtain an order on the set of strings over the set $S$, known as the *lexicographic order* by setting[33]

$$\begin{aligned} s_1 s_2 \cdots s_m \preccurlyeq s'_1 s'_s \ldots s'_n \qquad & \text{if and only if} \\ & \text{for } i \text{ smallest number in } \mathbb{N} \text{ with } s_i \neq s'_i \\ \text{we have} \qquad & s_i < s'_i \text{ or } s_i \text{ not defined.} \end{aligned}$$

Show that the relation defined in this way is indeed reflexive, anti-symmetric and transitive.

(i) The relation on the set of first year students in the Department where student $A$ is related to student $B$ if and only if student $B$ is taller than student $A$.

(j) The relation on the set of first year students in the Department where student $A$ is related to student $B$ if and only if the registration number of student $A$ is less than or equal to that of student $B$.

(k) The relation on all valid Java programs where programme $P$ is less than or equal to program $Q$ if and only if as a string, in the lexicographic order (also known as the 'dictionary order') program $P$ comes before program $Q$.

(l) The following relation on the set $\{a, b, c, d, e\}$:

$$\{(a, a), (b, b), (c, c), (d, d), (e, e)\}.$$

(m) The reflexive closure of the following relation on $\{a, b, c, d, e\}$:

$$\{(a, b), (c, d)\}.$$

(n) The reflexive transitive closure of the following relation on the same set.

$$\{(a, b), (b, c), (c, d), (d, e)\}.$$

(o) The reflexive transitive closure of the following relation on the same set.

$$\{(a, b), (b, a), (b, c)\}.$$

(p) The subset relation on the powerset of the set $\{0, 1, 2\}$.

*In parts (a) to (e) you have an existing relation $\leq$ on a set of numbers ($\mathbb{N}$ or $\mathbb{R}$), and a new relation is defined based on that. You may find it less confusing to use a new symbol for the new relation, maybe $\sqsubseteq$ or $\preccurlyeq$ as I do in part (h).*

**Example 7.67.** We show how to define a partial order on $\mathsf{FBTrees}_S$, where $S$ is an arbitrary set. We would like to define[34]

**Base cases $\leq$.** For all $s \in S$, we have $\texttt{tree}\, s \leq \texttt{tree}\, s$.

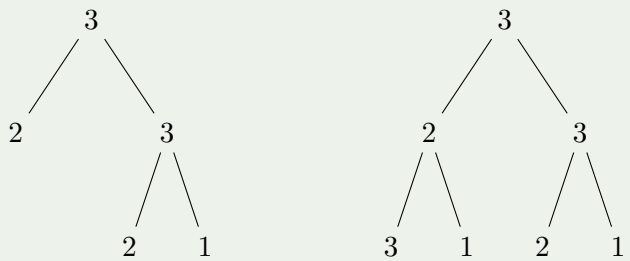
For all $s \in S$, $t$, $t' \in \mathsf{FBTrees}_S$, $\quad \texttt{tree}\, s \leq \texttt{tree}_s(t, t')$.

**Step case $\leq$.** If $t \leq t''$ and $t' \leq t'''$ then $\texttt{tree}_s(t, t') \leq \texttt{tree}_s(t'', t''')$.

This relates two trees if and only if the second arises from the first by repeatedly extending the tree, that is

- pick a leaf with label $s$ (that is a subtree of the form $\texttt{tree}\, s$) and
- replace that leaf by a tree whose root label is $s$, that is, a tree of the form $\texttt{tree}_s(t, t')$.

So, for example, we have that the tree on the left is less than or equal to the one on the right.




[32]For infinite or large sets just try to get a general idea what such a diagram might look like and draw a part of it.

[33]Any dictionary gives you the words in this order. Also, the Linux list function presents the contents of some directory in this order.

Alternatively we can think of the relation $\leq$ as relating a tree $t$ to a tree $t'$ if and only if we can obtain the first tree from the second tree by cutting off some of the branches.

We can show that this is a partial order. For reflexivity we have an inductive proof.

**Base cases $\leq$.** For $s \in S$ we know that $\texttt{tree}\, s \leq \texttt{tree}\, s$ by the first base case of $\leq$.

**Ind hyp.** We know that $t \leq t$ and $t' \leq t'$ for some full binary trees $t$ and $t'$ over $S$.

**Step case $\leq$.** From the induction hypothesis we may use the step case of $\leq$ to deduce that for every $s \in S$ we have

$$\texttt{tree}_s(t, t') \leq \texttt{tree}_s(t, t').$$

We can show anti-symmetry by a second induction proof.

**Base cases $\leq$.** For $t, t' \in \mathsf{FBTrees}_S$ if $t \leq t'$ by one of the base cases of the definition then it must be the case that we can find $s \in S$ such that $t = \texttt{tree}\, s$.

In order for $t' \leq t$ to also hold it must be the case, since we know that $t = \texttt{tree}\, s$, that this also occurs by the first base case of the definition of $\leq$, and so we must have that there is $s' \in S$ with $t' = \texttt{tree}\, s'$, but But if they are related by the base case of the definition of $\leq$ then it must also be the case that $s = s'$, and so

$$t = \texttt{tree}\, s = \texttt{tree}\, s' = t'.$$

**Ind hyp.** If $t \leq t'$ and $t' \leq t$ then $t = t'$, and if $t'' \leq t'''$ and $t''' \leq t''$ then $t'' = t'''$.

**Step case $\leq$.** The only remaining case is that

$$\texttt{tree}_s(t, t') \leq \texttt{tree}_s(t'', t''') \qquad \text{and} \qquad \texttt{tree}_s(t'', t''') \leq \texttt{tree}_s(t, t')$$

both by the step case, and so

$$t \leq t'' \text{ and } t' \leq t''' \qquad \text{and} \qquad t'' \leq t \text{ and } t''' \leq t'$$

and by the induction hypothesis

$$t = t'' \qquad \text{and} \qquad t' = t'''.$$

Note that this is a more complicated induction proof than others contained in these notes. This is required to ensure that all possible cases are covered.

The proof of transitivity is a similar induction proof, see the following exercise.

**Exercise 180.** This exercise studies in more detail the partial order on trees described in the preceding example.

(a) Rewrite the recursive definition of this partial order to define a set

$$R_{\leq} \subseteq \mathsf{FBTrees}_S \times \mathsf{FBTrees}_S.$$

[34] Note that there are many base cases here, one for each tree!

(b) Show by induction that for

$$(t, t') \in R_{\leq}$$

we have that the roots of the two trees have the same label.

(c) Show that we have for $t, t' \in \mathsf{FBTrees}_S$ and the partial order $\leq$ from Example 7.67 that

$$t \leq t' \qquad \text{if and only if} \qquad (t, t') \in R_{\leq}.$$

(d) Show that $\leq$ as defined in Example 7.67 is transitive.

(e) Write a program that takes two objects of class BTree and returns true if and only if the first is less than or equal to the second in this order.

**Example 7.68.** We can define a partial order on the set $\mathsf{Lists}_S$ of lists over a set $S$, where one list is less than or equal to another if and only if the second list is an extension of the first list, that is, it arises from the first list by adding more elements (from the left as usual). Note that while this has many similarities to the previous example there are subtle differences.

So, for example, if $S = \mathbb{N}$ we have

$$[4, 3, 2, 1] \leq [7, 9, 3, 4, 4, 3, 2, 1],$$

because we have that

$$[7, 9, 3, 4, 4, 3, 2, 1]$$

may be obtained from

$$[4, 3, 2, 1]$$

by adding the elements

$$4,\ 3,\ 9,\ \text{and } 7$$

in that order. On the other hand, the list

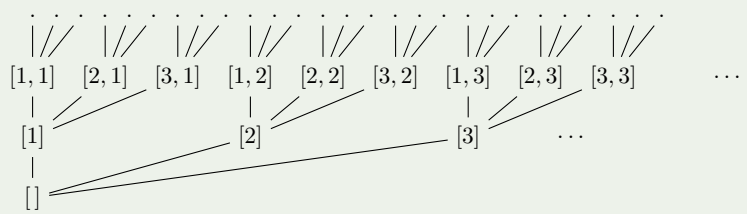
$$[4, 3, 2]$$

is not less than or equal to

$$[4, 3, 2, 1]$$

because the two lists don't start out in the same way.

We can begin to sketch a Hasse diagram for this poset.




We can give a recursive definition for the set of all pairs that belong to this relation $R_{\leq}$ on $\mathsf{Lists}_S$. In other words, we recursively define

$$R_{\leq} \subseteq \mathsf{Lists}_S \times \mathsf{Lists}_S$$

similar to the way we defined the subset of $\mathbb{N}$ that consists of all even numbers (Example 6.45), or of all powers of 2 (Example 6.46).

**Base cases** $R_{\leq}$. $l \in \mathsf{Lists}_S$ implies $(l, l) \in R_{\leq}$.

**Step case** $R_{\leq}$. $(l, l') \in R_{\leq}$, $s \in S$ implies $(l, s : l') \in R_{\leq}$.

If we are allowed to apply the reflexive transitive closure operation afterwards we can get away with a simpler definition of a relation, say $\hat{R}$:

- For $l \in \mathsf{Lists}_S$ and $s \in S$ we have $(l, s : l) \in \hat{R}$.

The relation $R$ is the reflexive transitive closure of the relation $\hat{R}$.

Alternatively we can write a method which takes as input two objects of class List and returns true if and only if the first list is less than or equal to the second list. (You may want to compare this to Exercise 140.) Here we use the reverse method from Code Code Example 6.4.

```
public static boolean lessthan (List l1, List l2)
{
    if (l1 == null)
        return true;
    else {
        if (l2 == null)
            return false;
        else {
            List r1 = reverse (l1);
            List r2 = reverse (l2);
            if (r1.value == r2.value)
                return lessthan (reverse(l1.next), reverse(l2.next));
            else
                return false;
        }
    }
}
```

If you wonder why we reverse the two lists before comparing them, have a go at trying to write a method which does not require this step.

In some ways it is easier to recursively define the relation than to define a method to check whether it holds. In other words, defining an algorithm that checks whether two lists are in the relation is harder than defining the relation. Indeed, finding *decision procedures* (which you encounter in COMP11212) can be difficult even for properties that can be described in a simple manner. [35]

[35] However, one can take that recursive definition of the relation and turn it into the decision procedure given following a specific set of rules! This idea takes us beyond the scope of this course unit.

**Exercise 181.** Show the following for the relation $R_{\leq}$ defined in the preceding example:

(a) The relation $R_{\leq}$ is reflexive.

(b) If $(l, l') \in R_{\leq}$ then $\operatorname{len} l \leq \operatorname{len} l'$.

(c) If $(l, l') \in R_{\leq}$ and $\operatorname{len} l = \operatorname{len} l'$ then $l = l'$.

(d) The given relation is anti-symmetric.

(e) Show that if we have $l$ and $l'$ in $\mathsf{Lists}_S$, and $n \in \mathbb{N}$, and $s_1, s_2, \ldots s_n$ in $S$, with

$$l' = s_n : (s_{n-1} : (\ldots (s_1 : l)))$$

then

$$(l, l') \in R_{\leq}.$$

(f) Show the converse of the previous part, that is, given $(l, l') \in R_{\leq}$ then we can find elements in $S$ to satisfy the given condition.

(g) Show that the given relation is transitive.

*For any given part you may need to use a previous part to give the desired proof.*

Note that when programming, we sometimes come across ways of comparing elements which are *not* partial orders. For example, the CompareTo method for many classes in Java typically implements a comparison that is *not* anti-symmetric. It is an instance of a formal notion explored in the following (optional) exercise.

**Optional Exercise 34.** A *pre-order* $\preceq$ is a binary relation on a set $S$ which is reflexive and transitive. For such a pre-order it makes sense to define a relation $\approx$ by, for $s$ and $s'$ in $S$,

$$s \approx s' \qquad \text{if and only if} \qquad s \preceq s' \text{ and } s' \preceq s.$$

(a) Can you think of an example of a pre-order? Having an example at hand my help you with the following parts.

(b) Show that $\approx$ is an equivalence relation.

(c) Use $\preceq$ to define a partial order on $S/_{\approx}$ and show that your relation is indeed a partial order. Conclude that for every pre-order there is a corresponding partial order on a related set.

(d) The CompareTo() method in Java for many classes is concerned with a pre-order, rather than a partial one. If you want to think more about pre-orders you could verify this claim, and think about why the method was designed in this way.

(e) Another pre-order is given by the relation between sets where $S$ is related to $T$ if and only if the size of $S$ is at most that of $T$. Why is this not a partial order?

**Exercise 182.** Show that the set of all functions

$$\mathsf{Fun}(X, N),$$

where

- $X$ is any set, and
- $N$ is a set of numbers between $\mathbb{N}$ and $\mathbb{R}$

carries a partial order given by the 'is dominated by' relation from Section 5.1, which is known as the *pointwise order* (compare CExercise 179 (e)).

**Exercise 183.** Show that if $(P, \leq)$ is a poset then we obtain another partial order by the following means.

(a) By setting

$$p \sqsubseteq p' \qquad \text{if and only if} \qquad p' \leq p.$$

This is known as the *opposite (partial) order* of the poset, since $\sqsubseteq$ is $\leq^{\mathsf{op}}$. It means turning the poset upside-down.

(b) By taking a subset $Q$ of $P$ and defining, for $q$ and $q'$ in $Q$,

$$q \sqsubseteq q' \qquad \text{if and only if} \qquad q \leq q' \text{ as elements of } P.$$

This is known as the *subset (partial) order* on $Q$.

**Optional Exercise 35.** The total orders on $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ and $\mathbb{R}$ you are used to are very well behaved. For example, for numbers $m$, $n$, $m'$ and $n'$ we have

$$m \leq m' \text{ and } n \leq n' \qquad \text{implies} \qquad m + n \leq m' + n'.$$

(a) Prove this property for $\mathbb{N}$ using induction, and the recursive definitions of $\leq$ and $+$.

(b) Prove this property for $\mathbb{Z}$ and $\mathbb{Q}$ using their formal definitions from the previous section.

(c) Can you think of a similar property that holds for multiplication? *Hint: Have a look at Fact 7.*

(d) There is no standard partial order on the complex numbers $\mathbb{C}$. While it is possible to define a number of partial orders on this set (compare CExercise 179) most of these are not total,[36] nor will the operations be well-behaved with respect to these partial orders. Conduct some experiments by defining orders on $\mathbb{C}$ and exploring their properties.

### 7.5.2 Maximal, minimal, greatest, least

Having a partial order on a set allows us to find elements with distinctive properties.

[36] What would a total order on the set $\mathbb{C}$ look like?

**Definition 63: maximal/minimal**

An element $p$ of a poset $(P, \leq)$ is a **maximal element of** $P$ if and only if for all $p' \in P$,

$$\text{if } p \leq p' \qquad \text{then} \qquad p = p'.$$

An element $p$ of a poset $(P, \leq)$ is a **minimal element of** $P$ if and only if for all $p' \in P$ we have

$$\text{if } p' \leq p \qquad \text{then} \qquad p' = p.$$

**Takeaway**

There's no element strictly bigger than a maximal element, and no element is strictly smaller than a minimal element.

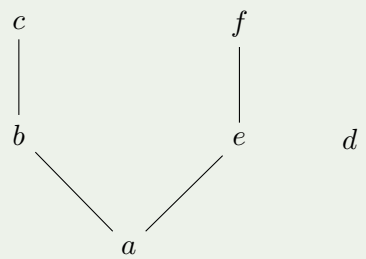
The propositional formula in the predicate calculus that expresses this property requires a constant $p$ to be a maximal element for a partial order, a binary predicate symbol $R$ for the partial order, and a binary relation $E$ for equality. It looks as follows.

$$\forall x. \, (R(p, x) \to E(p, x))$$

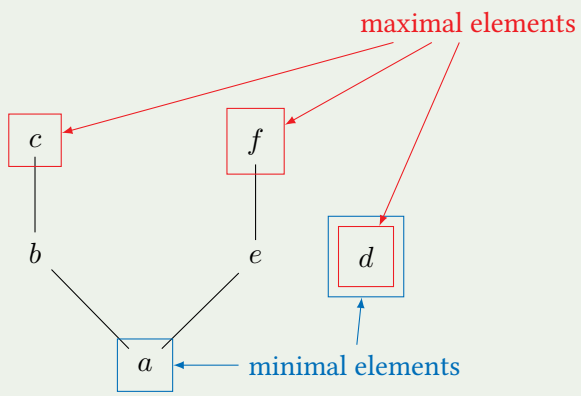
The way to understand this definition is to note that for a maximal element $p$ of a poset there cannot be another element above it (because any element above has to be equal to $p$). Analogously, a minimal element cannot have another element below it.

Note that a poset can have more than one maximal or minimal element, see the following example.

**Example 7.69.** If we go back to our running example from above,



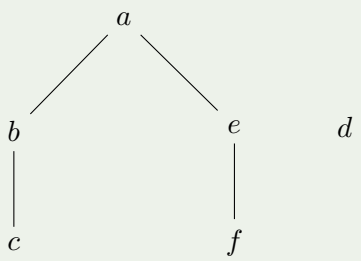

we can see that the maximal elements are $c$, $f$ and $d$, and that there are two minimal elements, namely $a$ and $d$.



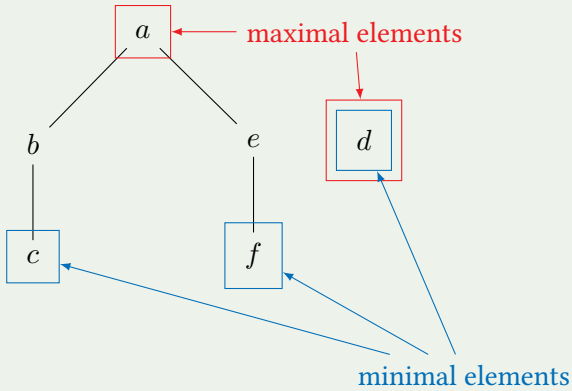

This example illustrates that a maximal element does not have to be above all the other

elements (for example, $f$ is maximal but not above $c$, $b$ or $d$, and $d$ is maximal but not above anything other than itself).

**Example 7.70.** By Exercise 183 we may turn our given poset upside-down to obtain another poset:




In this poset there are three minimal elements, $c$ and $f$ and $d$, and two maximal elements, $a$ and $d$.




It is immediately clear from the definition that when we turn a poset upside-down then maximal elements become minimal, and vice versa.

We look at another concept which is more restricted than being minimal or maximal.

**Definition 64: greatest/least**

An element $p$ is the[37] **greatest element of a poset** $(P, \leq)$ if and only if it is the case that

$$\text{for all } p' \in P \qquad \text{we have} \qquad p' \leq p.$$

The greatest element is usually called[38] $\top$.

An element $p$ is the[37] **least element of a poset** $(P, \leq)$ if and only if it is the case that

$$\text{for all } p' \in P \qquad \text{we have} \qquad p \leq p'.$$

The least element is usually called[39] $\bot$.

[37] The use of the definite article here is justified by the following exercise.
[38] Yet another usage of this symbol! It is shaped like a $t$, short for 'top', and that is the name of the symbol.
[39] Yet another usage of this symbol! It is shaped like the opposite of $\top$. It is known as 'bottom'.

**Takeaway**

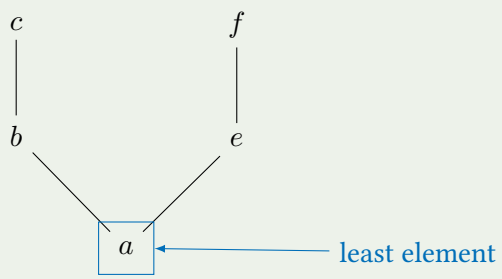
A greatest element is greater than or equal to every element of the poset, and a least element is less than or equal to every element of the poset.

We sometimes use **smallest element** for the least element of a poset, and **largest element** for the greatest element. This usage matches how we typically talk about things that are ordered. Note that, for example, the reflexive closure of a binary relation $R$, see 368, which is the smallest relation containing $R$ which is also reflexive, is the least element of the poset of all binary relations containing $R$ which are reflexive. Similar considerations apply to the symmetric, and the transitive, closure of a relation.

Using the same convention as before the proposition corresponding to a parameter $p$ describing a largest element for a partial order given by a binary predicate symbol $R$ is

$$\forall x.\, R(x, p).$$

**Example 7.71.** Our original example poset has no greatest or least element. If we remove the element $d$ from the set we obtain the following.




This poset has no greatest element; a least element exists, namely $a$. Note that this is also a minimal element, and that there is only one minimal element. See the following exercises for this and other important properties of least and greatest elements.

**Example 7.72.** Consider the set $\mathbb{N} \setminus \{0\}$ where

$$m \leq n \qquad \text{if and only if} \qquad m \text{ divides } n,$$

compare Example 7.60.

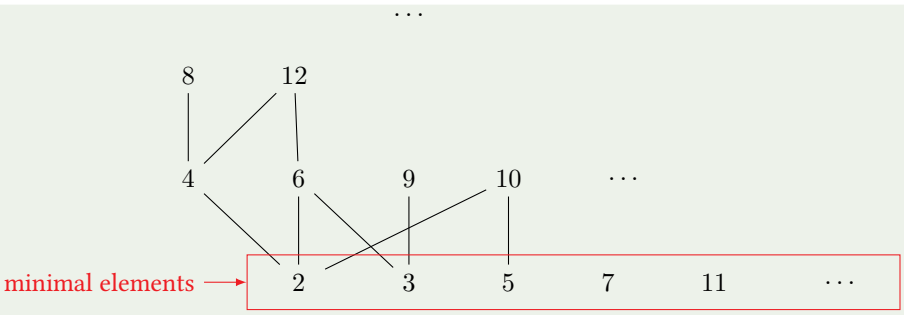
Since 1 divides every number it is the least element. This means that it is the only minimal element.

Since no number is divided by all natural numbers there is no largest element. But there are no maximal elements either: For every element $n \in \mathbb{N} \setminus \{0\}$ we have that $n$ divides $2n$, and so for any given $n \in \mathbb{N} \setminus \{0\}$ we can always find a larger element.

**Example 7.73.** We take the previous example but restrict to the set $\mathbb{N} \setminus \{0, 1\}$ where we again define

$$m \leq n \qquad \text{if and only if} \qquad m \text{ divides } n,$$

compare Example 7.60. Compared to that example we have removed the number 1 from consideration.




There is no number in $\mathbb{N} \setminus \{0, 1\}$ that divides all numbers, so there is no least element. There are a lot of minimal elements however: In order for an element $n$ to be minimal we must have that $m \leq n$ implies $m = n$; in other words, we are are looking for numbers $n$ with the property that for all $m \in \mathbb{N} \setminus \{0, 1\}$,

$$m \text{ divides } n \qquad \text{implies} \qquad m = n.$$

This is the case exactly when $n$ is a prime number, and so all prime numbers are minimal.

Since no number is divided by all natural numbers there is no largest element. But there are no maximal elements either: As in the previous example we have that for every element $n \in \mathbb{N} \setminus \{0\}$ we have that $n$ divides $2n$, and so we can always find a larger element.

**Example 7.74.** For the partial order on $\mathsf{Lists}_S$ from Example 7.68 we have a least element, namely the empty list $[\,]$. There are no maximal elements since given an arbitrary list we can always make it bigger by adding another element.

**CExercise 184.** For the following posets, try to draw a Hasse diagram and determine any maximal, minimal, least and greatest elements.

(a) The real numbers with the usual order.

(b) The natural numbers with the usual order.

(c) The negative integers with the usual order.

(d) For the partial order on $\mathsf{FBTrees}_S$ from Example 7.67.

(e) The powerset of a set $X$ with subset inclusion as the order.

(f) The set $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ of functions from $\mathbb{N}$ to $\mathbb{N}$ with the pointwise order, given by:

$$f \leq g \qquad \text{if and only if} \qquad \text{for all } n \in \mathbb{N} \text{ we have } fn \leq gn.$$

(g) The following relation on $\{a, b, c, d, e\}$.

$$\{(a, a), (b, b), (c, c), (d, d), (e, e)\}.$$

(h) The reflexive closure of the following relation on $\{a, b, c, d, e\}$:

$$\{(a, b), (c, d)\}.$$

(i) The reflexive transitive closure of the following relation on that set.

$$\{(a,b),(a,c),(b,d),(b,e),(c,d),(c,e)$$

(j) The reflexive transitive closure of the following relation on that set.

$$\{(a,b),(b,c),(c,d),(d,e)\}.$$

(k) For your programme of study, and the set of compulsory COMP units on that programme (through years 1 to 3), the partial order which is the transitive reflexive closure of the 'is a prerequisite of' relation (ignoring corequisites).

(l) For Chapter 6, take all operations defined in that chapter (including exercises) for $\mathbb{N}$ and for lists, and use the transitive reflexive closure of the 'is required to define' relation (for example, the addition operation is required to define multiplication of natural numbers).

**EExercise 185.** Greatest and least elements have special properties.

(a) Show that there is at most one greatest, and at most one least, element in each poset.

(b) Show that if $p$ is the greatest element of a poset then it is maximal. Similarly, show that if $p$ is the least element of a poset then it is minimal.

(c) Show that if a poset has a greatest element then it has exactly one maximal element. Similarly show that if a poset has a least element then it has exactly one minimal element.

(d) Show that if $(P,\leq)$ is a totally ordered poset then every maximal element is a greatest element, and every minimal element is a least element. Conclude that a totally ordered set can have at most one maximal, and one minimal, element.

### 7.5.3 Upper and lower bounds

When looking at posets we often care about whether a collection of elements can be safely 'overestimated' (or 'underestimated') by using a single element. For example, when we are looking at the complexity of a problem we may be interested in the fact that it is 'as most as complicated as some function $f$'—that means that there is an algorithm with complexity function less than or equal to $f$. Or maybe we have a group of algorithms or programs we would like to discuss in one go, and we can state that the complexity function for each of these lies below some 'upper bound' given by some function $f$.

**Definition 65: upper/lower bound**

Let $S$ be a subset of a poset $(P,\leq)$. We say that an element $p$ of[40] $P$ is an **upper bound for** $S$ if and only if it is the case that

$$\text{for all } p' \in S \qquad \text{we have} \qquad p' \leq p.$$

We say that an element $p$ of[41] $P$ is a **lower bound for** $S$ if and only if

$$\text{for all } p' \in S \qquad \text{we have} \qquad p \leq p'.$$

[40] Note that $p$ need not be an element of $S$.
[41] Again note that $p$ need not be an element of $S$.

**Takeaway**

An upper bound for a set is greater than or equal to every element of that set.

Again we give a first order proposition to describe this definition. Assume we have

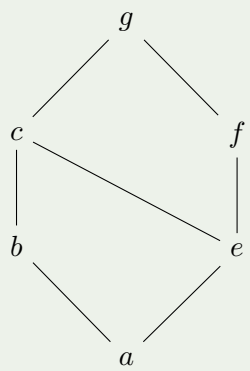
- a binary predicate symbol $R$ for the partial order,
- a unary predicate $S$ that describes the elements of the set $S$, so $S(x)$ is interpreted as 1 if and only if $x$ is in $S$,

then the proposition establishing $p$ as an upper bound of $S$ is

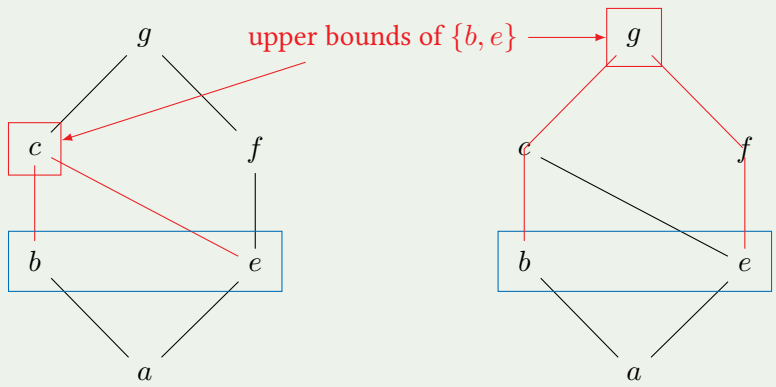
$$\forall y.\, (S(y) \to R(y, p)).$$

The example poset we have been using so far is too simple to contain many interesting upper or lower bounds. For the version of the example poset that appears in Example 7.71, the only interesting observation is that $a$ is the only lower bound for the sets $\{c, f\}$, $\{b, f\}$, $\{c, e\}$ and $\{b, e\}$.

**Example 7.75.** Consider the poset drawn below.

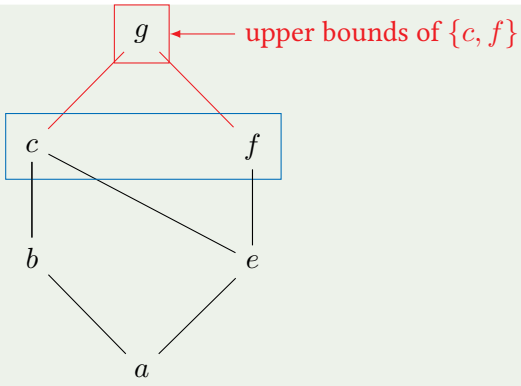



What are the upper bounds for the set $\{b, e\}$? They are the elements which are above both, $b$ and $e$.




We can see that the upper bounds are $c$ and $g$.

What are the upper bounds of $\{c, f\}$?




The element $g$ is the only upper bound of the sets $\{c, f\}$ (and also of $\{b, f\}$).

**Example 7.76.** In Java we can order classes by the subclass relation. For this partial order two classes have a common upper bound if and only if there is a class which is a superclass for both.

**Exercise 186.** For the poset from Example 7.75, find all lower bounds for the sets $\{b, e\}$, $\{g, b\}$, $\{g, e\}$, $\{c, f\}$, $\{b, f\}$.

Sometimes we can find a 'best' upper bound for a subset of a poset, that is, amongst all the upper bounds there may be a least one.

**Definition 66: least upper/greatest lower bound**

Let $S$ be a subset of the poset $(P, \leq)$. We say that an element $p$ of $P$ is the[42] **least upper bound (or supremum) for** $S$ if and only if

- $p$ is an upper bound for $S$ and
- if $p'$ is an upper bound for $S$ then $p \leq p'$.

The least upper bound of two elements $p$ and $p'$ is often written as $p \vee p'$.

We say that an element $p$ of $P$ is the[42] **greatest lower bound (or infimum) for** $S$ if and only if

- $p$ is a lower bound for $S$ and
- if $p'$ is a lower bound for $S$ then $p' \leq p$.

The greatest lower bound of two elements $p$ and $p'$ is often written as $p \wedge p'$.

**Takeaway**

A least upper bound for a set is greater than or equal to every element of that set, and it's the least element with that property.

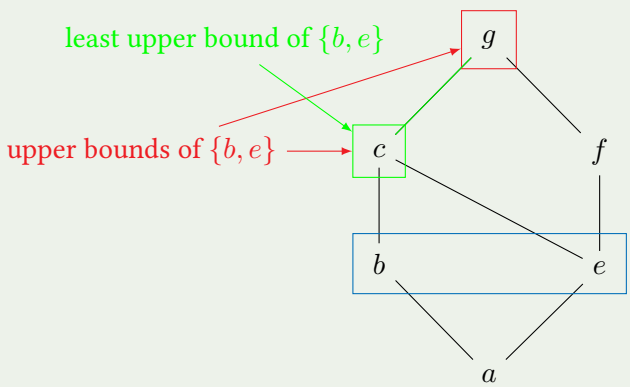
[42] The definite article is justified by Exercise 188.

This statement is fairly complicated if we look at the number of quantifiers involved when we unravel the statements from above. In an appropriate formal system[43] for $p$ to be the least upper bound of $S$ we need

$$\begin{aligned} &\forall y.\,(S(y) \to R(y,p)) \\ \wedge \qquad &\forall y.\,((\forall z.S(z) \to R(z,y)) \to R(p,y)). \end{aligned}$$

You can see that by building up the definitions slowly we have a final definition which looks simpler. But taking it apart reveals that the concept is quite complicated, as can be seen from looking at the corresponding proposition above.

**Example 7.77.** Going back to the example from above: The set $\{b, e\}$ has two upper bounds, namely $c$ and $g$. This set has a least element, namely $c$, and so $c$ is the least upper bound of $\{b, e\}$.




An alternative way of looking at least upper bonds is the following: We can form a new set from $S$, namely

$$\{p' \in P \mid p' \text{ upper bound for } S\},$$

and then

$$p \qquad \text{is a least upper bound for} \qquad S$$

if and only

$$p \qquad \text{is the least element of} \qquad \{p' \in P \mid p' \text{ upper bound for } S\}.$$

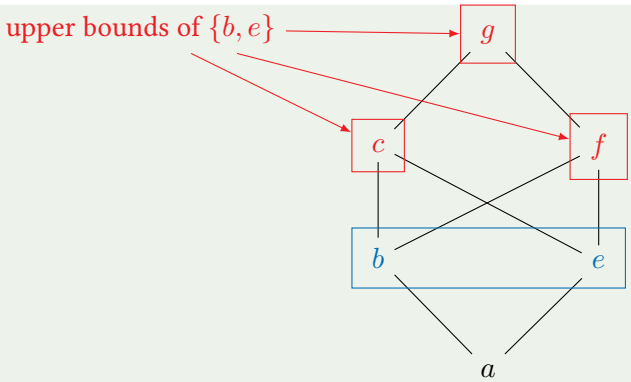
In particular,

$$\{p' \in P \mid p' \text{ upper bound for } S\}$$

must have a least element for $S$ to have a least upper bound.

**Example 7.78.** If, on the other hand, we change our underlying poset slightly to

[43] We need a one-placed predicate symbol for 'is an element of $S$' and a binary predicate symbol $R$ for the partial order.




then $c$, $f$ and $g$ are all upper bounds of $\{b, e\}$, but none of them is the least upper bound since the set

$$\{c, f, g\}$$

has no least element.

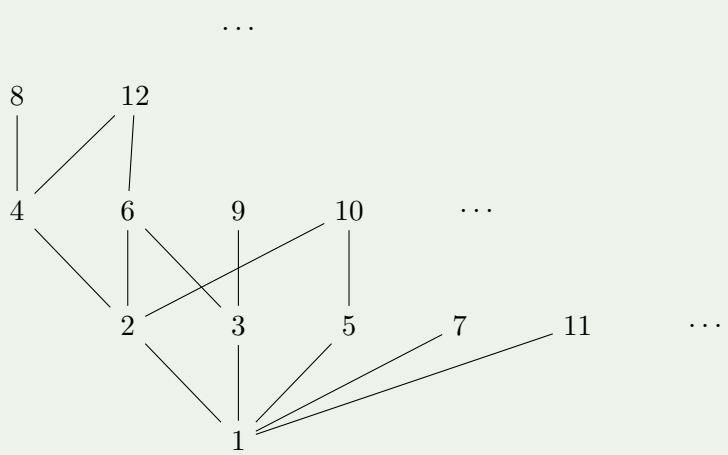
Note that the least upper bound of a set may be an element of that set. In the preceding Example 7.78 the least upper bound of $\{c, f, g\}$ is $g$. In the natural numbers with the usual order, the least upper bound of $\{2, 4, 8, 16\}$ is 16. Similar remarks apply to the greatest lower bound.

**Exercise 187.** Let $(P, \leq)$ be a partial order, and let $S$ be a subset of $P$. Show that if $S$ has a greatest element then it is the least upper bound of $S$.

**Example 7.79.** We return to another of our running examples. Consider once again the set $\mathbb{N} \setminus \{0\}$ with the partial order given by

$$m \leq n \qquad \text{if and only if} \qquad m \text{ divides } n,$$

compare Examples 7.60 and 7.72. We repeat the Hasse diagram.




If we have two numbers $m$ and $n$ then their lower bounds are the numbers which are below both of them, that is, all those numbers which divide both, $m$ and $n$. For example, for

$$12 \qquad \text{and} \qquad 18$$

these numbers are

$$\{1, 2, 3, 6\}.$$

The set of common divisors of two numbers $m$ and $n$ always has a largest element for the order under consideration (and this is also the largest element when we consider the usual order on $\mathbb{N}$, namely the *greatest common divisor* of $m$ and $n$, compare Example 6.44. Hence for every two elements there is a greatest lower bound, given by their greatest common divisor.

Similarly, given two numbers $m$ and $n$, their upper bounds are those numbers which have both of them as divisors, that is, all the *common multiples* of the two. For example, for

$$6 \qquad \text{and} \qquad 4$$

these are all the numbers that are multiples of both,[44] 4 and 3.

$$\{12,\ 24,\ 36,\ \ldots\}$$

which means they are all the multiples of 12,

$$\{12n \in \mathbb{N} \mid n \in \mathbb{N}\}.$$

This set always has a smallest element, in this case 12. In general, the least upper bound of two numbers $m$ and $n$ is *their smallest common multiple.*

**Example 7.80.** If we return to the partial order on the set $\mathsf{Lists}_S$ of lists over a set $S$ given in Example 7.68 we note the following:

Given any two lists, their greatest lower bound is the longest list they both have in common, so for

$$[4, 3, 2, 1] \qquad \text{and} \qquad [5, 4, 2, 1],$$

the greatest lower bound is

$$[2, 1].$$

Once those two elements have been added to the list the paths of getting to the two lists diverge. Note that there is always such a largest list (it might be the empty list), so for every pair of element a greatest lower bound exists. If you look at the Hasse diagram for this poset, which is drawn in Example 7.68 you can see that this looks like a tree. To find the greatest lower bound for two lists, you go down the branches of the tree which contain them until these branches come together.

On the other hand if we look at least upper bounds for two lists we find that they almost never exist. This is because two lists which are not related by the order do not have any upper bounds. For example, an upper bound for both

$$[2, 1] \qquad \text{and} \qquad [3, 1]$$

would have to be obtainable by adding elements to both these lists, and clearly this cannot happen. Hence the only case in which any upper bounds exist is in the case where

$$l \leq l',$$

[44] You may want to convince yourself that all numbers that are multiples of both, 4 and 3, are also multiples of both, 6 and 4.

in which case $l'$ is the least upper bound for the two.

**Exercise 188.** Show that every subset of a poset $P$ has at most one least upper, or greatest lower, bound in $P$.

So least upper (greatest lower) bounds of sets exist, or don't, but where they exist they are unique.

**CExercise 189.** This exercise is concerned with calculating greatest lower and least upper bounds. Note that all the settings described here also appear in CExercise 179 and that you may want to solve the corresponding part of that exercise first.

(a) For the natural numbers with the usual order, how would you describe the greatest lower, and least upper, bounds for sets of the form $\{m, n\}$? Can you extend this to all finite subsets of $\mathbb{N}$? Do you need finiteness? Try to extend your result to all subsets of $\mathbb{N}$ as far as possible.

(b) For the powerset of the set $\{0, 1, 2\}$ how does one calculate the greatest lower, and least upper, bound of a subset? *Hint: You may want to draw the Hasse diagram. What are the greatest lower and least upper bounds of* $\{\{0, 1\}, \{1, 2\}\}$

(c) Consider the powerset $\mathcal{P}X$ of a set $X$, with subset inclusion providing a partial order. Show that for every subset $\mathcal{S}$ of $\mathcal{P}X$ we have

- The least upper bound of $\mathcal{S}$ exists and is given by

$$\bigcup \mathcal{S} = \bigcup \{S \subseteq X \mid S \in \mathcal{S}\} = \{x \in X \mid \exists S \in \mathcal{S}.\, x \in S\}.$$

- The greatest lower bound $\mathcal{S}$ exists and is given by

$$\bigcap \mathcal{S} = \bigcap \{S \subseteq X \mid S \in \mathcal{S}\} = \{x \in X \mid \forall S \in \mathcal{S}.\, x \in S\}.$$

(d) Consider the set of strings made from symbols from a set $S$. Consider the partial order where a string is less than or equal to another if it is a prefix of the latter. What are the greatest lower, and least upper, bounds of sets of strings (where they exist)? *Hint: You have already seen a partial Hasse diagram for this set. Start from there by looking at the two-element subsets.*

(e) Consider the set $\mathsf{FBTrees}_S$ of full binary trees over a set $S$ with the partial order from Example 7.67. When do the least upper, and greatest lower, bounds of two trees exist and what are they? For this part it is sufficient to describe your answer in English and to give an example.

**Exercise 190.** Consider the following relation for the set $\mathsf{FBTrees}_{\mathbb{N}}$ of full binary trees over the set of labels $\mathbb{N}$. Recall the function $k_1$ from CExercise 125 which maps a natural number to the number 1. Also recall the $\mathrm{map}$ function from the same exercise, which has the property that

$$(\mathrm{map}\, k_1)t$$

replaces every label in the tree $t$ by the number 1. Finally recall the partial order $\leq$ on $\mathsf{FBTrees}_S$ described in Example 7.67. We use this to define a new partial order on the same

set.

For two trees $t$ and $t'$ we set

$$t \sqsubseteq t'$$

if and only if

$$(\operatorname{map} k_1)t \leq (\operatorname{map} k_1)t'.$$

(a) Try to describe in English under which circumstances we have that $t \sqsubseteq t'$.

(b) Modify the code for the lessthan method from Example 7.67 in such a way that it implements this new relation.

(c) Is this relation a partial order?

(d) For the set $S = \{1\}$, consider the partial order $\leq$ on $\mathsf{FBTrees}_S$ used above. Describe the greatest lower bound of two elements of this set in English.

(e) For the same poset as in the previous part describe the least upper bound of two elements in English.

**EExercise 191.** Consider the following relation on the set of partial functions from a set $S$ to a set $T$:

$$f \preccurlyeq g \qquad \text{if and only if} \qquad \operatorname{dom} f \subseteq \operatorname{dom} g \text{ and } s \in \operatorname{dom} f \text{ implies } fs = gs$$

(a) Show that this relation is a partial order.

(b) What are the minimal elements of this poset? Is there a least element?

(c) What are the maximal elements of this poset? Is there a greatest element?

(d) Given two partial functions $f$ and $g$ from our set, under which circumstances is their greatest lower bound $f \wedge g$ defined? Can you describe this partial function?

(e) Given two partial functions $f$ and $g$ from our set, under which circumstances is their least upper bound $f \vee g$ defined? Can you describe this partial function? *Hint: Think about the intersection of the domains of definition of $f$ and $g$, and any properties $f$ and $g$ have to have for inputs from this set, in order to for their least upper bound to exist.*

The mathematical discipline of *order theory* is concerned with studying partially ordered sets and their properties.

# Chapter 8

# Applications to Computer Science

The eventual aim of this chapter is to collect examples and exercises from computer science where mathematical techniques are routinely employed, but without stating explicitly which areas of mathematics might be involved. Currently a number of suitable exercises are located in various chapters. While they serve there to motivate some concepts and techniques, from a didactic point of view it would be good if students gained some experience in solving such problems without too many hints regarding what techniques to employ.

Examples are:

- The use of precise language, and the predicate calculus, in particular first order logic, to describe specifications for a computer program (some examples appear below).
- To verify the correctness of programs and protocols using techniques from logic, see for example COMP31111.
- Programming with logical formulae in the language Prolog which is taught in COMP24412.
- Solving optimization problems with tools from logic in COMP21111.
- The use of the expected value of a random variable to calculate the complexity of a given algorithm.
- The use of probabilities and expected values to evaluate risks.
- The notion of one function growing at most as fast as another to talk about complexity classes of problems or algorithms—this material is covered in COMP11212, COMP26120 and COMP36111.
- The use of the notion of the size of a set to answer questions from computer science, for example: Are there functions from $\mathbb{N}$ to $\mathbb{N}$ which cannot be implemented using a computer?
- The use of recursion to solve particular problems. Various aspects of this idea are covered in COMP16212, COMP26120 and COMP36111.
- The use of recursively defined function to work out the complexity of a particular algorithm using recurrence relations.
- The use of induction as a technique to prove properties of many entities that appear in various course units.
- Using relations to express properties of programming constructs, or of elements of some other domain of discourse.

- The use of vectors and matrices in particular in the context of graphics, which appears in COMP27112.

For the moment only a small number of exercises appear in this chapter. They are all concerned with analysing a particular problem with a view to creating a precise specification for the desired solution.

Various exercises that appear in other chapters in these notes could be included in this chapter, but I did not want to postpone your tackling them until the end of term.

- **Chapter 2**. Various questions are concerned with the properties of Java operations, and with the properties of functions that assign user names.
- **Chapter 4**. Bayesian updating is a technique from Machine Learning and is covered in CExercise 79 and Exercises 80 and 81. Using expected values to calculate average complexities for algorithms is also a computer science topic, which is the topic of EExercise 98. We look a little bit at the reliability of hardware in EExercise 90.
- **Chapter 6**. All the exercises asking you to produce code are applications of the mathematical idea of recursion to programming. Proofs by induction that a given recursively defined function has certain properties can be transplanted to recursive programs, and are required if you want to show that your code behaves as expected/specified.
- **Chapter 7**. We look at algorithms that calculate the powers of some number in modular arithmetic in EExercise 164. There are also exercises asking you to produce code for some problems in this chapter.

Much of the time programmers do not make the effort to precisely define what they want their code to do, but under some circumstances this is a very useful tool, for example

- when code already written shows unexpected behaviour or
- when one is trying to define the behaviour of a compiler of a programming language[1] or
- for safety-critical applications, such as the control of a space or aircraft, when one would ideally have verification that the code implemented has particular properties.[2]

**CExercise 192.** Imagine you are a programmer tasked with writing a program that takes as its input an array of natural numbers, and returns a sorted version of that array. The aim of this exercise is to specify what exactly this means.

For example, you would expect for the input array $a$

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| 9 | 35 | 0 | 5 | 58 | 7 | 1 | 42 |

to produce the following output array, say $b$:

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 5 | 7 | 9 | 35 | 42 | 58 |

For the conditions (a) and (b) below, do the following:

- Write out as precise a description as possible using the English language.

[1] You may be surprised by this, but a number of programming languages do not have precise definitions of their behaviour and different compilers (or interpreters) can behave differently for some programs.

[2] A unit that looks at this issue is COMP31111, *Verified Development*.

- Write a formula in first order logic that captures the required property. You may assume the following:
  - The domain of interpretation is the set of natural numbers $\mathbb{N}$.
  - You have (unary) function symbols $a$ and $b$ to describe the input and output arrays respectively.
  - You have a parameter $n$ that describes the final index of the input array (so the size of the input array is $n + 1$, with indices from 0 to $n$).
  - You have a binary 'less than or equal' predicate $L$ so that the interpretation of $L(x, y)$ is $Ix \leq Iy$, where $I$ is the interpretation function.
  - You have a binary equality predicate $E$ so that the interpretation of $E(x, y)$ is $Ix = Iy$, where again $I$ is the interpretation function.

(a) The output array $b$ is sorted.

(b) The array $b$ contains exactly the numbers from array $a$. You may assume that all the entries in $a$ are different. *Hint: Split the property into two parts.*

(c) Does your statement from part b) still work if $a$ may contain duplicate entries? Check whether you can construct an example of arrays $a$ and $b$ which satisfy your condition, but where $b$ has fewer elements than $a$.

(d) Can you see how to improve your previous so that it also works for arrays with duplicate entries? Can you transfer your condition to first order predicate logic? If not, could this be dealt with by expanding the system?

**Exercise 193.** Assume you are given an array of natural numbers. You are tasked with writing a program that finds the *median* of all these numbers, that is the number you would find at the halfway point of the array if it were sorted. You may think of this as the 'middle number' of all those present.[3]

(a) Write down the properties required to ensure that the number your program finds is indeed the desired median. You may assume that all the numbers in your array are different. *Hint: You may want to use the function $|\cdot|$ that maps a finite set to the number of elements of that set.*

(b) Now amend your properties so that it can also cope with the case where the same number may appear more than once in the array. *Hint: Consider the set you used in the previous part. Can you think of changing that to a different set that will give you the desired property?*

**Exercise 194.** Assume you are tasked with writing a program that constructs an individual timetable for each student (better than the one on the Student System) and which therefore needs to know about all the undergraduate students within the School. This means you will require some way of representing each student in your program.

(a) You have to define a function whose source is the set of all undergraduate students in

[3] The median income of all the employees of a company, for example, is the one with the property that half the employees earn as most as much, and half the employees earn at least as much.

the school, and whose target is something that your programme knows about. What would a suitable target for such a function be?

(b) Which properties should your function have?

(c) Define the function that you would use.

(d) Justify your choice of function by arguing that it satisfies the properties stipulated in Part (b).

When when we have a recursively defined procedure we can create a *recurrence relation* (compare Examples 6.50 and 6.51 and cExercise 146) that describes its behaviour.

**Example 8.1.** Consider the function that sums all the elements from a list of numbers, given in Example 6.12.

**Base case** sum. $\quad \mathrm{sum}\,[\,] = 0.$[4]

**Step case** sum. $\quad \mathrm{sum}(s : l) = s + \mathrm{sum}\, l.$

We want to count the number of additions the function has to carry out for a list with $n$ elements, which defines a function

$$f : \mathbb{N} \longrightarrow \mathbb{N}.$$

We can read off that

$$f0 = 0,$$

since in the base case, where the list has 0 elements, no additions are required. The step case shows us that

$$f(n+1) = 1 + fn.$$

Hence we have found a recurrence relation as studied in Section 6.4.5. Using the methods from that chapter we can show that the function we have defined is the assignment

$$n \longmapsto n,$$

that is, our function is the identity function on $\mathbb{N}$.

**Exercise 195.** For the following algorithms, create recursive function definitions similar to those from Examples 6.50 and 6.51 and cExercise 146. Is it possible to solve them in the same way as those examples? If not, why not?

(a) The function that calculates the factorial of a number from Example 6.43. Count the number of multiplications the function has to carry out for a given input.

(b) The function that reverses a list from Example 6.13. For $n$ use the number of elements in the list and count the number of calls the function makes to itself.

[4] If you were wondering how to deal with the empty list then note that adding no elements at all is usually taken to describe the number 0. More generally, for an operation with unit $e$, applying the operation to 0 many elements should return $e$.

(c) The binary search algorithm from Example 4.99. Set $n$ to be the size of the array and count the number of array look-ups required.

# Glossary[5]

**anti-symmetric** 416

A binary relation $R$ on a set $S$ is anti-symmetric if $(s, s')$ and $(s', s)$ both being in the relation implies $s = s'$.

**binary operation** 446, 447, 451, 455

A function of the type $S \times S \longrightarrow S$, which takes two elements of a set $S$ as input and produces another element of $S$.

**binary relation** 383, 446, 449, 452, 454, 455

A connection between a source set $S$ and a target set $T$ which is not necessarily a function. It is specified by the collection of all pairs of the form $(s, t)$ in $S \times T$ that belong to it.

**binary tree with labels from a set $S$** 318, 452

This is a tree where each node has a label from $S$ and where each node has either 0, 1 or 2 children. Formally this is another recursively defined notion.

**coefficient** 411

The coefficients of a polynomial function are the numbers that appear as factors in front of a power of the variable.

**complex number** 446, 447, 450, 452, 454

Number of the form $a + ib$.

**composite of partial functions** 361

Similar to the composite of two functions, but the result is undefined if either of the two functions is not defined where required.

**degree of a polynomial** 411

The degree of a polynomial function is the largest index whose coefficient is unequal to 0.

**directed graph** 364

A set (of *nodes*) connected by edges; can be described using a binary relation on the set.

**discrete** 453

A random variable is discrete if and only if its range is countable.

[5]Note that page numbers for Chapters 1–4 may not match the printed notes for these chapters, and that for unknown reasons, hyperlinks in the glossary to entries in Chapters 6–8 are not working, but the page numbers given are correct for these.

**domain of definition** 361

For a partial function it is the set consisting of all those elements of the source set for which the partial function is defined.

**equivalence class with respect to $R$ generated by $s$, $[s]$** 384, 396

The set of all elements which are related to $s$ by the equivalence relation $R$.

**equivalence relation** 381

A binary relation on a set is an equivalence relation if it is reflexive, symmetric and transitive.

**equivalence relation generated by a binary relation $R$** 383

The transitive closure of the symmetric closure of the reflexive closure of $R$.

**expected value** 455

The expected value of a random variable can be thought of as the average value it takes. It is given by the integral of the product of a number which the probability that it is the value of the random variable. If the random variable is discrete then this is given by a sum.

**full binary tree with labels from a set $S$** 310

This is a tree where each node has a label from $S$ and where each node has either 0 or 2 children. Formally this is another recursively defined notion.

**function** 448–450, 452, 455

A function has a source and a target, and contains instructions to turn an element of the source set into an element of the target set. Where partial functions are discussed sometimes known as *total function*.

**greatest element, $\top$** 430, 451

An element of which is greater than or equal to every element of the given poset.

**greatest lower bound, infimum** 435

An element of a poset $(P, \leq)$ is a greatest lower bound of a given subset of that poset if it is both, a lower bound and greater than or equal to every lower bound of the given set.

**identity relation** 355

The identity relation on a set $S$ relates every element of $s$ to itself, and to nothing else.

**integer** 453, 454, 456

A whole number that may be positive or negative.

**integer division** 448, 451

Integer division is an operation on integers; given two integers $n$ and $m$ where $m \neq 0$, we get an integer quotient $n \operatorname{div} m$ and a remainder $n \bmod m$.

**largest element, $\top$** 431

An element which is greater than or equal to every element of the given poset; a synonym for greatest element.

**least element, $\perp$** 430, 454

An element which is less than or equal to every element of the given poset.

**least upper bound, supremum** 435

An element of a poset $(P, \leq)$ is a least upper bound of a given subset of that poset if it is both, a upper bound and less than or equal to every upper bound of the given set.

**list over a set $S$** 292

A list over a set $S$ is a recursively defined concept consisting of an ordered tuple of elements of the given set.

**lower bound** 433, 449

An element of a poset $(P, \leq)$ is a lower bound for a given subset of $P$ if it is less than or equal to every element of that set.

**maximal element** 429

An element of a poset which does not have any elements above it.

**measurable** 453

A function from the sample set of a probability space to the real numbers is measurable if and only if for every interval it is the case that the set of all outcomes mapped to that interval is an event.

**minimal element** 429

An element of a poset which does not have any elements below it.

$\mathbb{N}$ 330

The natural numbers as a set with a number of operations. This set and its operations are formally defined in Section 6.4.

**natural number** 452, 453

One of the 'counting numbers', 0, 1, 2, 3,....

**opposite relation of $R$, $R^{\mathsf{op}}$** 355

The relation consisting of those pairs $(t, s)$ for which $(s, t)$ is in $R$.

**ordered binary tree with labels from a set $S$** 320

A binary tree with labels from a set $S$ is ordered if the set $S$ is ordered, and if for every node, all the nodes in the left subtree have a label below that of the current node, while all the nodes in the right subtree have a label above.

**partial function** 360, 448

An assignment where every element of the source set is assigned at most one element of the target set; one may think of this as a function which is undefined for some of its inputs.

**partial order** 417, 453, 455

A binary relation on a set is a partial order provided it is reflexive, anti-symmetric and transitive.

**polynomial function** 447, 448

A function from numbers to numbers whose instruction is of the form $x$ is mapped to $\sum_{1i=1}^{n} a_i x^i$ (where the $a_i$ are from the appropriate set of numbers).

**poset** 417, 449, 451, 454, 455

A set with a partial order, also known as a *partially ordered set*.

**probability space** 446, 447, 450, 451, 453

A sample set together with a set of events and a probability distribution.

$\mathbb{Q}$ 411

The set of all rational numbers together with a variety of operations, formally defined in Definition 0.1.3.

**random variable** 447–450, 454, 455

A random variable is a measurable function from the set of outcomes of some probability space to the real numbers.

**rational number** 453

A number is rational if it can be written as the fraction of two integers. A formal definition is given on page 411 (and the preceding pages).

**real number** 453

We do not give a formal definition of the real numbers in this text.

**reflexive** 367

A binary relation on a set is reflexive if it relates each element of the set to itself.

**reflexive closure** 368, 449

The reflexive closure of a binary relation on a set $S$ is formed by adding all pairs of the form $(s, s)$ to the relation.

**relation** 353, 450, 452

A relation from a set $S$ to a set $T$ is given by a set of pairs with first component an element of $S$ and second component an element of $T$; in other words, it is a subset of $S \times T$.

**relational composite** 355

A generalization of composition for (partial) functions.

**smallest element, $\perp$** 431

An element which is less than or equal to every element of the given poset; a synonym for least element.

**string over a set $S$** 321

A formal word constructed by putting together symbols from $S$.

**symmetric closure** 370, 449

The symmetric closure of a binary relation on a set is formed by taking the union of the relation with its opposite.

**total order** 421

A total order is a partial order in which every two elements are comparable.

**transitive** 372

A binary relation on a set $S$ is transitive provided that $(s, s')$ and $(s', s'')$ being in the relation implies that $(s, s'')$ is in the relation.

**transitive closure** 374, 449

The transitive closure of a binary relation on a set is formed by adding all pairs of elements $(s_1, s_n)$ for which there is a list of elements $s_1$, $s_2$ to $s_n$ in $S$ such that $(s_i, s_{i+1})$ is in the relation.

**unit** 451

An element of a set is a unit for a binary operation on that set if and only if applying the operation to that, plus any of the other elements, returns that other element.

**upper bound** 433, 451

An element of a poset $(P, \leq)$ is an upper bound for a given subset of $P$ if it is greater than or equal to every element of that set.

$\mathbb{Z}$ 407, 409

The integers with various operations, see Definition 0.1.2 for a formal account.

# COMP11120, Semester 2

# Exercise Sheet 11

## For material from Week 1

## Core Exercises for this week

**CExercise 112** on page 250. Do any one part.

**CExercise 117** on page 304.

**CExercise 116** on page 303.

## Extensional Exercises for this week

**EExercise 108** on page 246. Do any one part.

**EExercise 119** on page 305. Do any three parts.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should now be ready to do all the exercises in Chapter 5 and Section 6.1.

# COMP11120, Semester 2

# Exercise Sheet 12

## For examples classes in Week 2

## Core Exercises for this week

**CExercise 122** on page 309. Do any two parts.

**CExercise 123** on page 317.

**CExercise 125** on page 318. Do (a) and (b) and any one of the remaining parts.

## Extensional Exercises for this week

**EExercise 126** on page 318.

**EExercise 130** on page 321.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should be able to do all exercises in Section 6.2.

# COMP11120, Semester 2

# Exercise Sheet 13

## For material from Week 3

## Core Exercises for this week

**CExercise 132** on page 323.

**CExercise 135** on page 335.

**CExercise 138** on page 336. Do part (a) and one of (c) or (d).

## Extensional Exercises for this week

**EExercise 133** on page 323.

**EExercise 141** on page 338. Do one part from (a)–(c) and one from (d)–(f)

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should be able to do all exercises up to and including Section 6.4.2 now.

# COMP11120, Semester 2

# Exercise Sheet 14

## For material from Week 4

## Core Exercises for this week

**CExercise 144** on page 344. Do any two parts.

**CExercise 146** on page 348. Do one part from (a)–(c) and one from (d)–(g).

**CExercise 151** on page 362.

## Extensional Exercises for this week

**EExercise 148** on page 350. Do any part.

**EExercise 152** on page 362.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should now be able to do Exercises up to Section 7.2.

# COMP11120, Semester 2

# Exercise Sheet 15

## For material from Week 5

## Core Exercises for this week

**CExercise 158** on page 379. Do one part from (a)–(b), one from (c)–(e) and one from (f)–(h).

**CExercise 160** on page 382. Do one part from (a)–(b), one from (c)–(e), one from (f)–(h) and one from (i)–(k).

**CExercise 163** on page 387. Do any one part.

## Extensional Exercises for this week

**EExercise 159** on page 380.

**EExercise 164** on page 395. Do all parts; it is sufficient to carry out two calculations each time instead of all four.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should now be ready to do all exercises up to and including Section 7.4.6.

# COMP11120, Semester 2

# Exercise Sheet 16

## For examples classes in Week 6

## Core Exercises for this week

**CExercise 165** on page 400. Do one from (a)–(c), one from (d)–(f) and one from (g)–(h).

**CExercise 166** on page 403.

**CExercise 179** on page 422. Do one part from (a)–(d),one from (e)–(h), one from (i)–(k) and one from (l)–(o).

## Extensional Exercises for this week

**EExercise 174** on page 414.

**EExercise 173** on page 413.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should now be ready to do all exercises up to and including Section 7.5.1.

# COMP11120, Semester 2

# Exercise Sheet 17

## For material from Week 7

## Core Exercises for this week

**CExercise 184** on page 432. Do one part from (a)–(c), one from (d)–(f), one from (g)–(j) and one from (k)–(l).

**CExercise 189** on page 439. Do one part from (a)–(c) and one from (d)–(e).

**CExercise 192** on page 442.

## Extensional Exercises for this week

**EExercise 185** on page 433. Do all parts, but in each part you only have to do one of the statements (for greatest/least, or maximal/minimal).

**EExercise 191** on page 440.

Remember that

- if you are stuck on an exercise move on to the next one after ten minutes, but write down why you got stuck, and consider coming back to that exercise again later;
- you may only use concepts which are defined in these notes (Chapter 0 establishes properties numbers, sets and functions), and for every concept you do use you should find the definition in the notes and work with that;
- you need to write down your working, not just give an answer;
- you should justify each step in your proofs;
- we won't penalize your mark if there are some mistakes in your answers.
- Please clearly mark which part of which question you are answering.

You should now be ready to do all exercises in the notes.